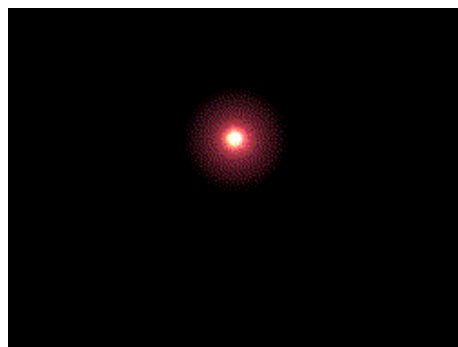


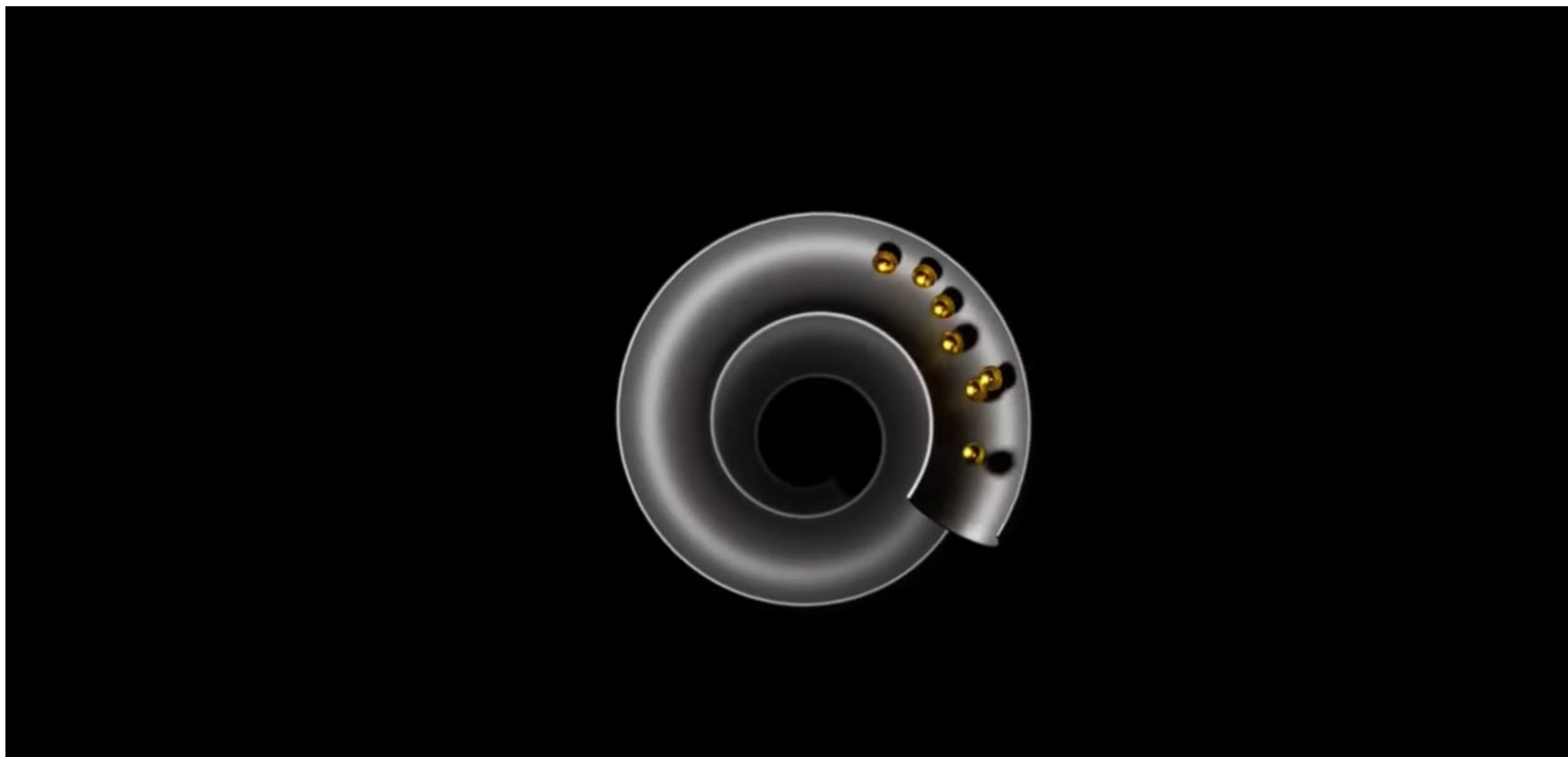
粒子系统(Particle Systems)



蒋浩

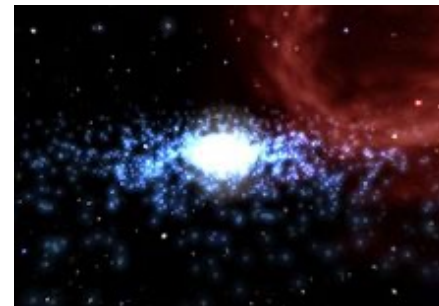
Email: jianghao@ict.ac.cn

粒子系统演示



粒子系统(Particle systems)

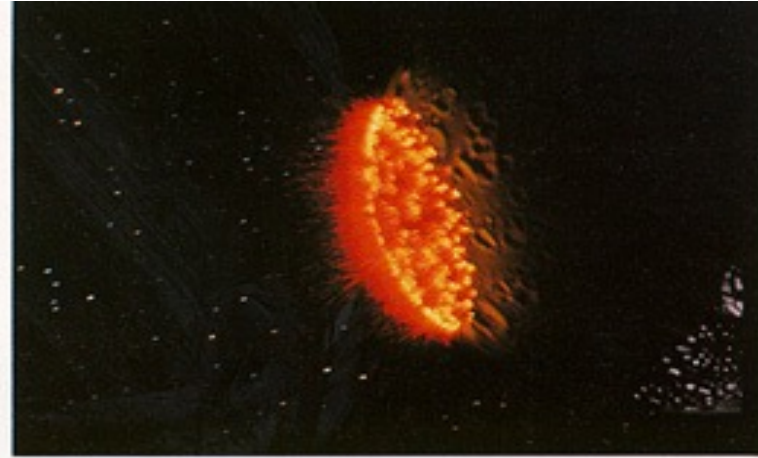
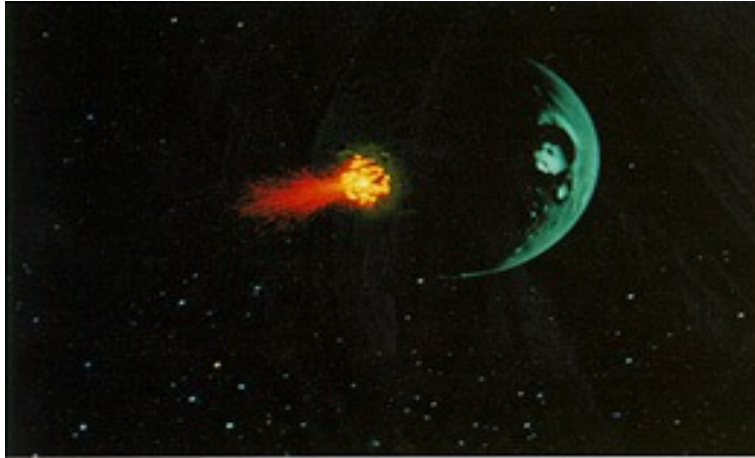
- 粒子系统是最实用的过程动画技术之一，是影视特技、游戏、图形学中生成**视觉特效**的一种**主要方法**。
- 这一方面的先驱是William T. Reeves，他在1983年所发表的Siggraph论文中成功地提出了一种模拟**不规则自然景物**("fuzzy" phenomena)生成和动画的系统，也就是所谓的粒子系统。



- 粒子系统采用了一套完全不同于以往的造型、绘制方法来构造、绘制景物，**造型和动画巧妙地连成一体**。景物被定义为由成千上万个不规则的、随机分布的粒子所组成，而每个粒子均有一定的生命周期，它们不断改变形状、不断运动。

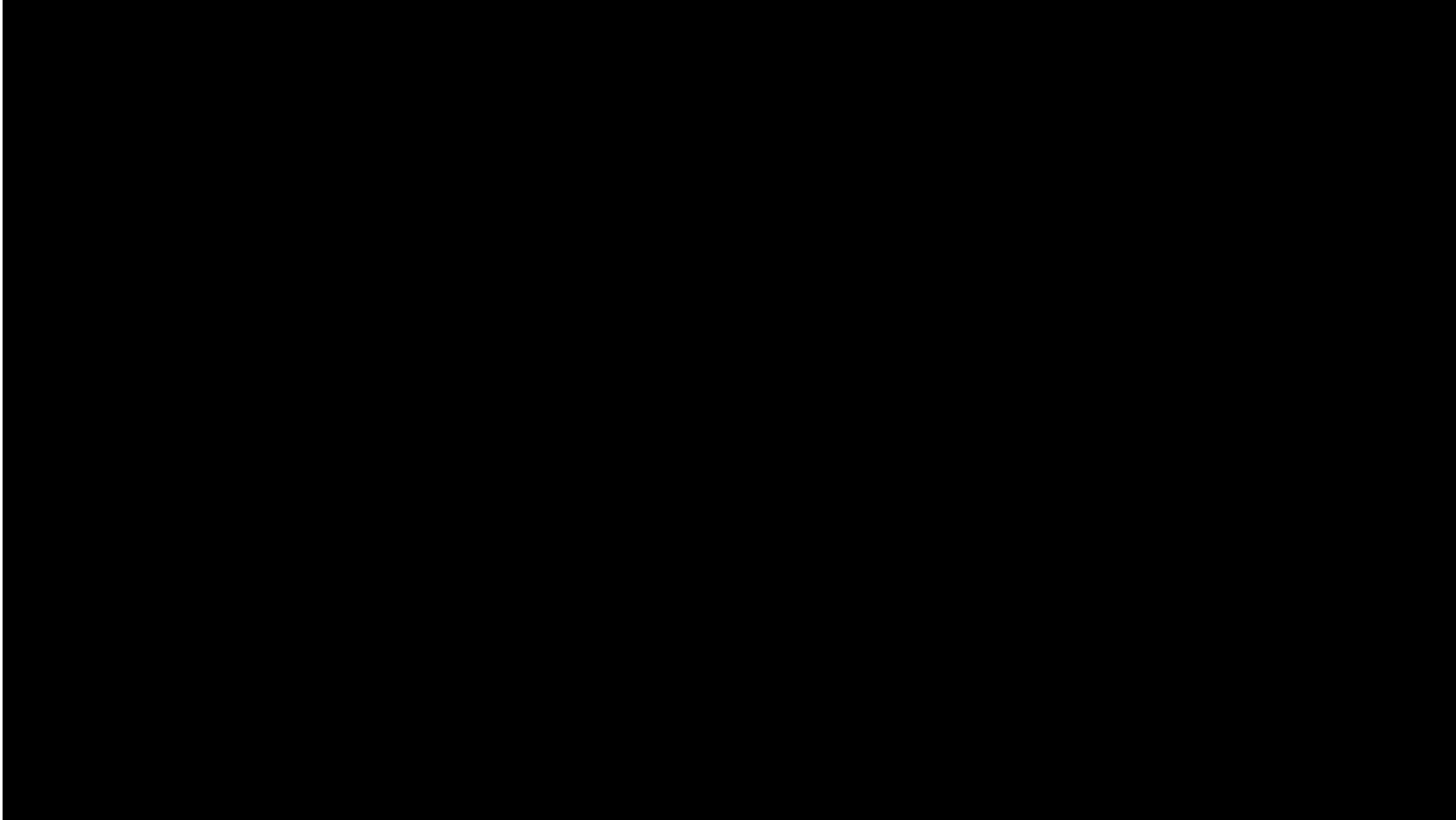
历史——星际迷航2: 可汗之怒, 1983

(Star Trek II - The Wrath of Khan)



历史——星际迷航2: 可汗之怒, 1983

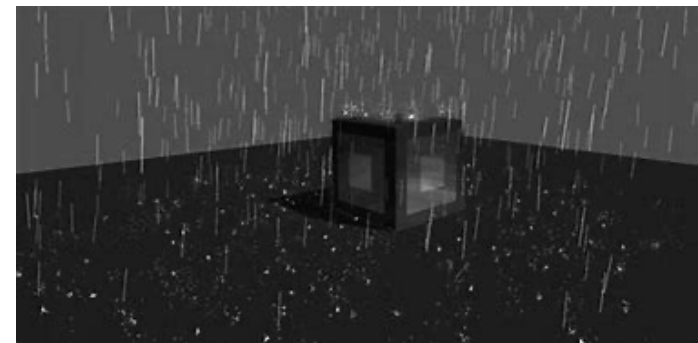
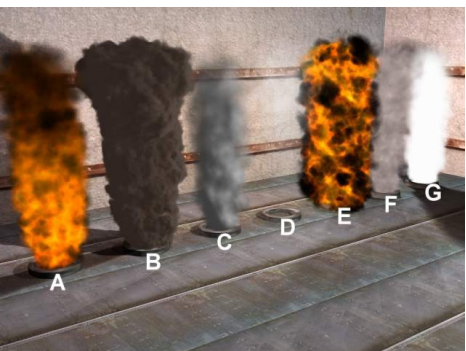
(Star Trek II - The Wrath of Khan)



01:37开始

应用极为广泛

- 云彩
- 烟雾
- 流动的水(瀑布、水花)
- 爆炸、碎片、尘土
- 火(火花)
- 喷泉
- 烟花
- 下雪、下雨、落叶
- 大群体
- 草、发丝，等等

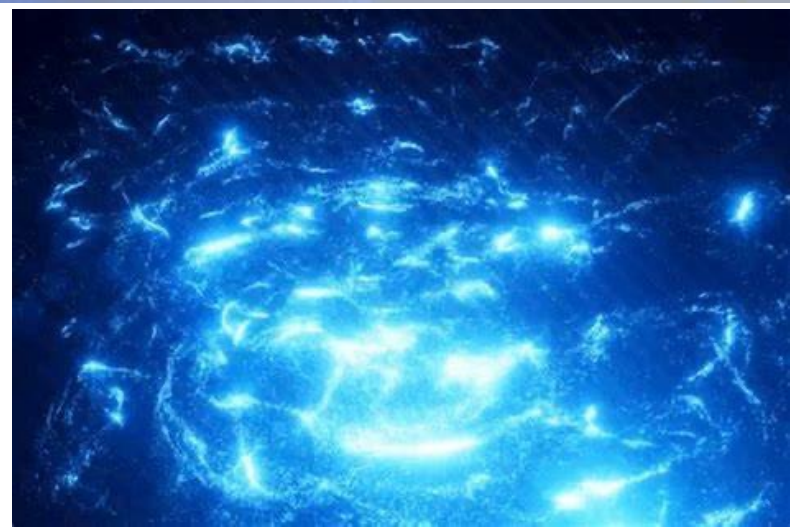
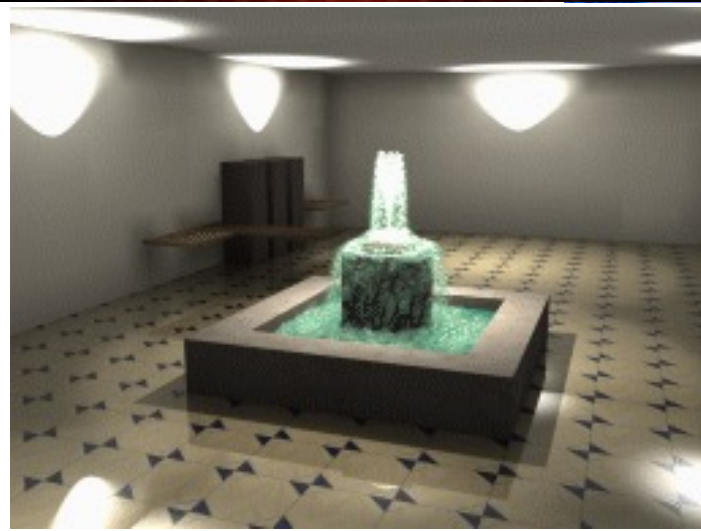




关于William T. Reeves(1959-)

- 参考文献
 - William T. Reeves: Particle Systems - a Technique for Modeling a Class of Fuzzy Objects. ACM Trans. Graph. 2(2): 91-108 (1983)
 - William T. Reeves, Ricki Balu: Approximate and probabilistic algorithms for shading and rendering structured particle systems. SIGGRAPH 1985: 313-322
- 截止到2023年10月15日，论文"Particle Systems" 被引用次数：**2769**次。
- 因《锡铁小兵Tin Toy》1988年获得第61届奥斯卡金像奖最佳动画短片(与John Lasseter) 。
 - Tin Toy (1988) (Producer/Technical Director/Modeler/Additional Animator)
 - Toy Story (1995) (Supervising Technical Director / Modeling / Animation System Development / Renderman Software Development)
 - A Bug's Life (1998) (Supervising Technical Director)
 - Finding Nemo (2003) (Lead Technical Development)

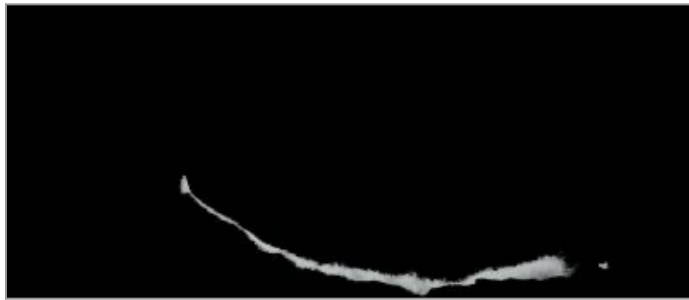
应用例子



应用例子



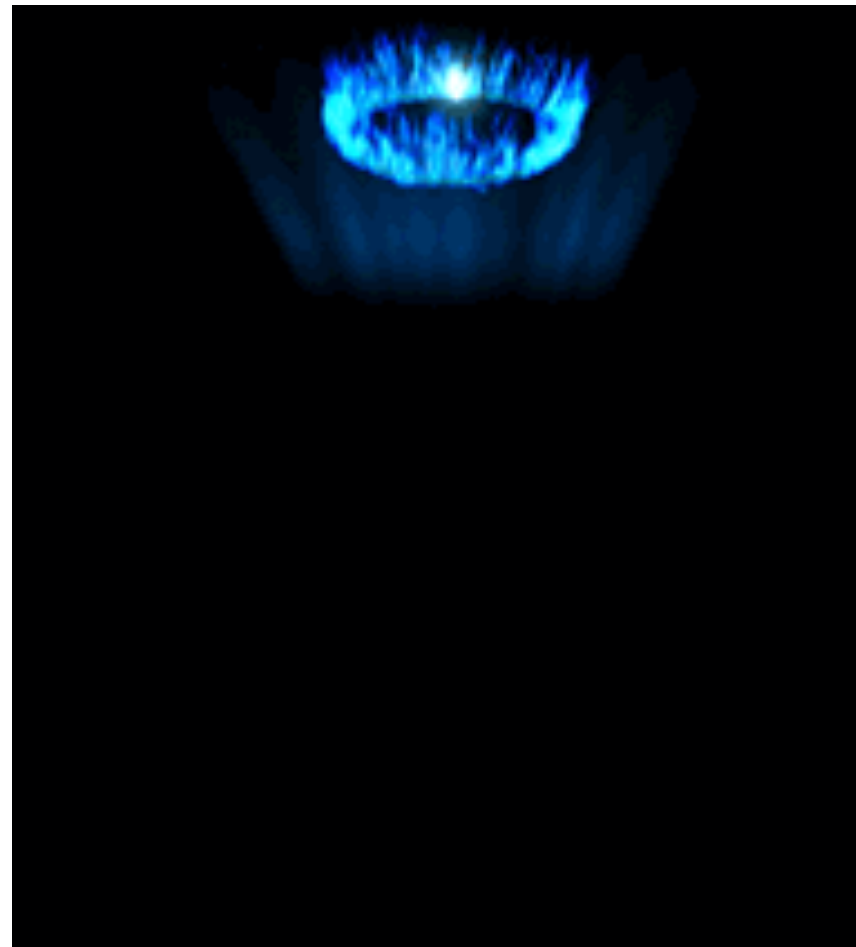
泡沫、溅起的水花采用粒子系统生成



《完美风暴》剧照

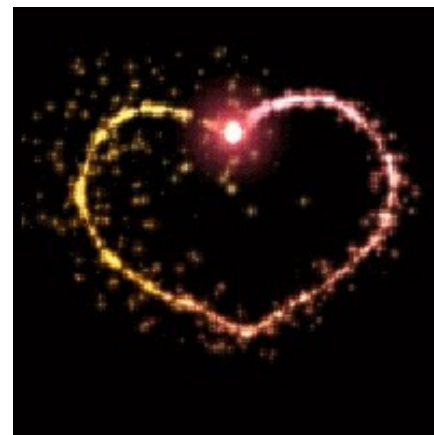
粒子系统——假设

- 粒子一般与其它粒子不碰撞
- 除非处于聚集状态，粒子不向其它
粒子投射阴影
- 粒子只向其它环境投射阴影
- 粒子不反射光
- 粒子通常有有限的生命周期



粒子系统——每一帧中的步骤

- 粒子系统的基本思想是将许多简单形状的微小粒子作为基本元素聚集起来形成一个**不规则的模糊物体**，每个粒子均经历出生、成长、衰老和死亡的过程，与粒子有关的每一个参数均将受到一个随机过程的控制。
- 生成粒子系统**某一帧画面**的基本步骤是：
 - 生成新的粒子并加入系统中；
 - 赋予每一新粒子以一定的属性；
 - 删除那些已经超过其生命周期的粒子；
 - 根据粒子的动态属性对粒子进行移动和变换；
 - 绘制并显示由有生命的粒子组成的图形。



粒子生成

- 对于每一帧，根据一个**控制的随机过程**生成粒子：
 - 用户可控制每帧的平均粒子数和其概率分布
 - 粒子数可以是时间的函数
- 方法一：
粒子数 $n=m+v*\text{rand}()$
其中， m 为平均粒子数；
 $\text{rand}()$ 返回一个 $[-1,1]$ 之间的随机数；
 v 为变化幅度；
- 方法二：
粒子数 $n=(m+v*\text{rand()})*A$
其中， m 为平均粒子数；
 $\text{rand}()$ 返回一个 $[-1,1]$ 之间的随机数；
 v 为变化幅度；
 A 为物体的屏幕面积 (Screen Area)

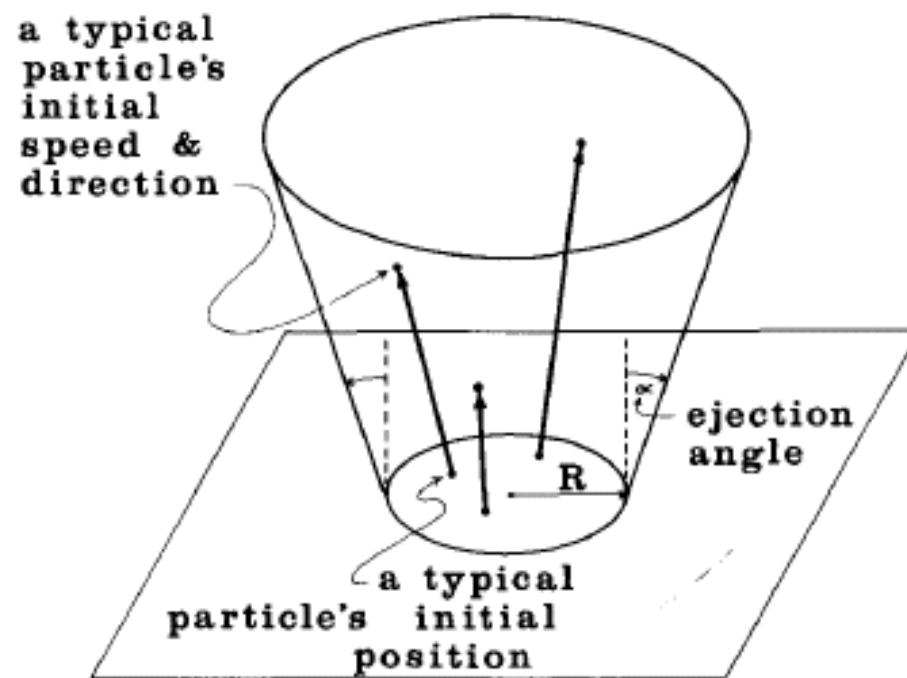
粒子属性(Attributes of a Particle)

- 一个粒子(质点)的属性可以包括

- 位置
- 速度
- 大小
- 质量
- 力累加器(force accumulator)
- 生命周期
- 绘制属性 (形状参数, 颜色, 透明度)
- ...

- 当粒子生成时, 对其属性进行初始化

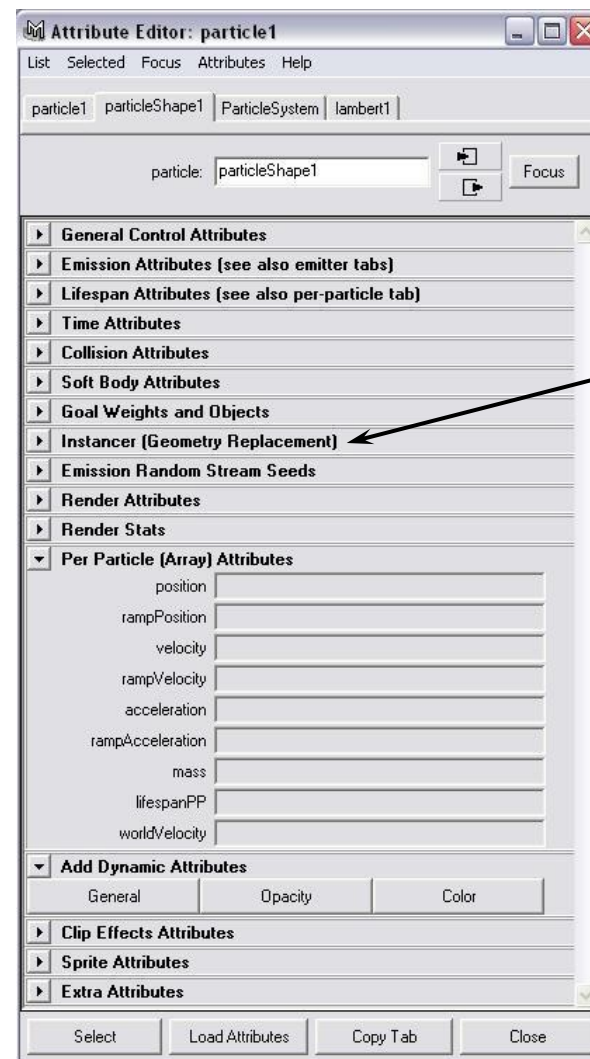
$$\text{Value} = \text{mean} + \text{Rand()} * \text{sqrt}(\text{variance})$$



粒子系统属性

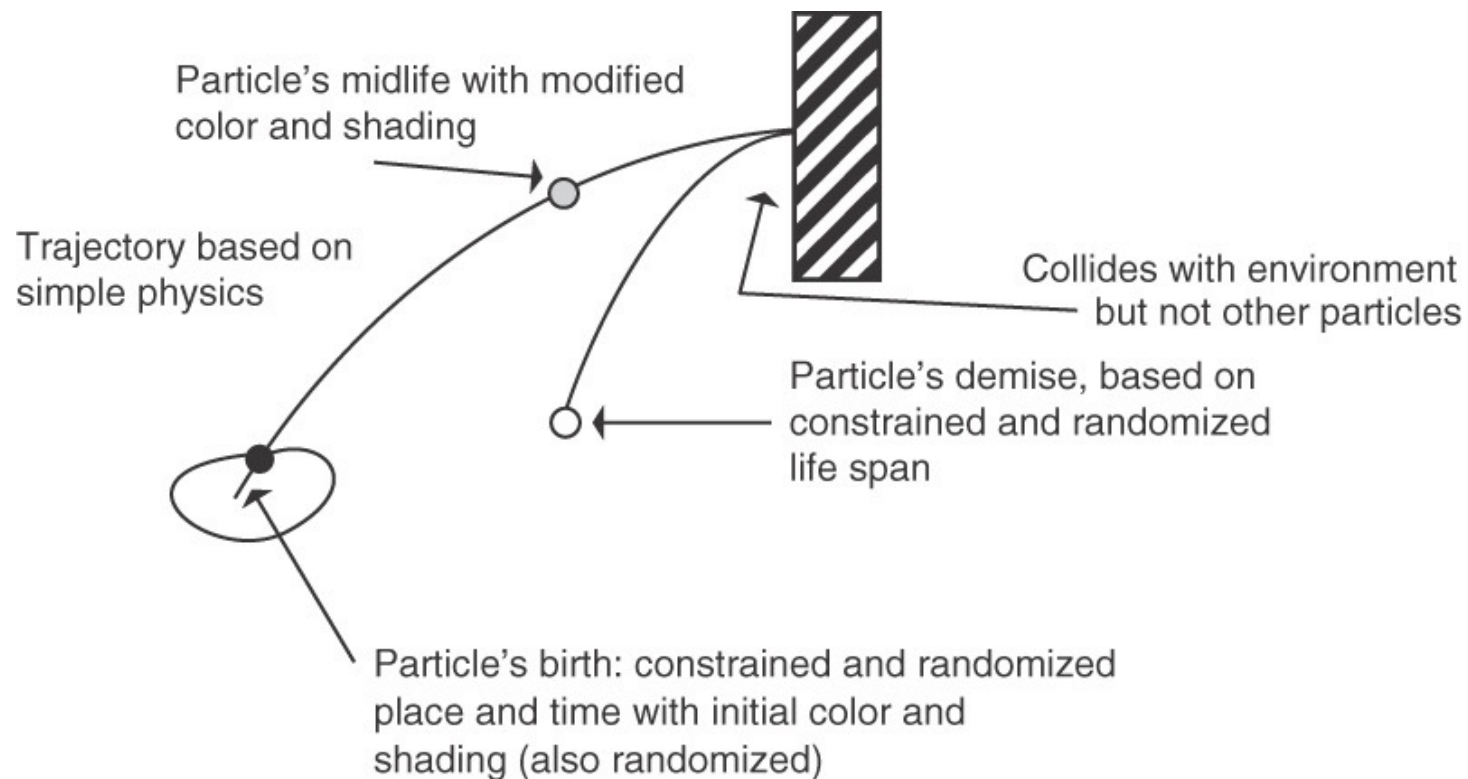
(Attributes of a Particle System)

- Particle List: 指向一个粒子系统的指针
- Position: 粒子系统的位置
- Emission Rate: 决定生成新粒子的创建方法
- Forces : 加入力可极大地增强物理真实程度
- Current State: 用来记录系统的状态, 例如在模拟爆炸时, 粒子系统会很快停止发射
- Blending: 用来设置粒子相互混合的方式
- Representation: 例如, 粒子的纹理
- ...



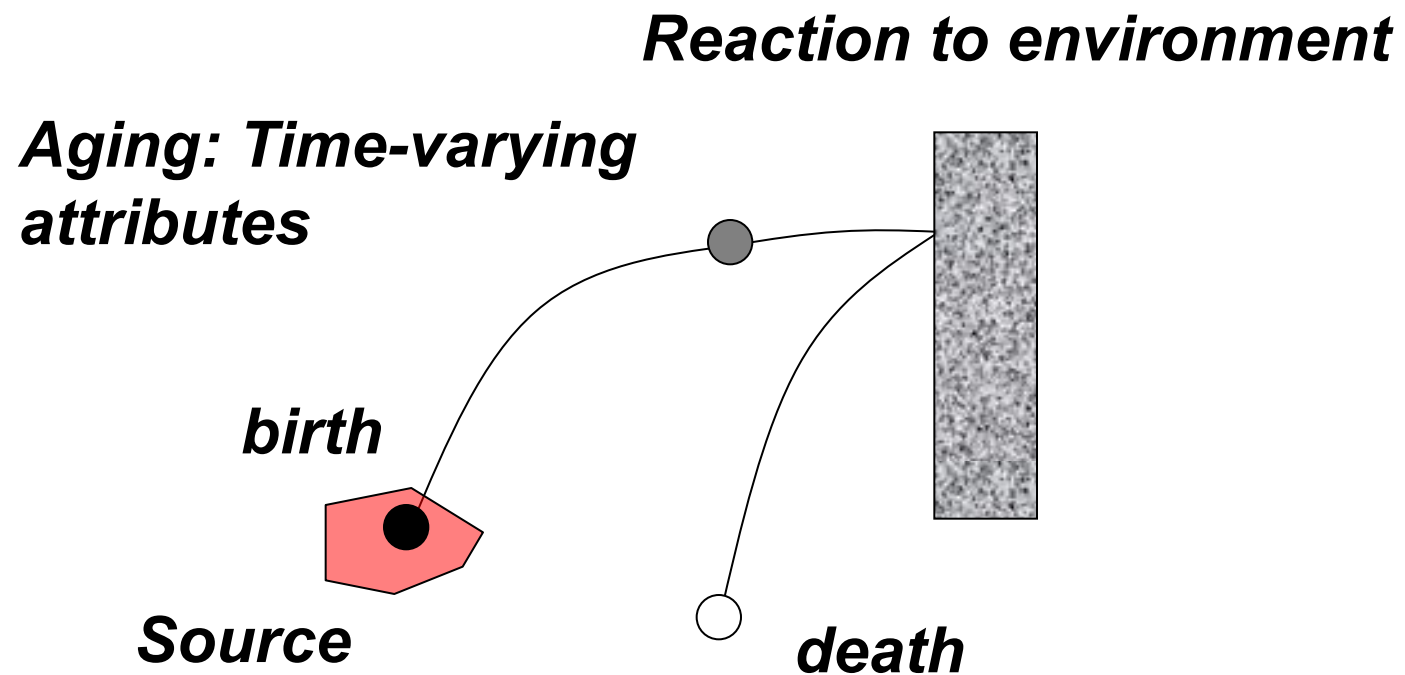
*Instancer
(Geometry
Replacement)*

粒子的生命周期



在每一新帧，每个粒子的生命周期减1，当该属性变成0时，把该粒子从系统中删除

粒子的生命周期



粒子消亡

- 当如下情形发生时，粒子将销毁：
 - 生命周期结束
 - 当创建一个粒子时，将赋予它一生命周期。每模拟一帧，该值减1。当该值为0时，该粒子将被销毁。
 - 淡出(Fades out)
 - 当粒子的颜色/不透明度小于一用户给定的阈值时，该粒子不可见并将被销毁。
 - 移走(Moves away)
 - 不在视域。

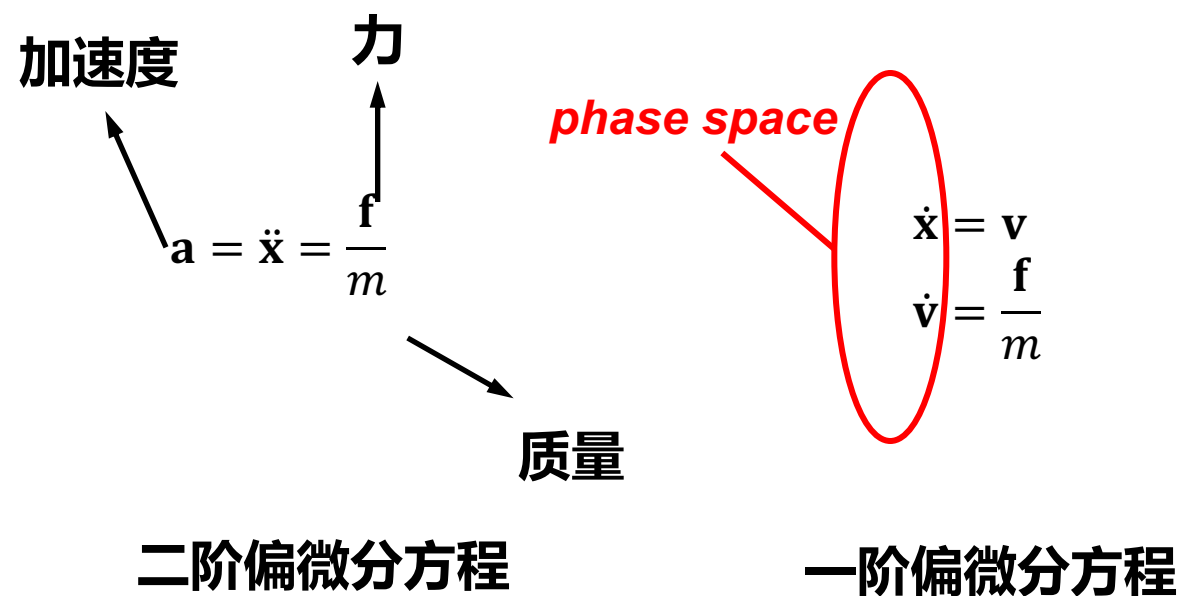
粒子动画

- 在生命周期内，对每个粒子设置动画
 - 包括位置、速度和绘制属性
- 位置和速度
 - 根据合力，计算粒子的加速度，进而更新粒子的速度和位置
- 颜色和透明度可以是时间（其剩余的生命周期、其高度等）的函数，形状可以是速度的函数

粒子上的力

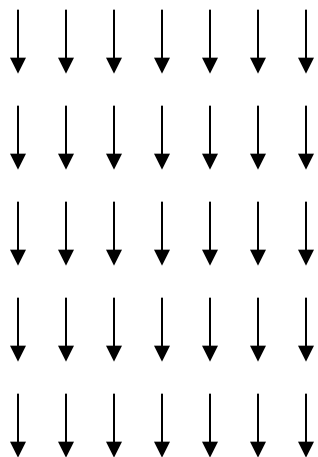
- 粒子上的力可以是一元力(unary force)、粒子的偶力(particle pair force)或环境力
 - 一元力
 - 重力(Gravity), 粘性阻力(viscous drag)
 - 粒子的偶力
 - 可表示为弹簧阻尼器
 - 环境力
 - 根据粒子与环境的关系来得到

- 粒子对力做出反应
- 我们用微分方程来表示

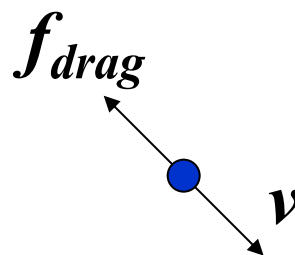


一元力(Unary Forces)

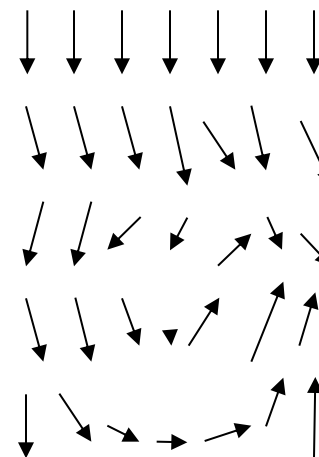
- 只与单个粒子相关的力



重力
Gravity
 $f = mg$



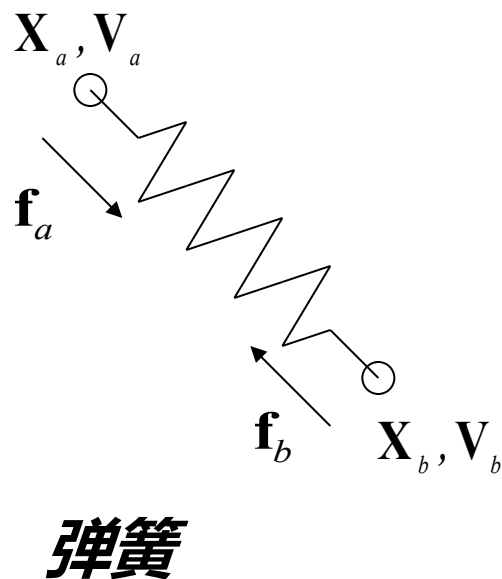
粘性阻力
Viscous Drag
 $f = -k_d v$



风场
Wind Fields
 $f = k v_{wind}$

n元力(n-ary Forces)

- 依赖于 n 个粒子的力
 - 例如: 两个粒子之间的二元力——弹簧阻尼力



$$\mathbf{f}_a = \underbrace{-k_s (|\mathbf{x}_a - \mathbf{x}_b| - l_0)}_{\text{弹簧力}} \frac{\mathbf{x}_a - \mathbf{x}_b}{|\mathbf{x}_a - \mathbf{x}_b|}$$

弹簧力

$$\underbrace{-k_d \left(\frac{(\mathbf{v}_a - \mathbf{v}_b) \cdot (\mathbf{x}_a - \mathbf{x}_b)}{|\mathbf{x}_a - \mathbf{x}_b|} \right)}_{\text{阻尼力}} \frac{\mathbf{x}_a - \mathbf{x}_b}{|\mathbf{x}_a - \mathbf{x}_b|}$$

阻尼力

n元力: 弹簧力

- 如果粒子比其静止位置远，则弹簧力需要把它拉回：

$$l = |\mathbf{x}_a - \mathbf{x}_b| - l_0 > 0 \quad \mathbf{f}_a = -k_s l \mathbf{d}, \quad k_s > 0$$

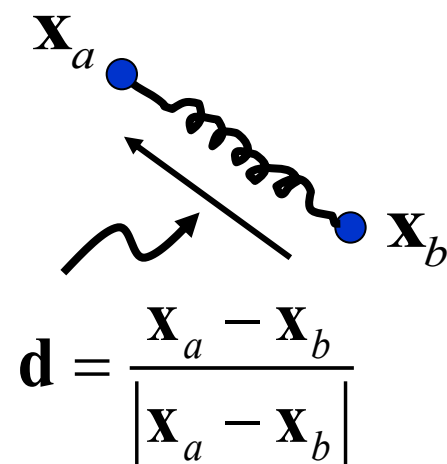
- 如果粒子比其静态位置近，则弹簧力需要把它拉开：

$$l = |\mathbf{x}_a - \mathbf{x}_b| - l_0 < 0 \quad \mathbf{f}_a = -k_s l \mathbf{d}, \quad k_s > 0$$

- 把上述两种情形相结合：

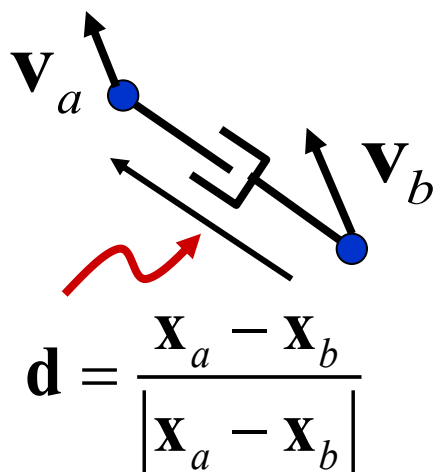
$$\mathbf{f}_a = -k_s (|\mathbf{x}_a - \mathbf{x}_b| - l_0) \mathbf{d}, \quad k_s > 0$$

$$= -k_s (|\mathbf{x}_a - \mathbf{x}_b| - l_0) \frac{\mathbf{x}_a - \mathbf{x}_b}{|\mathbf{x}_a - \mathbf{x}_b|}$$



n元力: 阻尼力

- 阻尼力/粘性阻力使粒子系统在外力消失后, 渐渐回到静止状态
- **推荐**: 每个粒子至少赋予一定的阻尼力
- 但是, 过度的阻尼力会使得粒子像在糖浆上漂浮一样 (能量消失太快, 不敏感)



- 如果两个粒子正在**远离**, 阻尼力会把他们拉回

$$(\mathbf{v}_a - \mathbf{v}_b) \cdot \mathbf{d} > 0$$

- 如果两个粒子正在**靠近**, 阻尼力会把他们推开

$$(\mathbf{v}_a - \mathbf{v}_b) \cdot \mathbf{d} < 0$$

- 结合这两种情形:

$$\begin{aligned} \mathbf{f}_a &= -k_d (|\mathbf{v}_a - \mathbf{v}_b| \cdot \mathbf{d}) \mathbf{d}, \quad k_d > 0 \\ &= -k_d \left(\frac{(\mathbf{v}_a - \mathbf{v}_b) \cdot (\mathbf{x}_a - \mathbf{x}_b)}{|\mathbf{x}_a - \mathbf{x}_b|} \right) \frac{(\mathbf{x}_a - \mathbf{x}_b)}{|\mathbf{x}_a - \mathbf{x}_b|} \end{aligned}$$

软约束(Soft Constraints)

软约束方程

- 通常, 指定一个**约束条件并使得它成立**比直接指定一个力来达到某种约束条件更加容易;
- **约束(或行为)函数**指定一个我们希望满足的条件

$$\mathbf{C}(\mathbf{x}_a, \mathbf{x}_b) = \mathbf{x}_a - \mathbf{x}_b = \mathbf{0}$$

或

$$C(\mathbf{x}_a, \mathbf{x}_b) = |\mathbf{x}_a - \mathbf{x}_b| - r = 0$$

约束条件

But how do we make it true?

能量方程(Energy Functions)

- 使得约束能量函数**最小化**:

$$E = \frac{k_s}{2} \mathbf{C} \bullet \mathbf{C}$$

- 从而得到一个作用在粒子上的**力**, 使得满足约束条件:

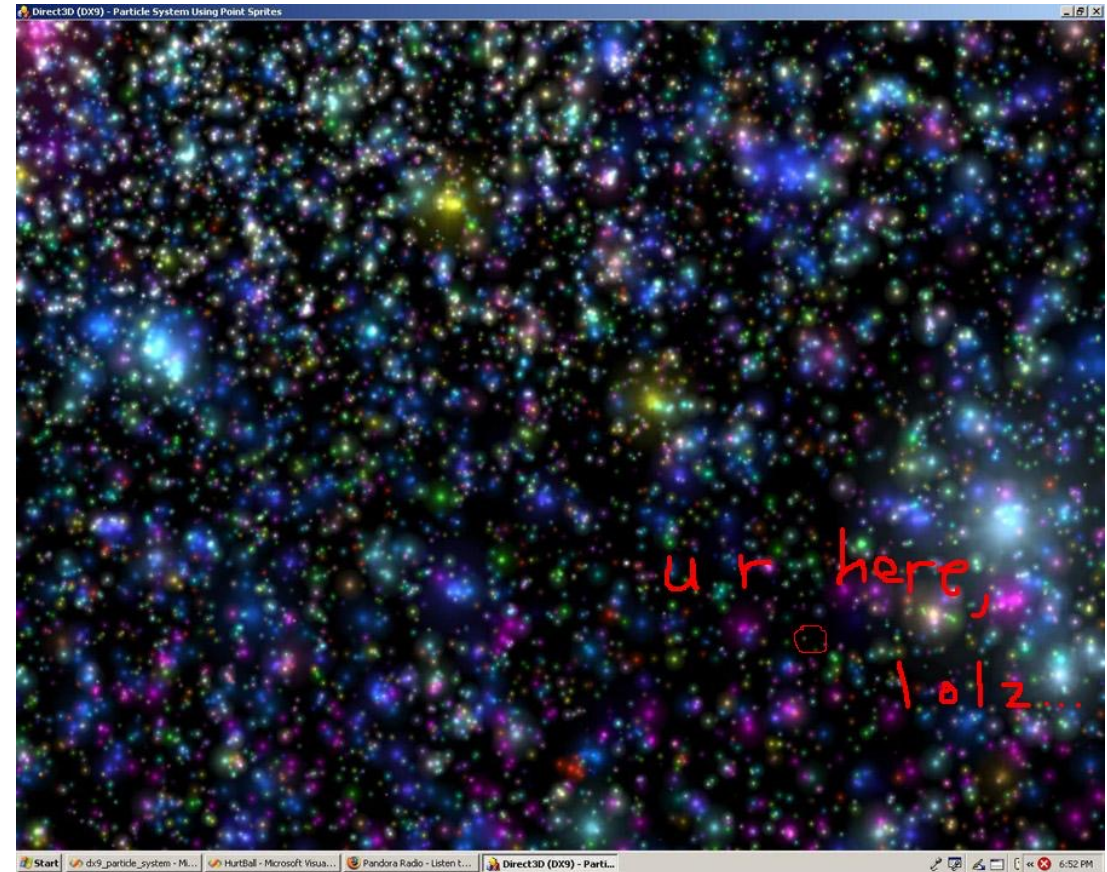
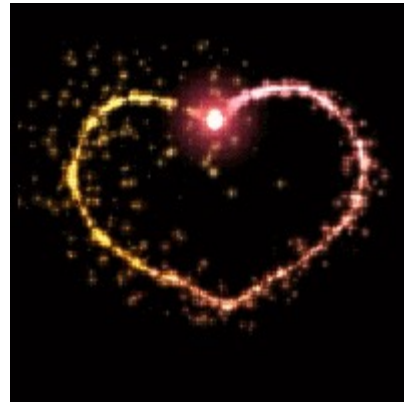
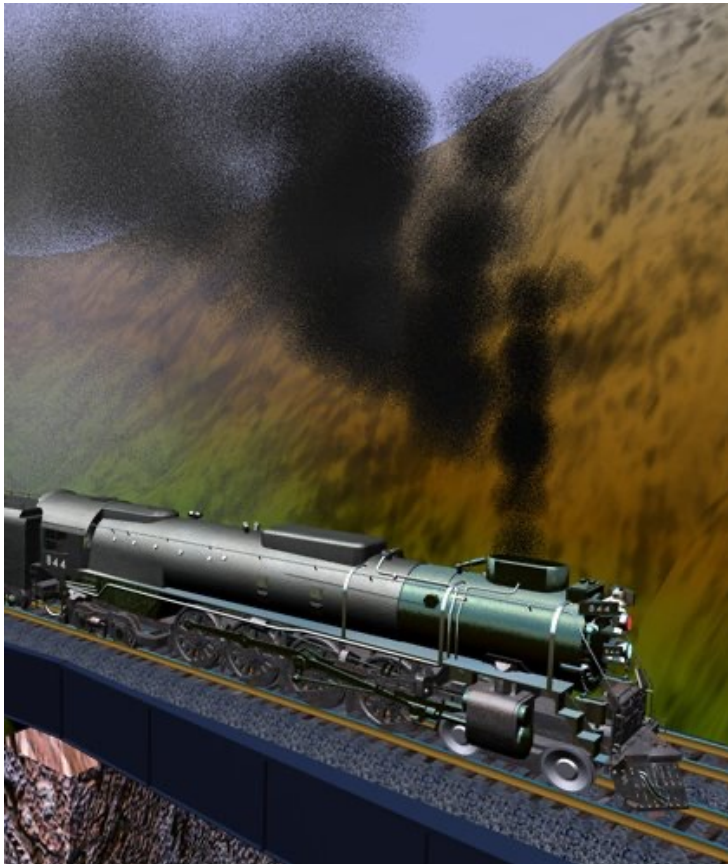
$$\mathbf{f}_i = -\frac{\partial E}{\partial \mathbf{x}_i} = -k_s \frac{\partial \mathbf{C}}{\partial \mathbf{x}_i} \mathbf{C}$$

$$\mathbf{f}_i^{\text{dampening}} = -k_d \frac{\partial \mathbf{C}}{\partial \mathbf{x}_i} \mathbf{C}$$

粒子的绘制 (Rendering)

- 包括如下方法：
 - 把每个粒子当成一个**点光源**
 - 把每个粒子绘制成一个小的图元；
 - 把映射到同一像素的颜色相加；
 - 把每个粒子建模为一个**带纹理的广告牌**(textured billboard)
 - 纹理多边形面向视点
 - 把每个粒子看成**元球**，绘制整个元球系统(用于模拟水等液态效果)
 - ...

粒子的绘制——把粒子当光源



粒子的绘制——把粒子当光源

submitted to CASA 2009

Fireworks Controller

Paper ID 44

Hanli Zhao, Ran Fan, Charlie C. L. Wang, Xiaogang Jin, Yuwei Meng: Fireworks controller. Computer Animation and Virtual Worlds, 20(2-3): 185-194 (2009)

粒子的典型应用：烟花

Unity VFX Graph FireWorks

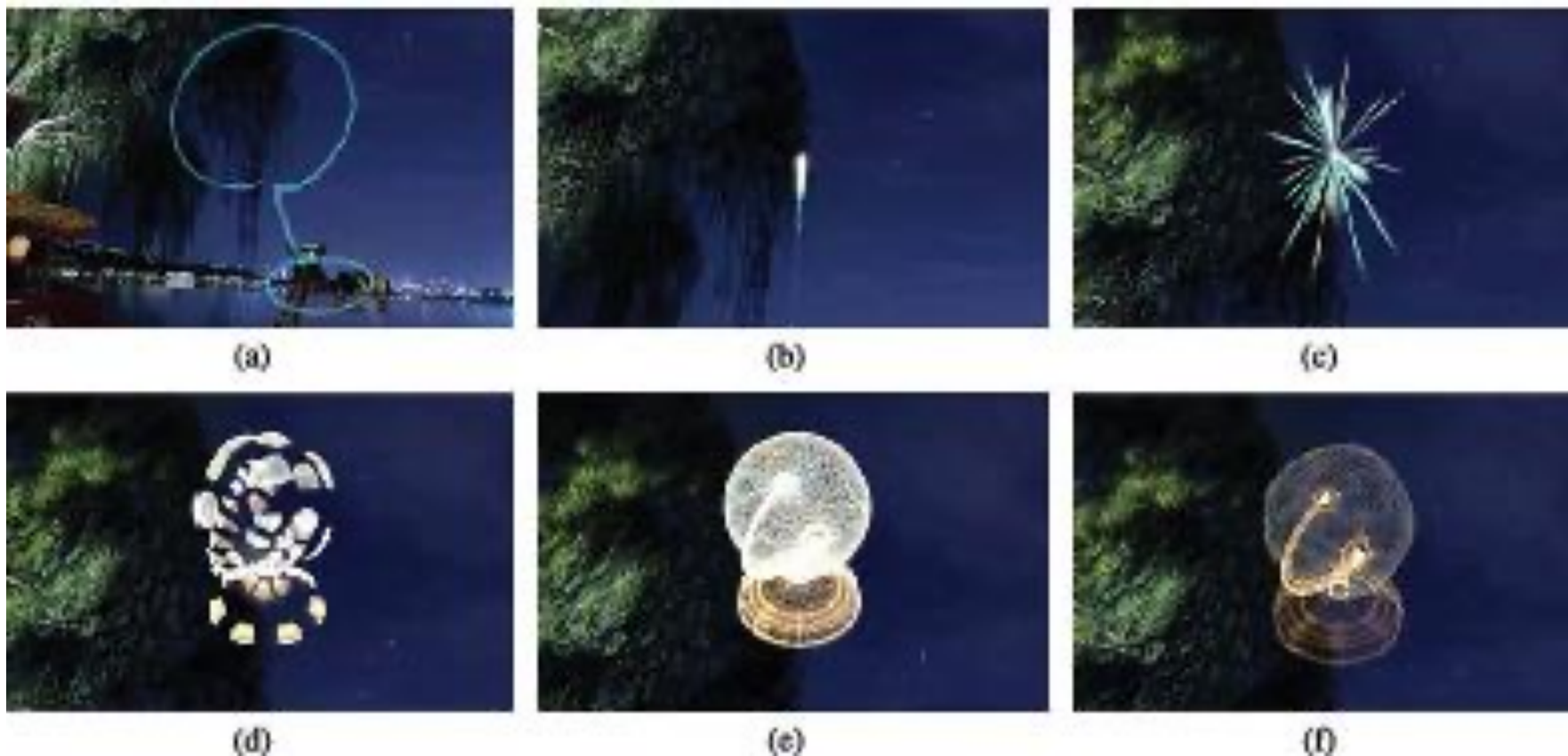


天梯：蔡国强的艺术

- 21年只为打造80秒烟花！他震撼张艺谋、感动5000万网友

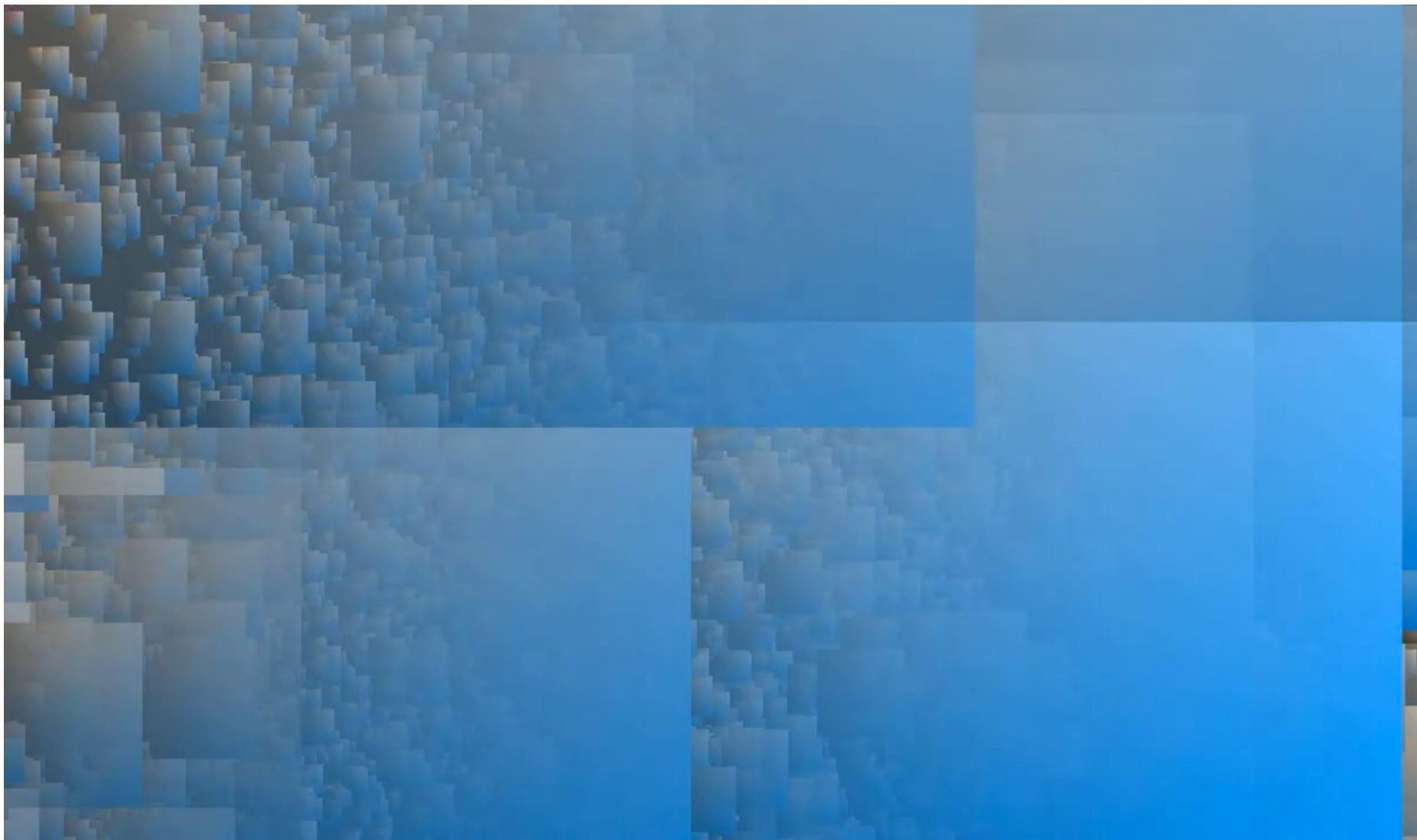


杭州亚运会-交互式烟花体验



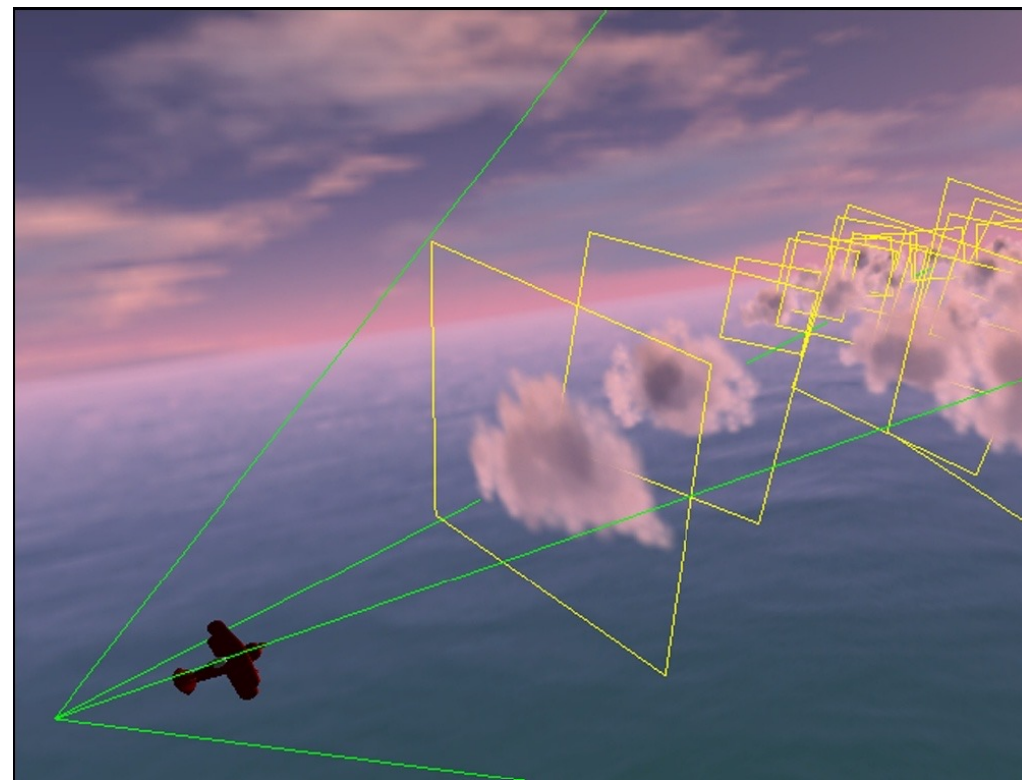
Xiaoyu Cui, Ruifan Cai, Xiangjun Tang, Zhigang Deng, Xiaogang Jin: Sketch-based shape-constrained fireworks simulation in head-mounted virtual reality. *Comput. Animat. Virtual Worlds* 31(2) (2020): e1920.

粒子绘制—把粒子作为带纹理的广告牌



粒子绘制—把粒子作为带纹理的广告牌

- 一个impostor: 把一片云用一个带纹理的广告牌来代替(纹理图像为某一视点的云)
 - 如果视点位置的变化将带来较大的误差, 则把纹理图像更新
 - Impostors对很多帧可重用
- 通过采用impostors, 我们可通过实时绘制成千上万的粒子来模拟云彩场景



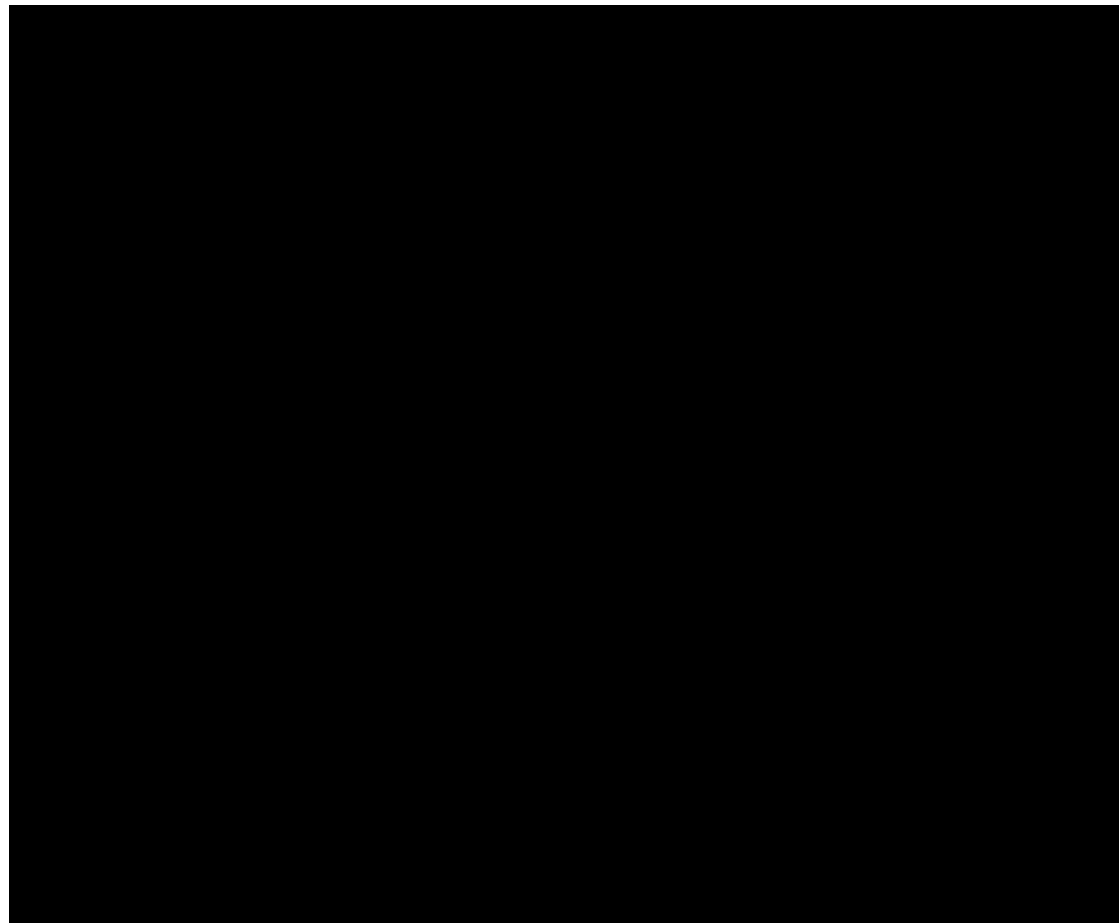
Impostors: textured, view-oriented polygons

- 参考: Niniane Wang, "Realistic and Fast Cloud Rendering", Journal of Graphics, Gpu, and Game Tools, 2004, 9(3): 21-40.

粒子绘制——把粒子作为带纹理的广告牌



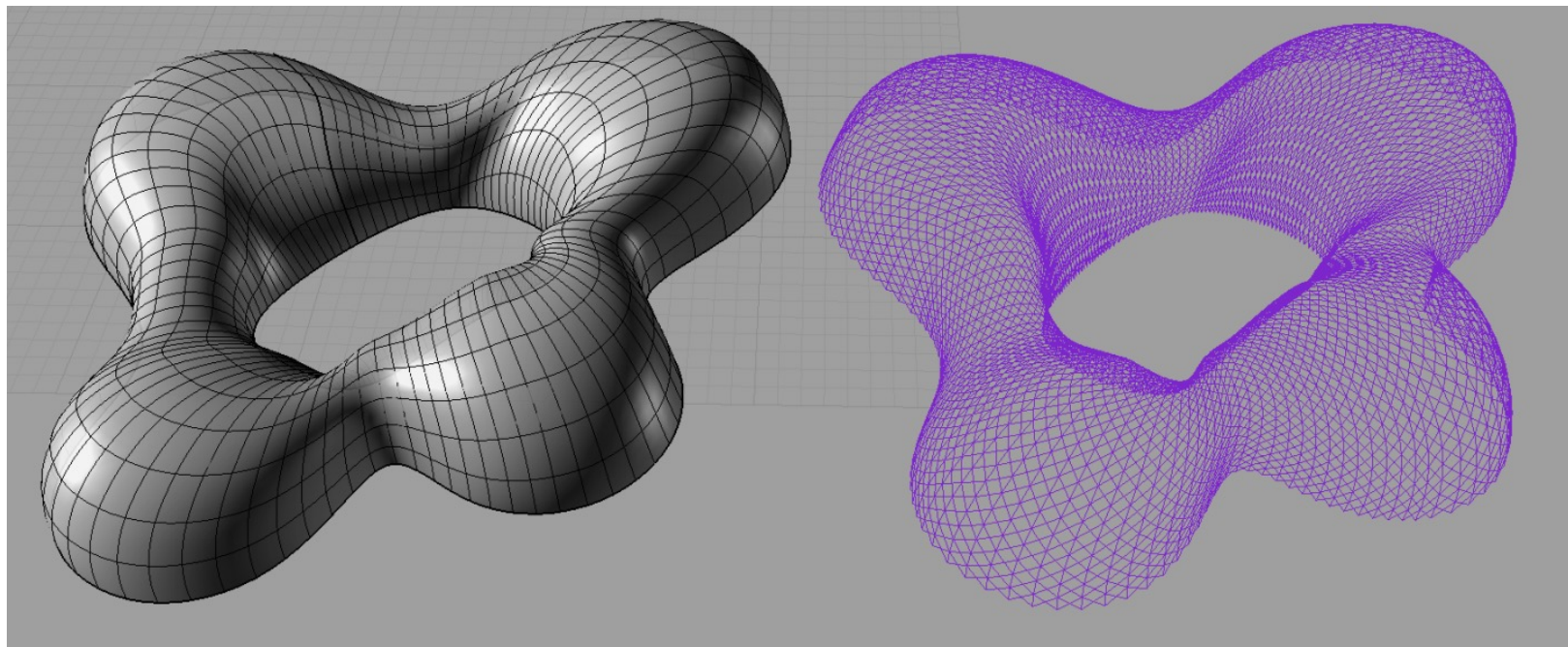
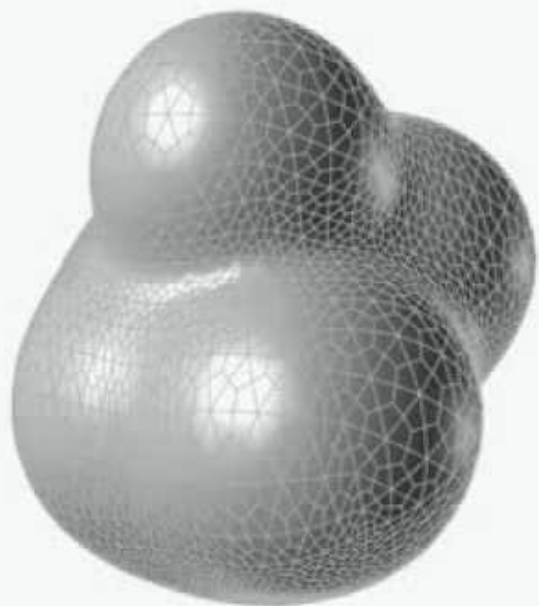
粒子绘制——把粒子作为带纹理的广告牌



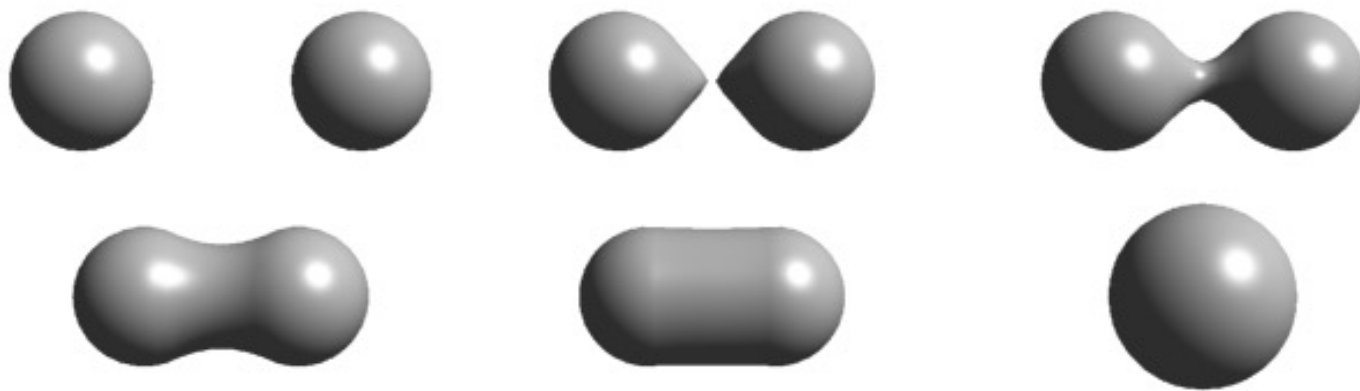
Video: Cloud rendering by Niniane Wang <http://www.ofb.net/~niniane/clouds/>

粒子绘制——把粒子作为元球

- 元球造型是一种隐式曲面造型技术，该技术采用具有等势场值的点集来定义曲面。因此，元球生成的面实际上是一张等势面。



粒子绘制——把粒子作为元球

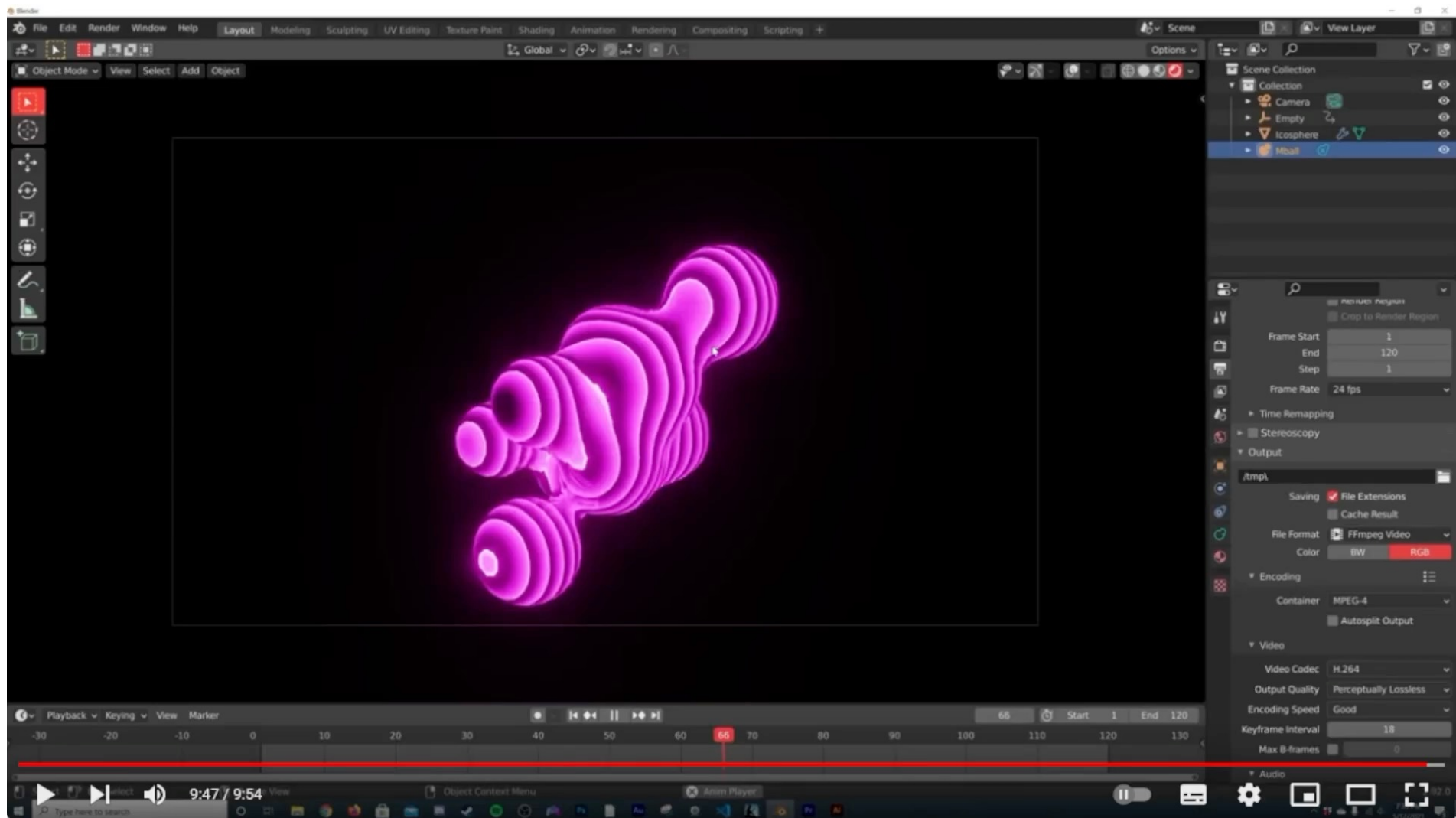


- 元球相互靠近到一定距离产生变形，再进一步靠近时则融合成光滑表面。
- 以两个元球为例，元球靠近时的变形过程如上图所示。最初是两个独立的球，彼此接近时，相对的面开始隆起变形，接近到一定程度就会象水滴（水银）一样融合成一个面。然后变成花生形状、胶囊形状、最后变成一个球。
- 上述过程实际上提供了一种模拟两滴水（水银）融合的动画过程，而用参数曲面和多边形网格造型方法来模拟此类动画过程是很困难的。
- 隐式曲面很适合表示可变形和可变拓扑的物体，因而对动画非常有用(如 morphing)。

粒子绘制——把粒子作为元球



粒子绘制——把粒子作为元球



采用Alpha Blending进行粒子的绘制



Without alpha-blending : particles are simply textured rectangles.



Alpha-blending is enabled

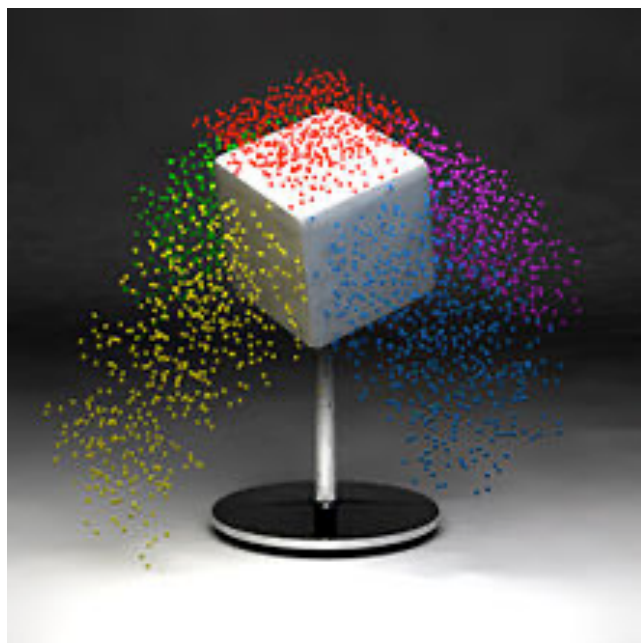
$$\text{FinalColor} = \text{DestColor} \times \text{DestBlendFactor} + \text{SrcColor} \times \text{SrcBlendFactor}$$



The more particles are on top of each other, the brighter the color in that area should be

“Snowflakes” versus “Hair”

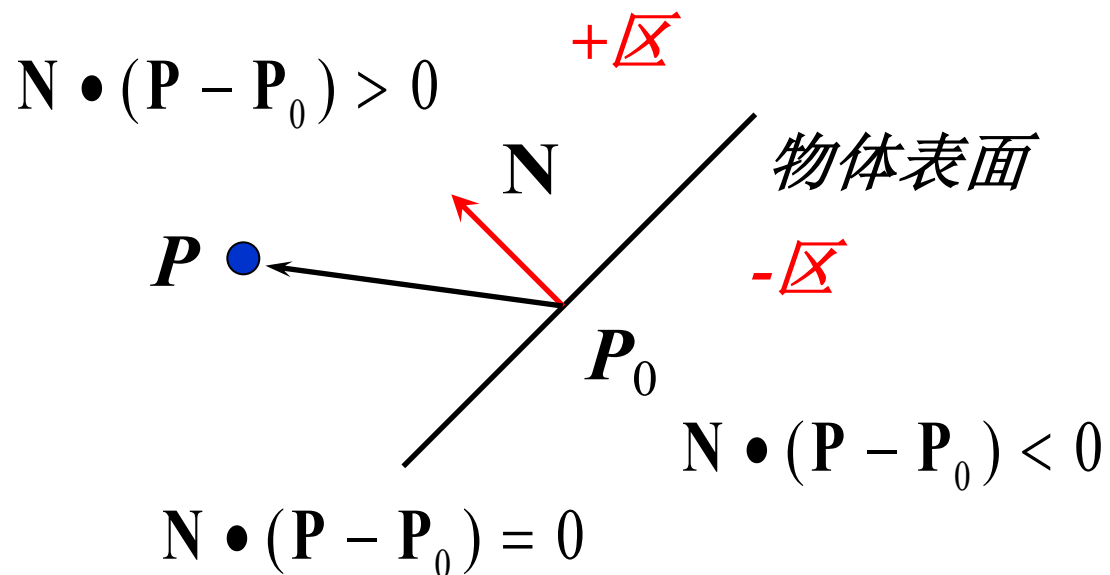
- 粒子系统可以是带动画的或者静止的。也就是说，每个粒子可在其生命周期中逐帧绘制，也可只绘制一次。
- 只绘制一次的**丝状物体**：头发、毛、草等。可用速度、力场、发射速度、偏移参数等控制。



一个立方体源发射5000个带动画的粒子，受重力影响 同样的立方体用静止的轨迹粒子(像发丝)绘制

粒子的碰撞检测

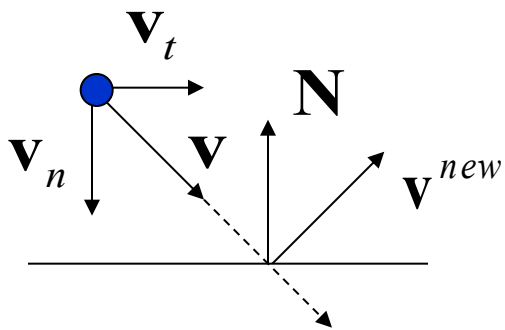
- 判断一个粒子是否与物体发生碰撞



- 当且仅当 $N \bullet (P - P_0) < 0$ 时，粒子与平面发生碰撞

碰撞响应(Collision Response)

- 当一个粒子已经发生碰撞时，该做什么？
- 正确的做法是：把模拟**回退到接触点**(the (point of contact))
- 只需修改粒子的**位置**和**速度**



碰撞后，新的速度：

$$\mathbf{V}^{new} = -\varepsilon \mathbf{V}_n + \mathbf{V}_t$$

回弹系数

(coefficient of restitution)

粒子运动方程的求解

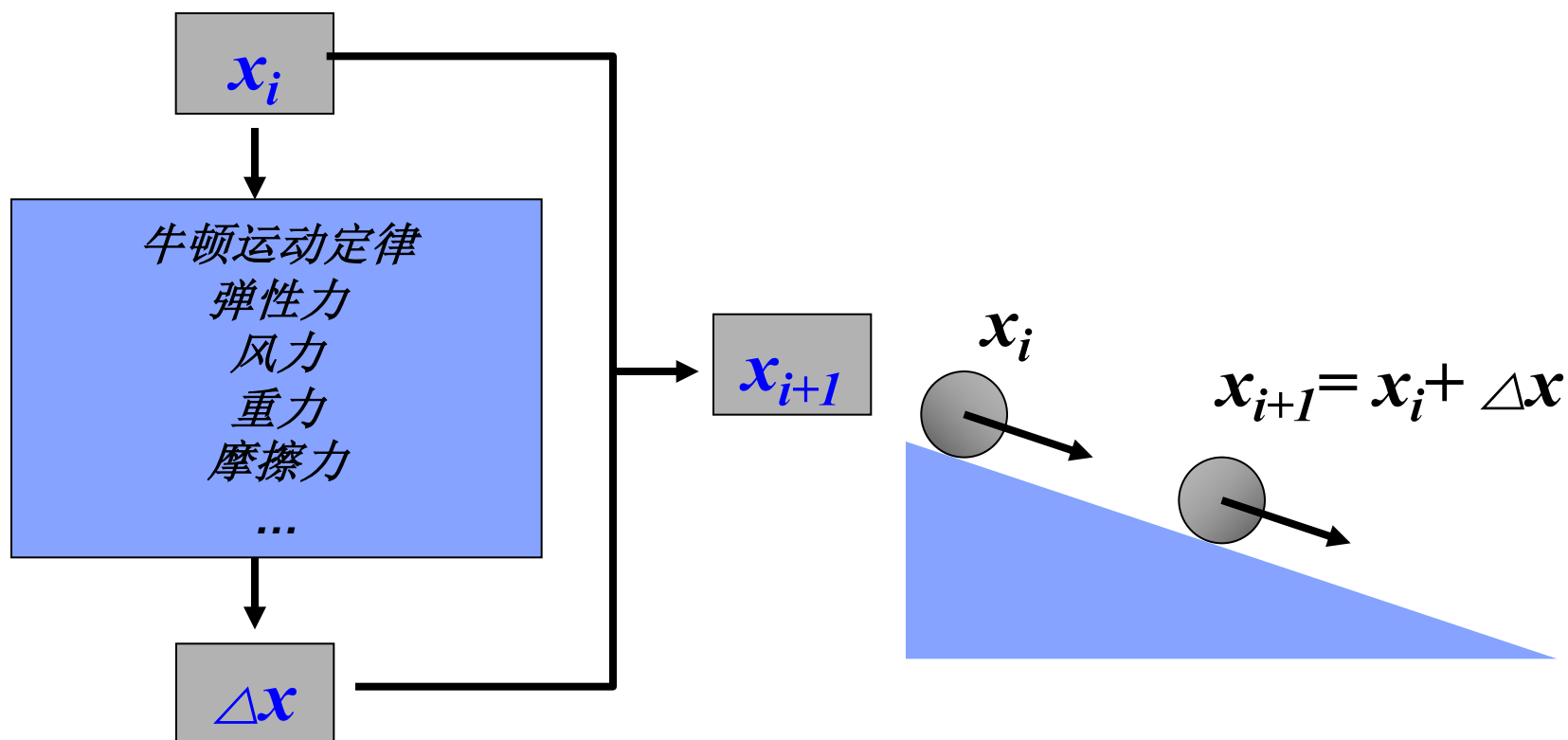
$$\mathbf{a} = \ddot{\mathbf{x}} = \frac{\mathbf{f}}{m}$$

- 连续处理 **相对于** 离散时间模拟
 - 假设在单位时间步长内是常数，
 - 单位时间步长内对**加速度**积分，得到**速度**的变化，
 - 单位时间步长内对**速度**积分，得到物体**位置**的变化
 - 如何更新？
 - 欧拉积分法
 - Verlet法
 - ...

粒子运动方程的求解

- 基于物理规律，生成一个系统的状态系列

$$\mathbf{a} = \ddot{\mathbf{x}} = \frac{\mathbf{f}}{m}$$



微分方程

微分方程

- 微分方程描述了一个未知函数与其导数之间的关系
- 解微分方程是为了找到满足该关系的函数
- 通常用数值求解方法!

常微分方程

- Ordinary differential equation (ODE)
 - 所有导数都关于一个独立变量, 通常为时间 t

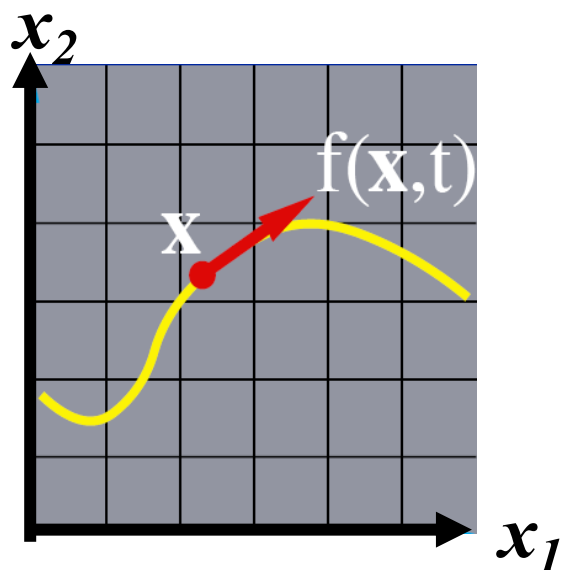
已知函数

$$\frac{d\mathbf{x}}{dt} = f(\mathbf{x}(t)) = f(\mathbf{x}, t)$$

未知函数对时间的导数

$\mathbf{x}_0 = \mathbf{x}(t_0)$ 初始值

常微分方程解的可视化



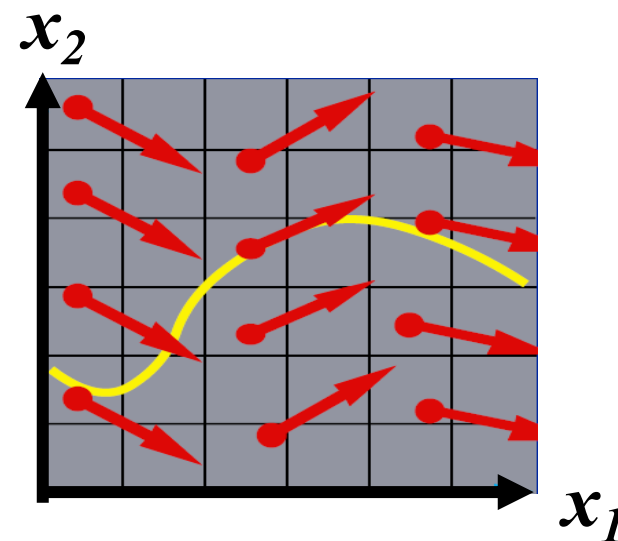
$$\dot{\mathbf{x}} = \frac{d\mathbf{x}}{dt} = f(\mathbf{x}, t)$$

$\mathbf{x}(t)$: 一个移动的点

$f(\mathbf{x}, t)$: $\mathbf{x}$ 的速度

微分方程 $\dot{\mathbf{x}} = f(\mathbf{x}, t)$ 的所有解定义了一个在平面 $(\mathbf{x}, t)$ 上的向量场

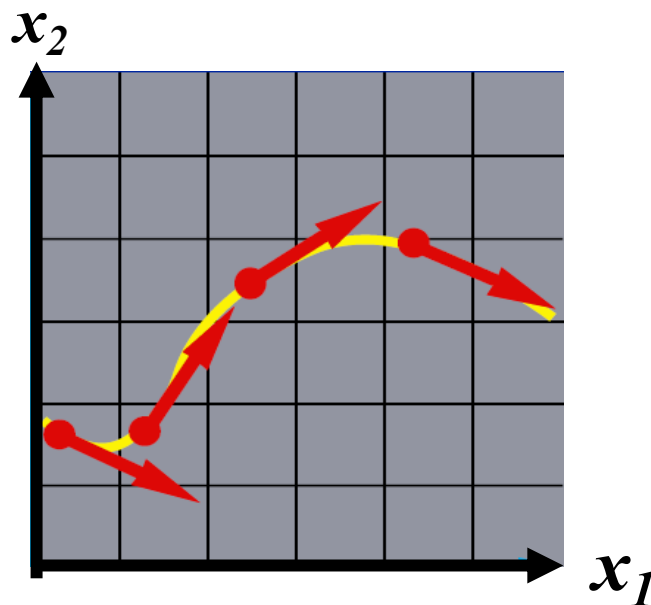
Think of this vector field as the sea, and the velocity of current at different places and time is defined by $f(\mathbf{x}, t)$



初值问题

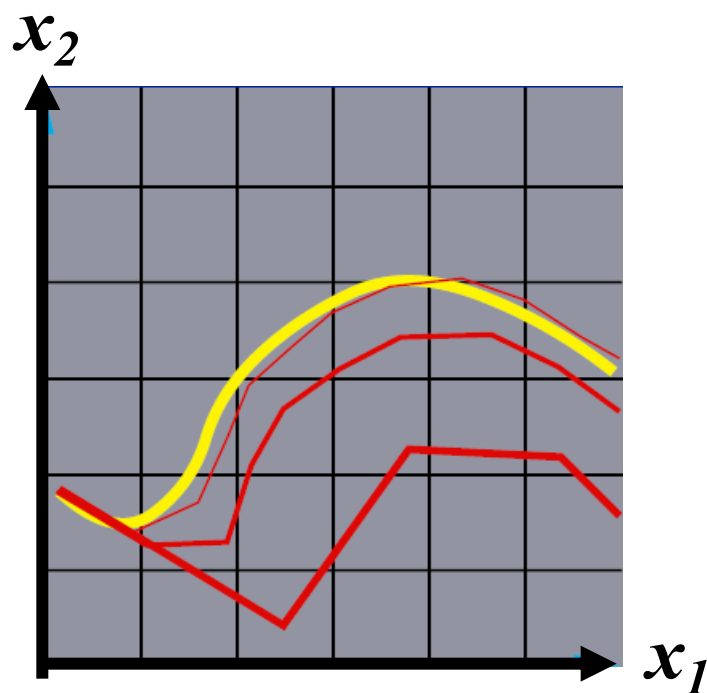
给定 $\dot{\mathbf{x}} = f(\mathbf{x}, t)$ 和 $\mathbf{x}_0 = \mathbf{x}(t_0)$, 求 $\mathbf{x}(t)$

- 对于一个特定的初值，其解是一条曲线
 - 给定初始点，顺着这条积分曲线



常微分方程解的数值解

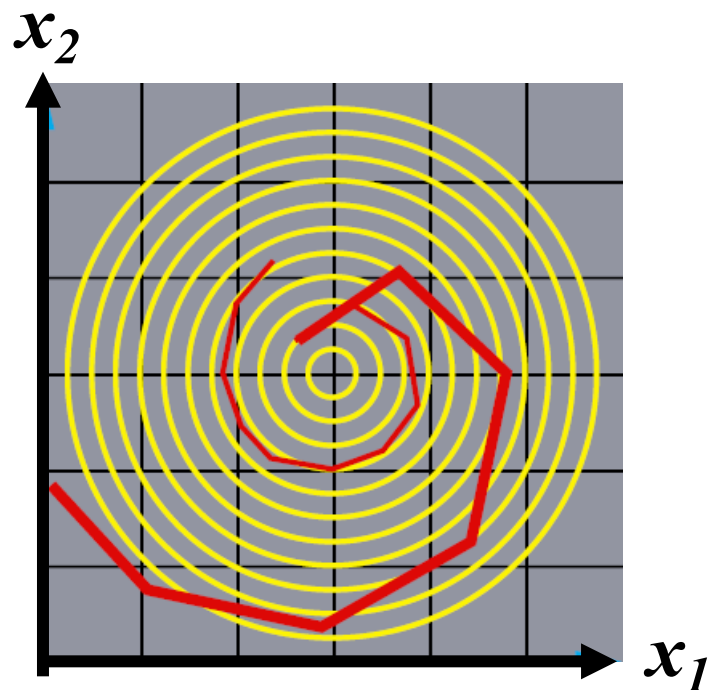
- 与真实的积分曲线不同，数值解通常为一条多边形折线
- 步长越大，误差越大



求解中可能产生的问题

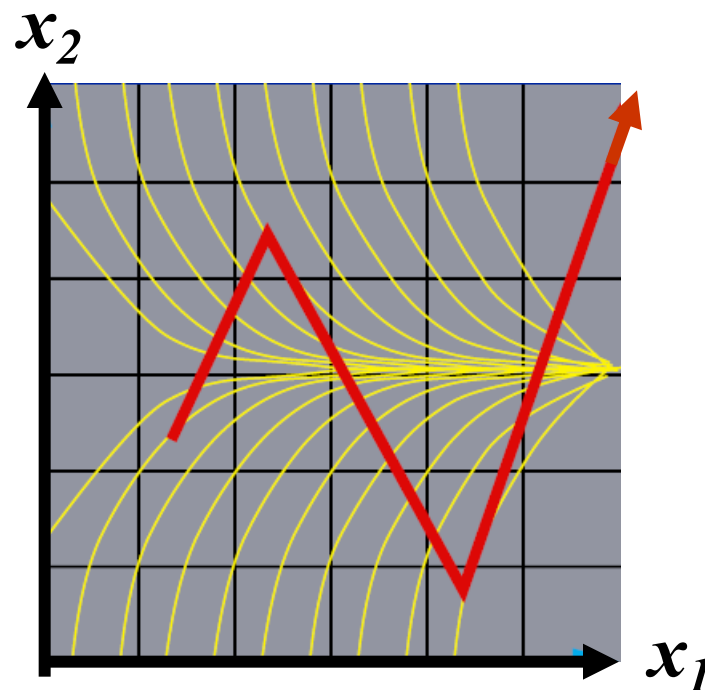
- 问题I: 精度(Inaccuracy)

- 误差把 $\mathbf{x}(t)$ 从一个圆变成了螺旋线!
- 可能跳到其它圆



- 问题II: 不稳定

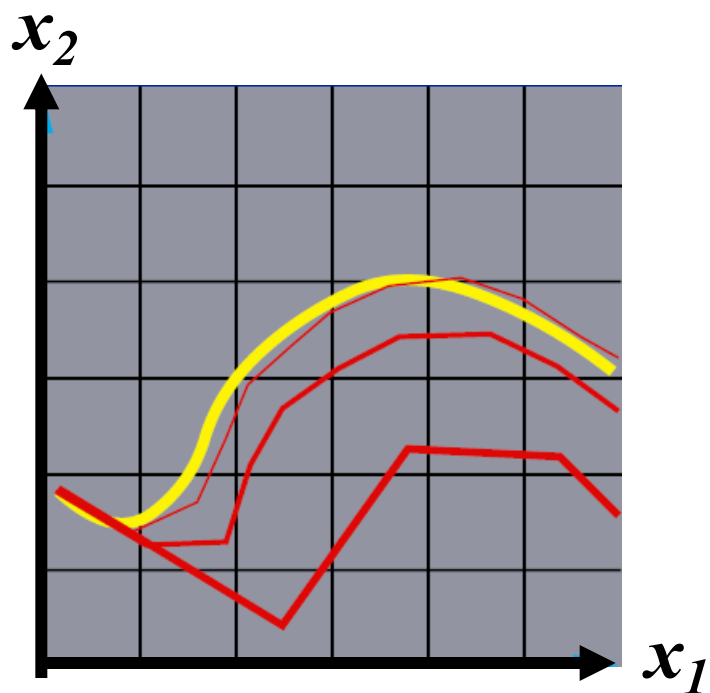
- 有可能发散!



求解：欧拉法

- 是最简单的数值求解方法
- 时间步长越大，误差越大

$$\mathbf{x}(t+h) = \mathbf{x}(t) + h \times f(\mathbf{x}, t)$$



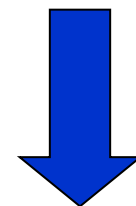
给定 $\dot{\mathbf{x}} = f(\mathbf{x}, t)$ 和 $\mathbf{x}_0 = \mathbf{x}(t_0)$, 求 $\mathbf{x}(t)$

通过一阶泰勒展开来求解微分方程:

$$\mathbf{x}(t+h) = \mathbf{x}(t) + h\dot{\mathbf{x}}(t) + \frac{h^2}{2}\ddot{\mathbf{x}}(\xi)$$

$$\dot{\mathbf{x}} = f(\mathbf{x}, t)$$

$$\mathbf{x}_0 = \mathbf{x}(t_0)$$



$$\mathbf{x}(t+h) \approx \mathbf{x}(t) + h \cdot f(\mathbf{x}, t)$$

$$\longrightarrow \mathbf{x}_{n+1} = \mathbf{x}_n + h\dot{\mathbf{x}}_n$$

$O(h^2)$:
二阶精度

欧拉法的缺点

- 精度差，难以实用
- 效率低
 - 需要用很小的时间步长以避免发散

Verlet积分方法

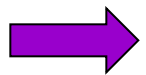
- The Verlet algorithm uses the positions and accelerations at time t , and the positions from the previous step $\mathbf{x}(t-h)$, to calculate the new positions $\mathbf{x}(t+h)$:

$$\mathbf{x}(t+h) = \mathbf{x}(t) + \mathbf{v}(t)h + \frac{1}{2}\mathbf{a}(t)h^2 + \dots$$

+

$$\mathbf{x}(t-h) = \mathbf{x}(t) - \mathbf{v}(t)h + \frac{1}{2}\mathbf{a}(t)h^2 - \dots$$

$$\mathbf{a}(t) = \frac{\mathbf{f}(t)}{m}$$



$$\mathbf{x}(t+h) = 2\mathbf{x}(t) - \mathbf{x}(t-h) + \mathbf{a}(t)h^2$$

- The velocities do not appear in the Verlet integration scheme: *velocities are not necessary for generating particle trajectories.*

Verlet积分方法

To obtain the **new velocities**, we can calculate them from the difference between the positions at two different times:

$$\mathbf{v}(t) = [\mathbf{x}(t+h) - \mathbf{x}(t-h)] / 2h$$

or

$$\mathbf{v}(t + \frac{1}{2}h) = [\mathbf{x}(t+h) - \mathbf{x}(t)] / h$$

Advantages:

Implementation is straightforward.

Storage requirements are modest:

two sets of positions and one set of accelerations.

9N stored numbers

Disadvantages:

Positions are calculated by adding a small term of order h^2 to the difference of two much larger terms, which may lead to a **loss in precision** (精度) :

$$\mathbf{x}(t+h) = 2\mathbf{x}(t) - \mathbf{x}(t-h) + \mathbf{a}(t)h^2$$

Velocities always lag behind positions.

Poor stability for large h.

Leap-Frog(蛙跳) Verlet积分方法

- Several variations on the Verlet algorithm have been developed, including the leap-frog algorithm:


$$\begin{aligned}\mathbf{x}(t+h) &= \mathbf{x}(t) + \mathbf{v}(t + \frac{1}{2}h)h \\ \mathbf{v}(t + \frac{1}{2}h) &= \mathbf{v}(t - \frac{1}{2}h) + \mathbf{a}(t)h\end{aligned}$$

- The velocities at time t can be calculated from:

$$\mathbf{v}(t) = \frac{1}{2} \left[\mathbf{v}(t + \frac{1}{2}h) + \mathbf{v}(t - \frac{1}{2}h) \right]$$

- The velocities leap-frog over the positions to give their values at $t+h/2$. The positions then leapfrog over the velocities to give their new values at $t+h$, then the velocities at $t+3h/2$, and so on.

Leap-Frog(蛙跳) Verlet积分方法

- Advantages over standard Verlet:

- Explicitly includes velocity (needed for kinetic energy).
- Does not require calculation of differences of large numbers, so more accurate.
- Same memory requirements (9N) as Verlet.

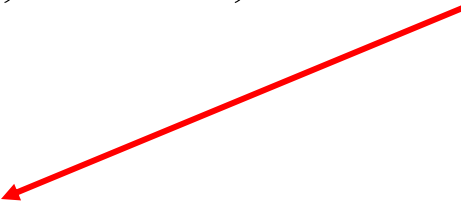
- Disadvantages over standard Verlet:

- Positions and velocities are not synchronized.
 - Cannot calculate kinetic energy (from velocities) and potential energy (from positions) at the same time.
 - Not a problem - can always estimate $v(t)$ from $v(t+h/2)$ and $v(t-h/2)$ if you need it.

Almost always used over Verlet.

Velocity Verlet积分方法

- The velocity Verlet method gives positions, velocities, and **accelerations** at the same time without compromising precision:

$$\mathbf{x}(t+h) = \mathbf{x}(t) + \mathbf{v}(t)h + \frac{1}{2}\mathbf{a}(t)h^2$$


$$\mathbf{v}(t+h) = \mathbf{v}(t) + \frac{1}{2}[\mathbf{a}(t) + \mathbf{a}(t+h)]h$$

- Implemented in three stages since calculating velocities requires accelerations at both t and $t+h$.

Velocity Verlet积分方法

- Actual implementation trick:

1. Velocities at $t+1/2h$ are calculated using information at t .

$$\mathbf{v}\left(t + \frac{1}{2}h\right) = \mathbf{v}(t) + \mathbf{a}(t) \frac{1}{2}h$$

2. With these terms in memory, calculate:

$$\mathbf{x}(t + h) = \mathbf{x}(t) + \mathbf{v}(t)h + \frac{1}{2}\mathbf{a}(t)h^2 = \mathbf{x}(t) + \mathbf{v}\left(t + \frac{1}{2}h\right)h$$

3. Calculate $\mathbf{a}(t+h)$ from forces using positions at $t+h$, and from this and $\mathbf{v}(t)$, calculate $\mathbf{v}(t+h)$.

$$\mathbf{v}(t + h) = \mathbf{v}(t) + \frac{1}{2}[\mathbf{a}(t) + \mathbf{a}(t + h)]h$$

欧拉法 vs Verlet方法

- Verlet方法和欧拉法都是基于离散化微分方程来近似求解的数值方法
- Verlet方法通常比欧拉法更准确。欧拉法是一阶方法，而Verlet方法可以是二阶或更高阶的
- Verlet积分法最突出的一点是不用保存对象的速度，取而代之的是保存对象的位置
- 欧拉法更简单易实现，但在需要更高精度和更好数值稳定性的情况下，Verlet方法通常更受欢迎

Implementation

Particle

```
struct Particle
{
    Vector3 m_pos;           // current position of the particle
    Vector3 m_prevPos;       // last position of the particle
    Vector3 m_velocity;       // direction and speed
    Vector3 m_acceleration;   // acceleration

    float m_energy;          // determines how long the particle is alive

    float m_size;            // size of particle
    float m_sizeDelta;       // amount to change the size over time

    float m_weight;          // determines how gravity affects the particle
    float m_weightDelta;     // change over time

    float m_color[4];        // current color of the particle
    float m_colorDelta[4];   // how the color changes with time
};
```

Particle System

```
class ParticleSystem{
public:
    ParticleSystem(int maxParticles, Vector3 origin);
    // abstract functions
    virtual void Update(float elapsedTime) = 0;
    virtual void Render() = 0;
    virtual int Emit(int numParticles);
    virtual void InitializeSystem();
    virtual void KillSystem();

protected:
    virtual void InitializeParticle(int index) = 0;
    Particle *m_particleList; // particles for this emitter
    int m_maxParticles; // maximum number of particles in total
    int m_numParticles; // indices of all free particles
    Vector3 m_origin; // center of the particle system
    float m_accumulatedTime; // track when was last particle emitted
    Vector3 m_force; // force (gravity, wind, etc.) acting on the particle system
};
```

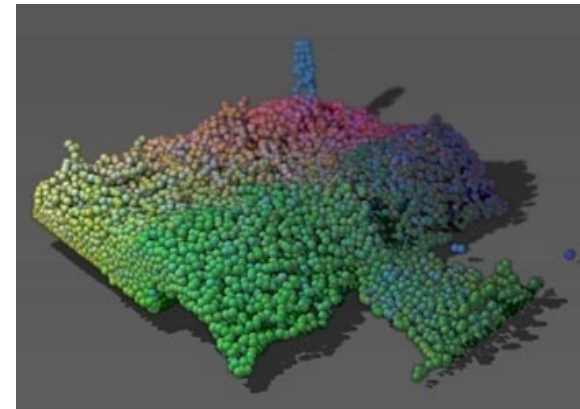
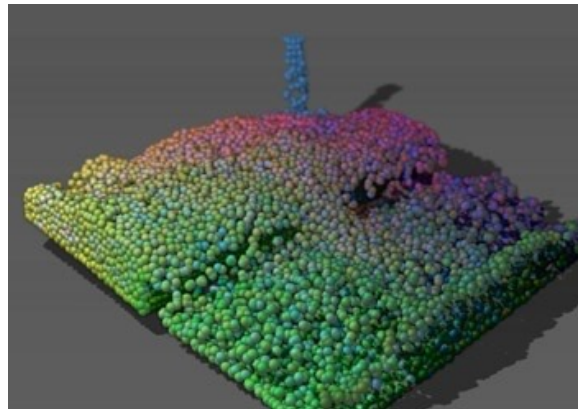
Particle System Source Codes

- A Basic Particles System

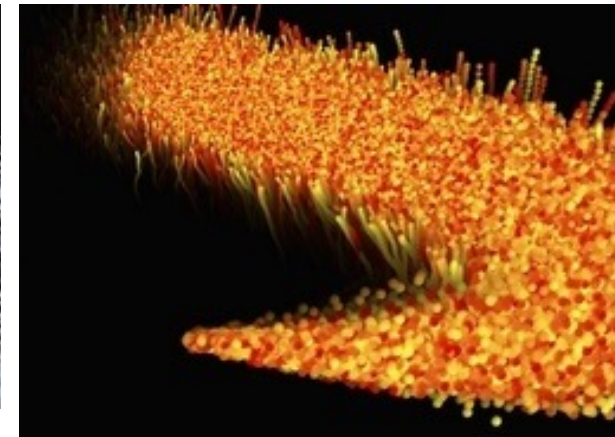
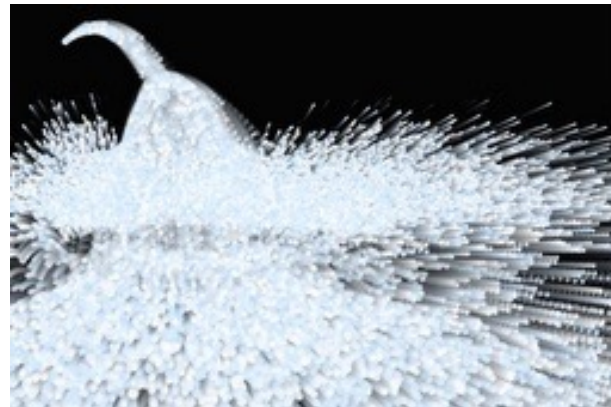
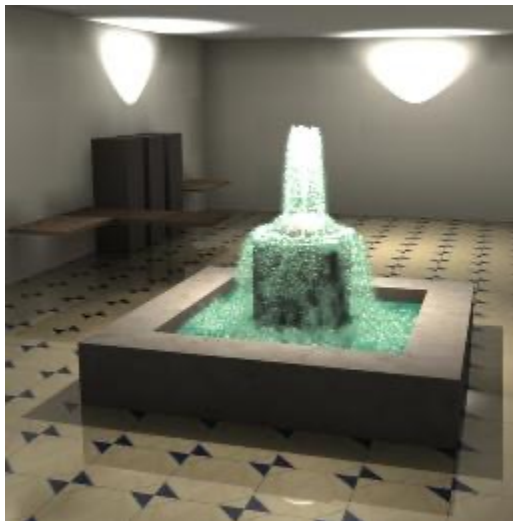
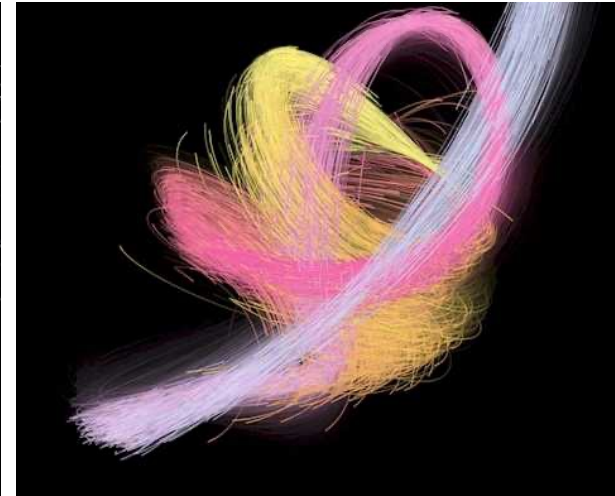
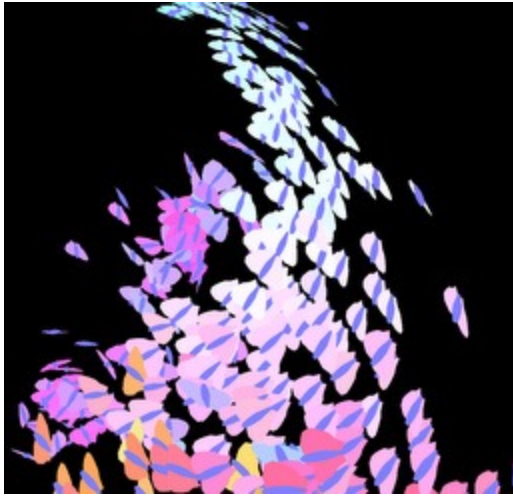
网址: <http://www.codeproject.com/Articles/10003/A-basic-Particles-System>

- FLUIDS v.2 - A Fast, Open Source, Fluid Simulator (2009)

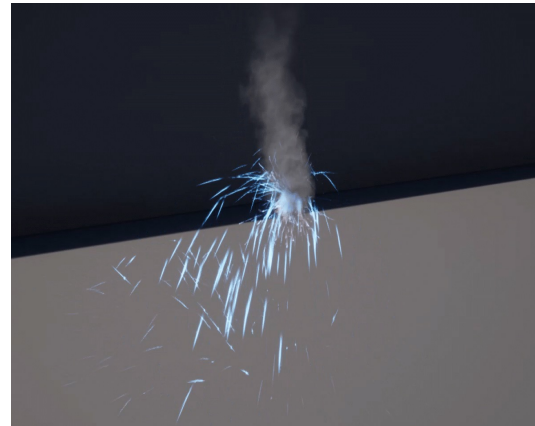
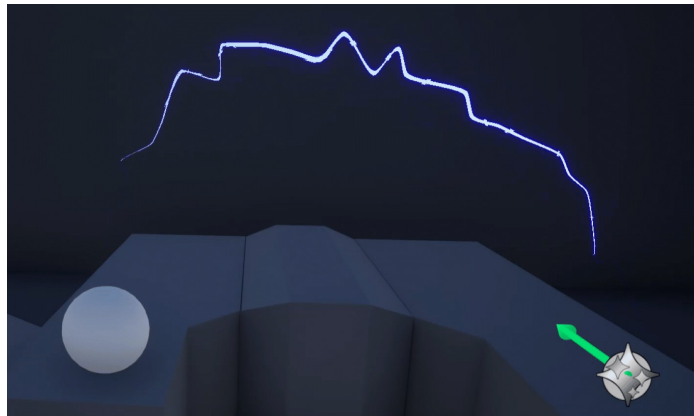
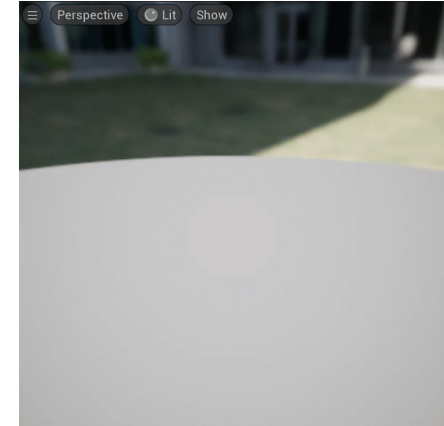
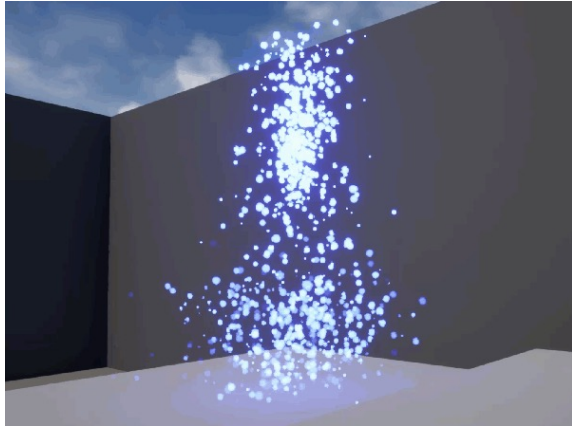
网址: <http://www.rchoetzlein.com/eng/graphics/fluids.htm>



Screenshot



DEMO



进一步拓展

- The **Material Point Method** for Simulating Continuum Materials

物质点法采用质点来离散材料区域，用网格计算空间导数和求解动量方程，兼具拉格朗日和欧拉算法的优势，非常适合模拟涉及材料特大变形和断裂破碎的问题。

Particle (material point) p holds

- Position $\mathbf{x}_p$
- Velocity $\mathbf{v}_p$
- Mass m_p
- Deformation gradient F_p



Jiang, C., Schroeder, C., Teran, J., Stomakhin, A., & Selle, A. (2016). The material point method for simulating continuum materials. In ACM SIGGRAPH 2016 Courses (pp. 1-52).

进一步拓展



Stomakhin, A., Schroeder, C., Chai, L., Teran, J. and Selle, A., 2013. A material point method for snow simulation. ACM Transactions on Graphics (TOG), 32(4), pp.1-10.

DEMO



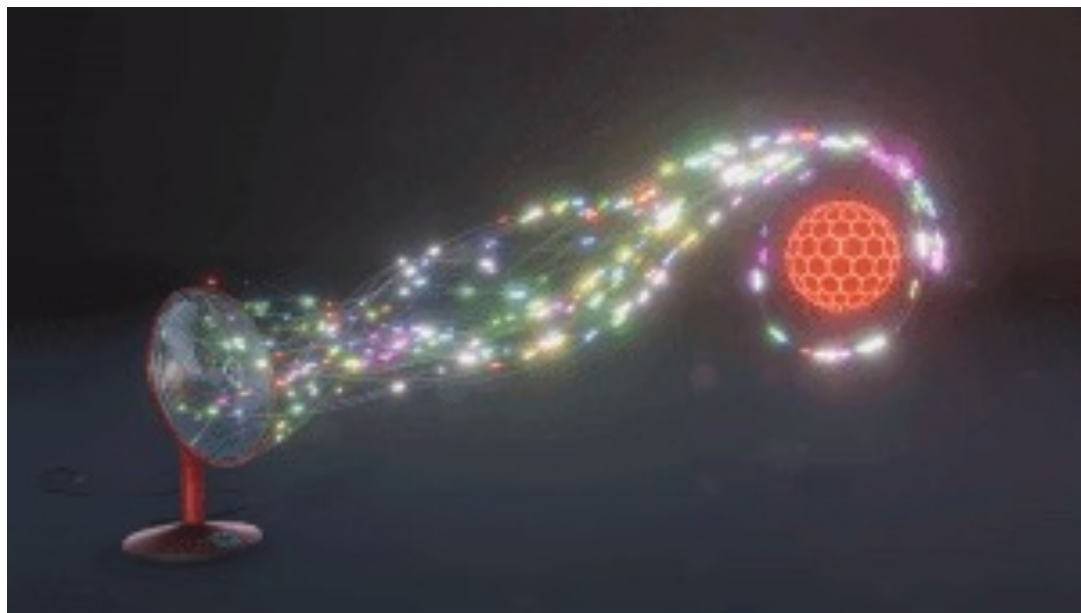
DEMO



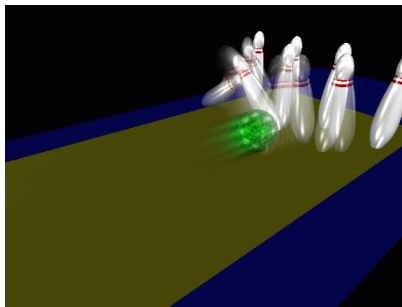
DEMO

总结

- 粒子系统应用**非常广泛**，可模拟火、爆炸、雨雪等很多很多动画特效；
- 是电影电视中常用的特效模拟方法；
- 对于高端应用，有专门的美工设计师；



碰撞检测和处理



蒋浩

Email: jianghao@ict.ac.cn

「碰撞检测」 Collision Detection, CD

- ❑ 碰撞检测是计算动画、计算机游戏和虚拟现实中最基本且非常重要的组成部分。
- ❑ 应用广泛，可用于
 - ❑ 计算机动画
 - ❑ 三维游戏
 - ❑ 基于物理的建模
 - ❑ 飞机和汽车驾驶模拟
 - ❑ 机器人
 - ❑ 路径和运动规划
 - ❑ 装配
 - ❑ 虚拟制造
 - ❑ CAD/CAM

「碰撞处理」

□ 碰撞检测 (Collision Detection)

- ✓ 返回两个或多个物体是否发生碰撞的**布尔判断**
- ✓ **越快越好!**

□ 碰撞确定 (Collision Determination)

- ✓ 找到物体之间的**实际相交位置**

□ 碰撞响应 (Collision Response)

- ✓ 针对两个物体之间的碰撞决定**采取何种操作**

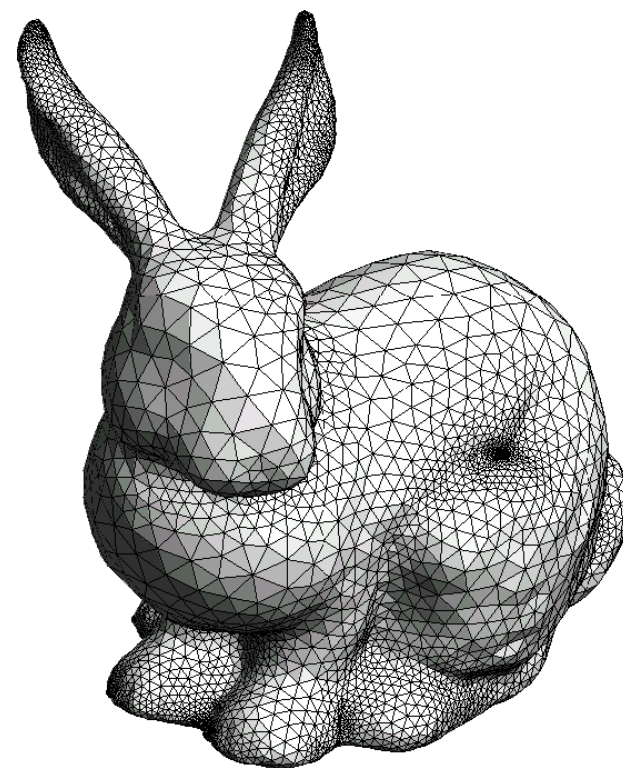
「碰撞检测」 Collision Detection

□ 碰撞检测是非常复杂的，原因如下：

- ✓ 碰撞物体的几何通常相当复杂，需要密集的计算测试
- ✓ 对于 n 个物体，因为每个物体可能与任何其它物体碰撞，所以最直接的解决方法需要 $O(n^2)$ 的时间复杂度

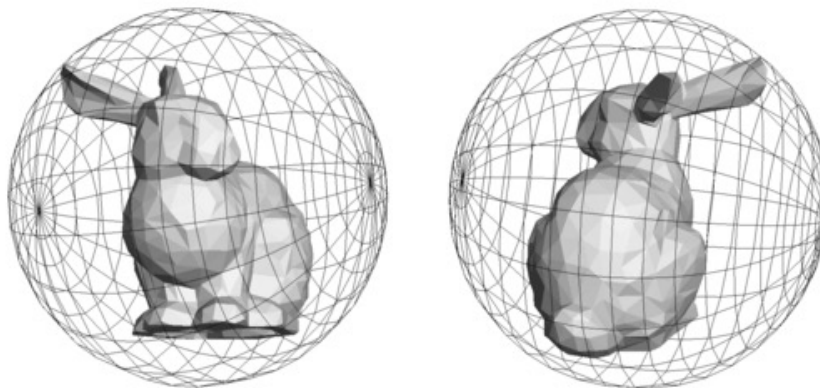
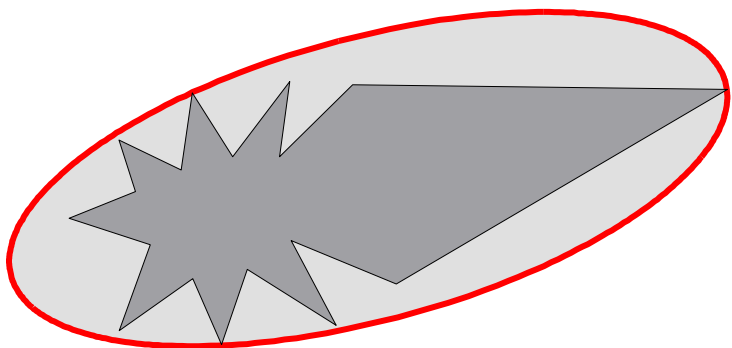
□ 两个研究方向：

- ✓ 复杂的物体必须简化
- ✓ 减少物体测试对的数目



「简化的几何」

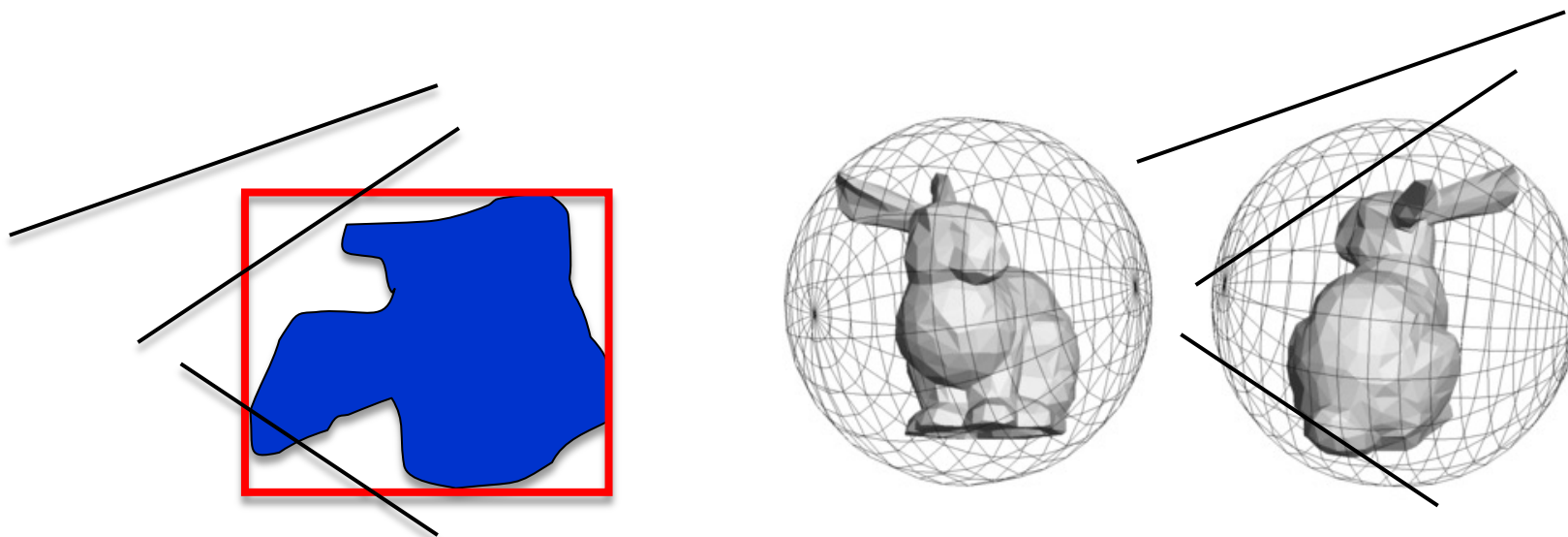
- 用简单的几何物体来逼近复杂的物体，如球体、长方体、椭球体等。
- 简单的形状测试的代价较低。



「包围体」 BV: Bounding Volume

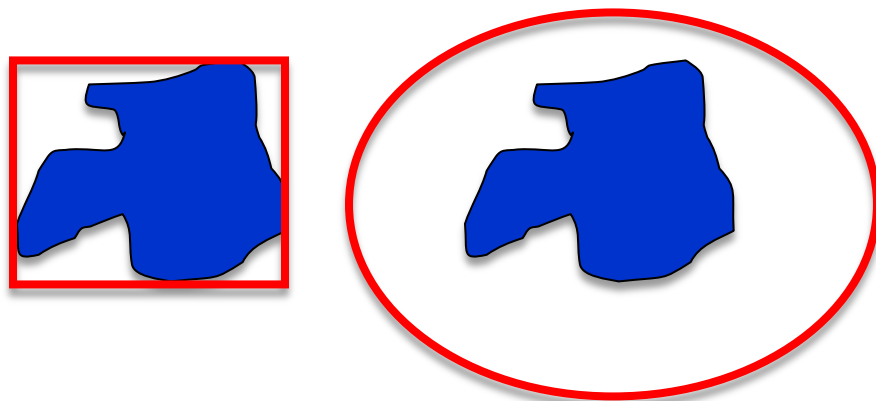
□ 核心思路

- ✓ 用一个简单的包围体把原物体包围起来
- ✓ 如果一个物体与包围体不碰撞，则与包围体内部的物体也不碰撞
- ✓ 通常，若要判断两个物体是否相交，首先测试它们的包围体



「如何选择包围体？」

- 有很多选择，每种选择都是某种**折中**
- **更好的包裹是更好的选择**
 - ✓ 可以剔除一些“假”的相交

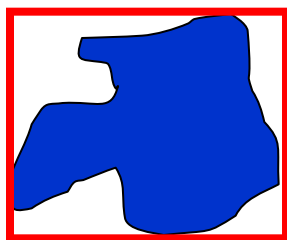


Good

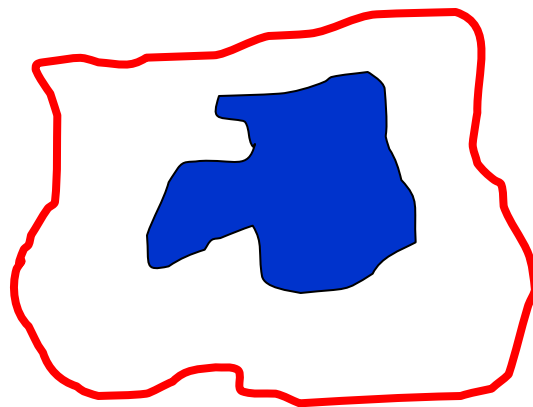
Bad

「如何选择包围体？」

- 有很多选择，每种选择都是某种**折中**
- 更好的包裹是更好的选择
- **物体的形状越简单，效果越好**
 - ✓ 因为简单的形状容易计算



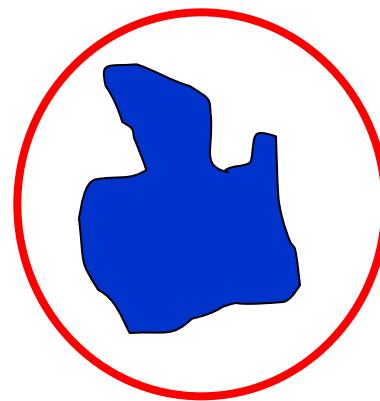
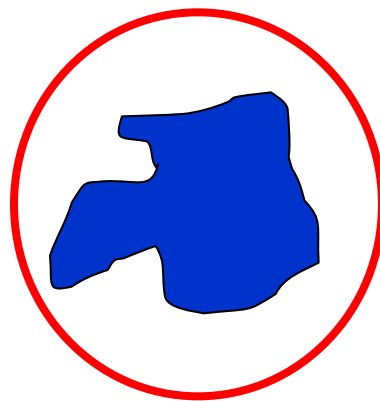
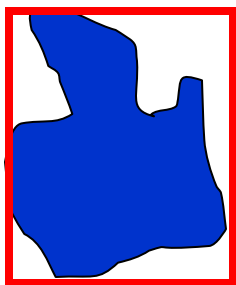
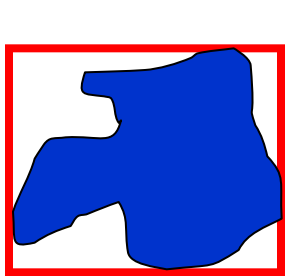
Good



Bad

「如何选择包围体？」

- 有很多选择，每种选择都是某种**折中**
- 更好的包裹是更好的选择
- 物体的形状越简单，效果越好
- **具有旋转不变性的包围体更好**

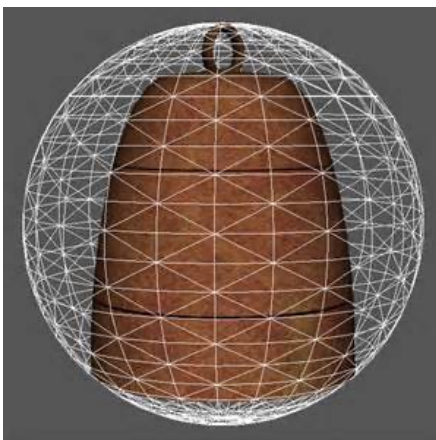
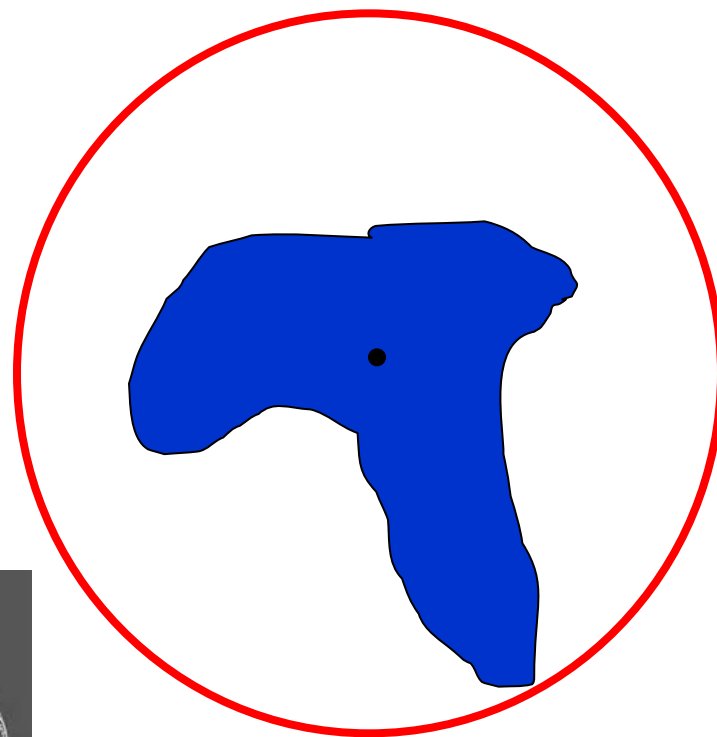


「碰撞检测算法中的图元」

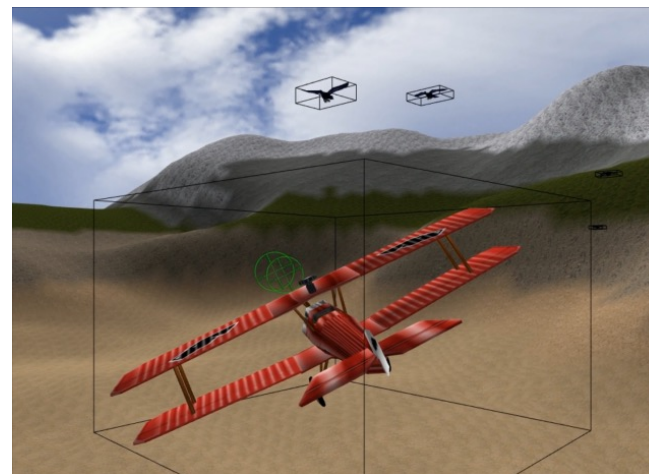
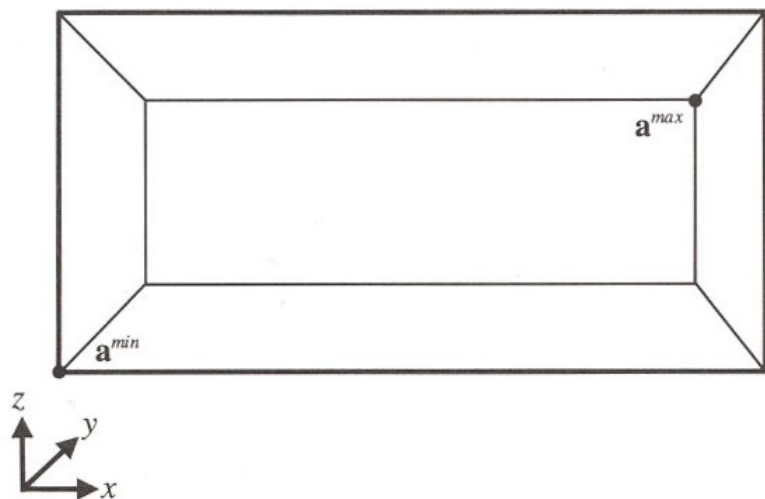
- 碰撞检测算法通常建立在**层次结构**上，最后涉及判断两个图元是否发生碰撞。
- 图元类型：
 - 三角形
 - 球体
 - 轴对齐包围盒 (Axis-Aligned Bounding Box, AABB)
 - 有向包围盒 (Oriented Bounding Box, OBB)
 - 离散有向多面体 (Discrete Oriented Polytope, k -DOP)
 - ...

「常用包围体：球体」 Sphere

- 具有旋转不变性
- 容易计算
- 用中心和半径来表示
- 但包裹得不紧密

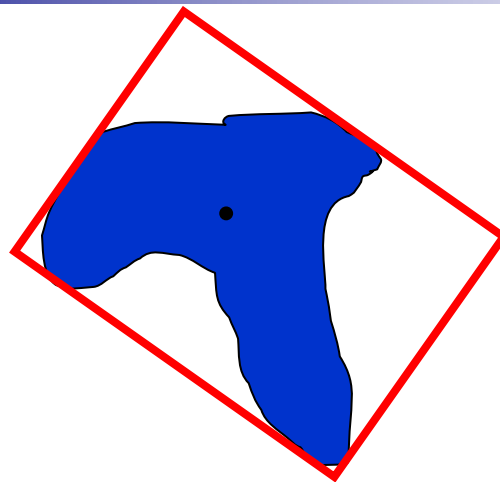
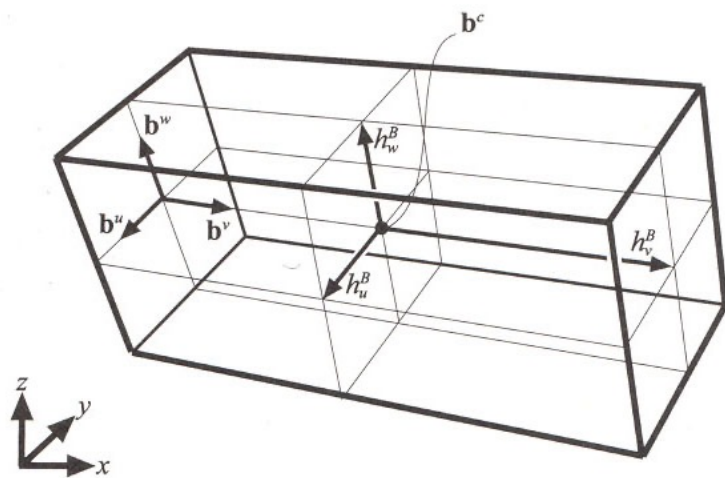


「AABB包围盒的定义」



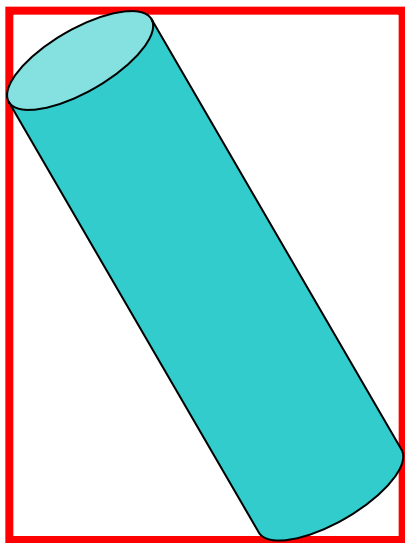
- 轴对齐包围盒(Axis-Aligned Bounding Box, AABB)
 - 是一个表面法向与坐标轴方向一致的长方体。
 - 可以用两个顶点 $a^{\min}$ 和 $a^{\max}$ 来表示物体A的AABB, 其中 $a_i^{\min} < a_i^{\max}, i \in \{x, y, z\}$
- 计算非常快、包裹性中等、物体旋转后必需更新包围盒(除非用一个包裹包围球的包围盒)。

「OBB包围盒的定义」

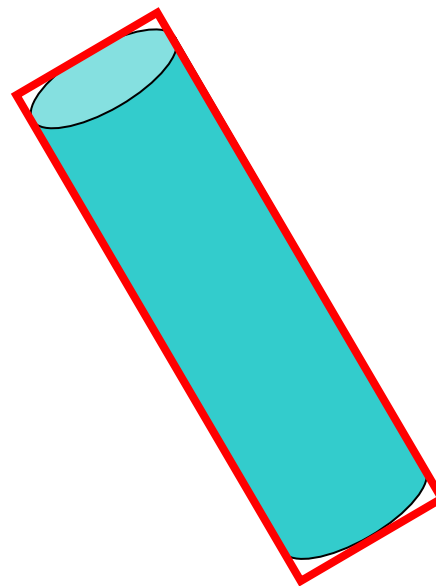


- 有向包围盒(Oriented Bounding Box, OBB)
 - 是一个表面法向两两垂直的长方体。是一个可以任意旋转的AABB
 - 可以用它的中心点 b^c 、三个归一化向量 b^u 、 b^v 、 b^w 以及半边长 h_u^B 、 h_v^B 、 h_w^B 来描述。
- 比AABB更紧密，但计算比AABB慢
- OBB随物体旋转，需更新三个归一化向量

AABB vs OBB



AABB



OBB

「离散有向多面体k-DOP」

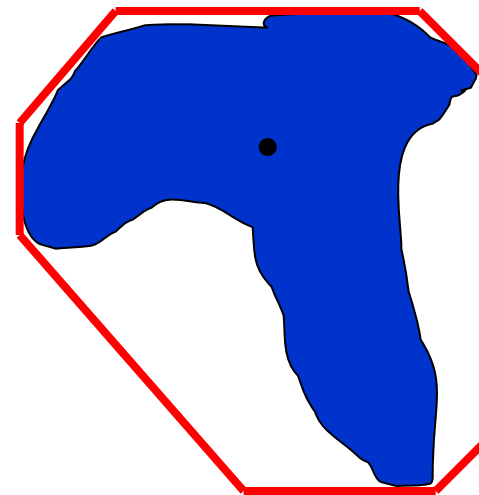
□ **K-DOP: k-discrete oriented polytopes**

➤ **Polyhedron:** 专指三维多面体

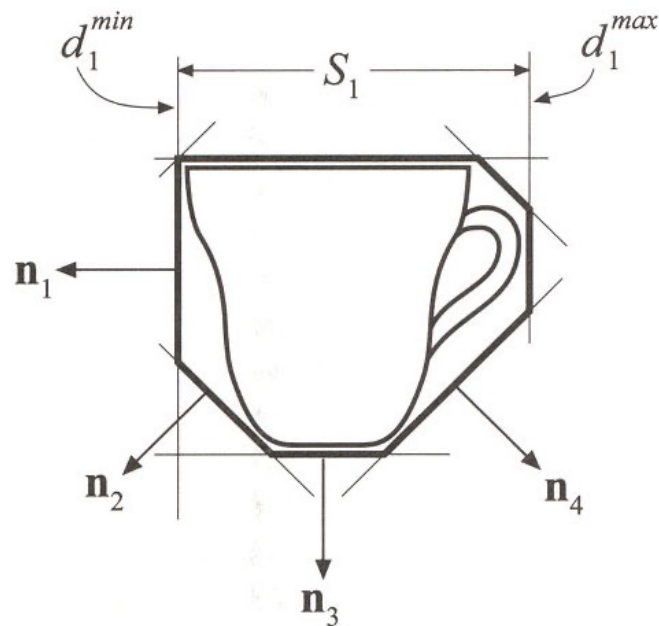
➤ **Polytope:** 任何维中由平面构成的图形

□ 与AABB类似的思想, 但用更多的轴

□ 包裹性比AABB 好, 但计算量更大



「离散有向多面体k-DOP的定义」



一个茶杯的二维8-DOP示例。

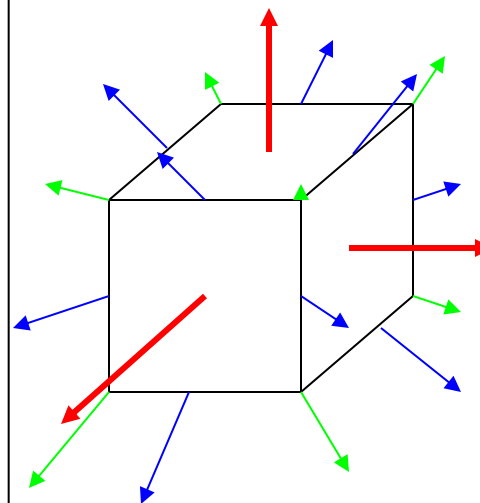
- 由 $k/2$ (k 是**偶数**)个归一化法向 $\mathbf{n}_i$ 来定义 ($1 \leq i \leq k/2$)，每个 $\mathbf{n}_i$ 有两个相关标量值 $d_i^{\min}$ 和 $d_i^{\max}$ ，其中 $d_i^{\min} < d_i^{\max}$ 。每个三元组 $(\mathbf{n}_i, d_i^{\min}, d_i^{\max})$ 描述一个平板层 S_i ，表示两个平面之间的空间。这两个平面是 $\mathbf{n}_i \cdot \mathbf{x} + d_i^{\min} = 0$ 和 $\mathbf{n}_i \cdot \mathbf{x} + d_i^{\max} = 0$ 。所有平板层的交集为 k -DOP的实际空间。

「k-dops轴的选择」

常用的轴: 从一个立方体中心出来的轴:

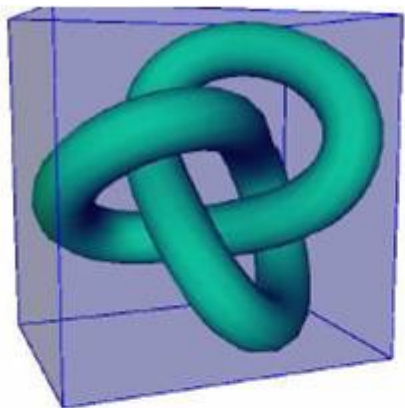
- ❑ Through **faces**: 6-dop
✓ 与AABB一样
- ❑ **Faces**和**vertices**: 14-dop (6+8)
- ❑ **Faces**和**edge centers**: 18-dop (6+12)
- ❑ **Faces**, **vertices**, and **edge centers**; 26-dop (6+8+12)

- ❑ 比26-dop更大并没有多大帮助
- ❑ 实验结果表明, 14 或18-dop具有最好的性能

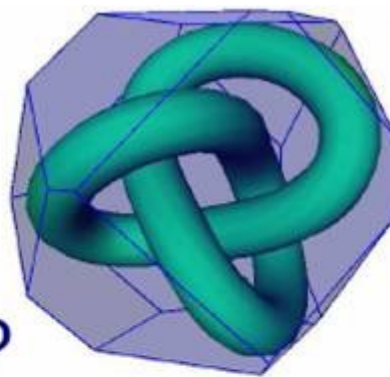


「k-dops中k的选择」

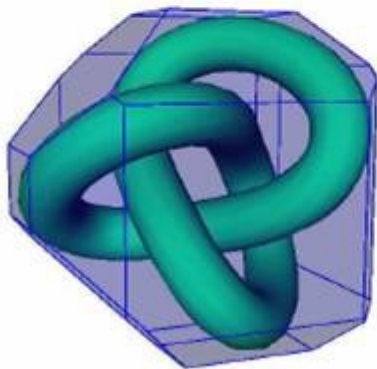
6-DOP
(AABB)



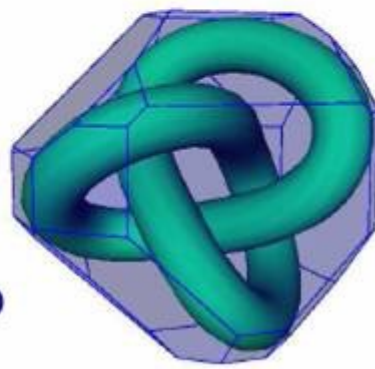
14-DOP



18-DOP

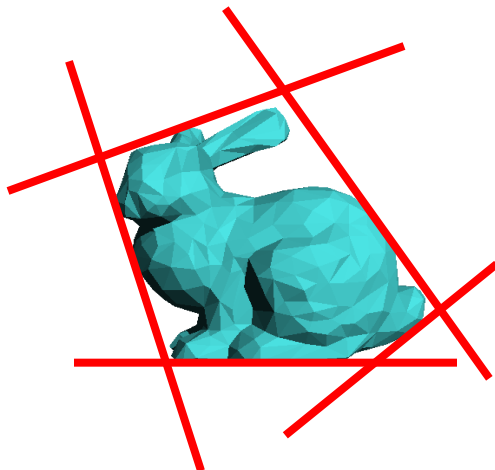
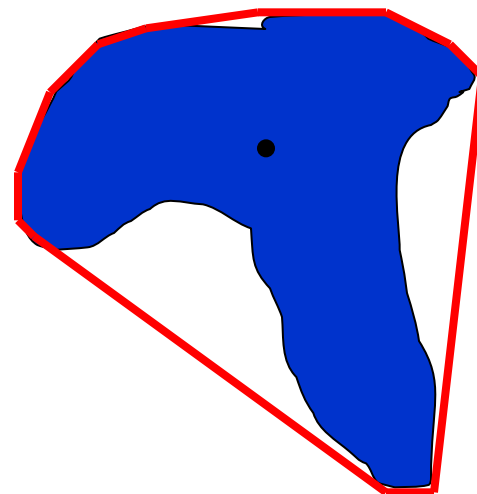


26-DOP

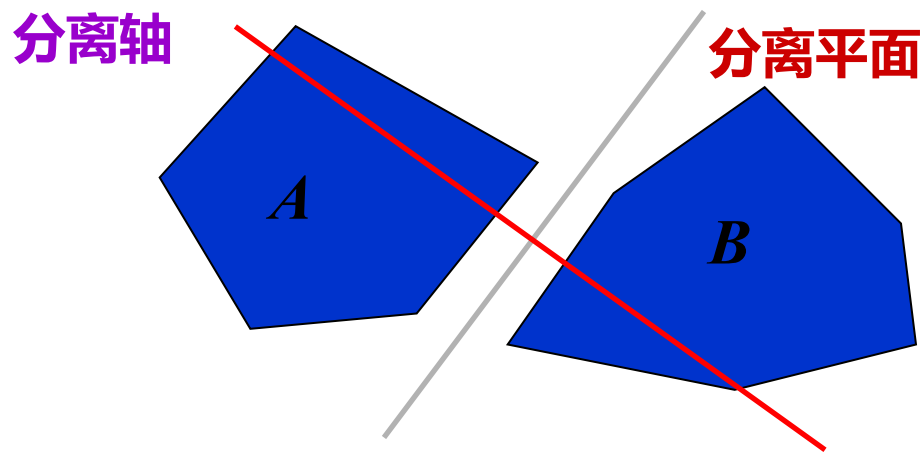


「常用包围体：凸包」 Convex Hull

- 很好的紧密性 (最紧密的凸包围体)
- 存贮：形成包围盒的多边形
- 可以随物体旋转凸包
- 在有些应用非常有效



「分离轴定理」 (Separating Axis Theorem, SAT)



- 对于任意两个互相分离的凸多面体A和B来说，**存在一条分离轴**，其中这两个多面体在这条轴上具有一定间隔而且**在轴上的投影也是互相分开的**。这条轴正交于下列其一(也就是由一个平面分开，该平面平行于下列其中一个平面):
 - A的一个面
 - B的一个面
 - 多面体的一条边
- 注：**这里指的广义上的凸多面体(包括线段和三角形等)

● 相交测试的优化方法

- 排除检测：通过在早期进行一些简单的计算来判断射线或物体是否与其它物体完全分开
- 投影：将三维物体投影到一个最优的正交平面上(xy, xz或yz)，然后在二维平面上进行处理。

● ϵ 问题

- 由于计算精度问题，相交测试常常会用到非常小的数字，用 ϵ 来表示。
- 选择 ϵ 的值：具体问题具体分析。

「包围盒的创建」

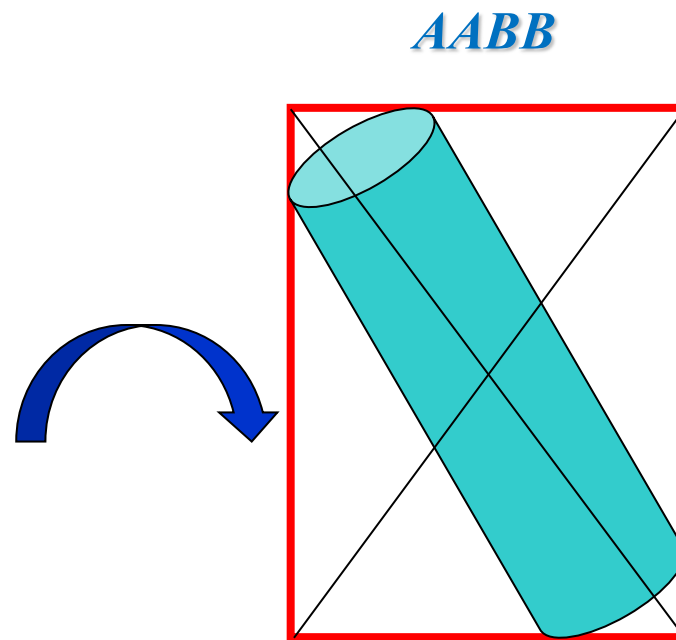
- 给定一组物体，选择合适的包围盒非常重要，因为可以将相交测试的代价最小化。
- 测试任意一条射线与凸体相交的排除过程，通常比直接计算射线与物体的交点要快，同时将表面积最小化可以有效提高相交算法的效率。
- 在碰撞检测算法中，应将**包围体的体积最小化**。

「AABB与k-DOP的创建」

- AABB的创建
 - 沿每条轴线方向分别取给定物体多边形组中顶点的最大值和最小值，就可得到AABB。
- k-DOP的创建
 - 将顶点投影到 k -DOP的每条法线方向 $\mathbf{n}_i$ 上，然后将投影的极小值和极大值分别保存在 $d_i^{\min}$ 和 $d_i^{\max}$ 中，这两个值定义了该方向上最紧密的平板层。所有这些值共同定义了一个最小的 k -DOP。

「包围球体的创建」

- 算法很多，取决于速度和质量之间的折中。
- 恒时单遍算法(Constant Time Single Pass)
 - 先为多边形组构建一个AABB，然后用该AABB的中心和对角线来构造球体包围体。
- 改进：从AABB的中心出发，将它作为球体包围盒的中心，然后再次遍历所有的顶点，找到距离最远的顶点，最后将最远距离最为球的新半径。
- Ritter的近似最优包围盒方法：先找到x、y、z轴线上最小和最大的顶点，然后从这三组顶点中找出距离最大的一组顶点，取这组的中心为球心，取它们之间距离的一半为半径。然后，遍历其它顶点，计算顶点到球心之间的距离 d ，如果这个顶点在球体半径 r 以外，则将球心朝着这个顶点方向移动 $(d-r)/2$ ，同时将半径大小修正为 $(d+r)/2$ 。重复上述过程，直到把不在这个球体内的顶点和旧的球体全部包含在新的球体内。



「OBB的创建」

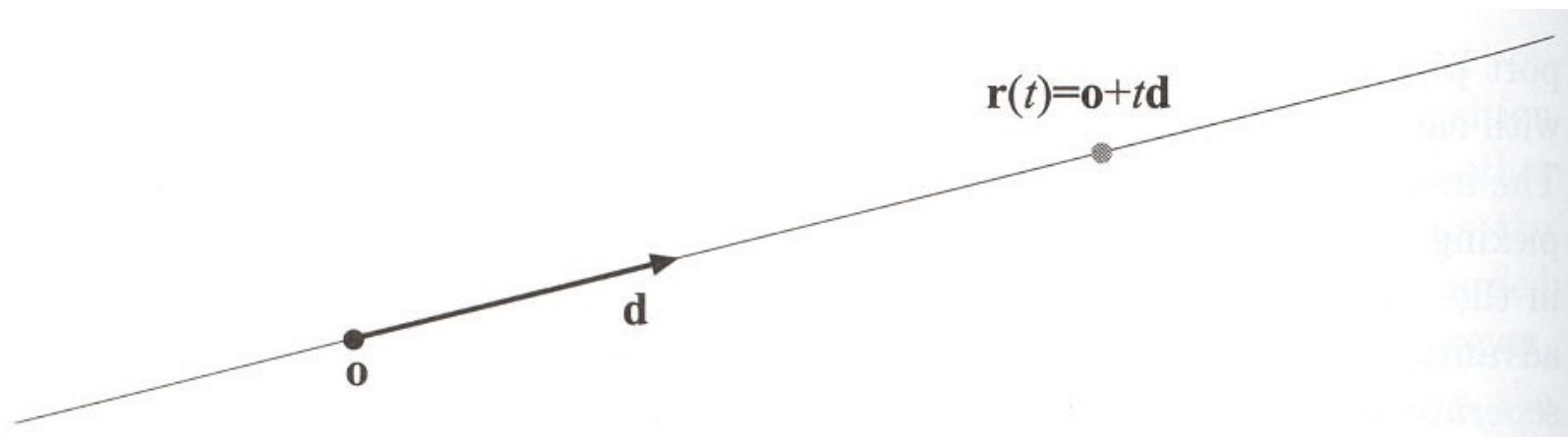
- Gottschalk方法：
 - 思想：首先从物体的凸包计算出一个方向，然后找到紧密贴合物体的OBB。
 - 凸包计算方法：QuickHull方法。计算复杂度为 $O(n\log n)$ 。
 - 方向计算方法：计算凸包的协方差矩阵，求其特征向量并将其归一化就是所求包围盒的方向向量。
 - OBB的中心和半边长计算方法：将凸包上的点投影到方向向量上，并求每个方向上的最大值和最小值。每个方向最大值和最小值的平均值决定了该方向的中心，最大值减去最小值的一半为该方向的半边长。

「求交测试的重要规则」

- 这些规则可以使得相交测试更快速、稳定、精确。
 - 尽可能利用简单的比较和计算来排除或确认各种相交类型，从而避免进一步的计算。
 - 尽可能充分利用上次的测试结果。
 - 如果使用了多种排除或确认测试，试着改变它们之间的内部顺序，这有可能产生一个更有效的测试结果。
 - 尽量搁置开销量大的计算，如三角函数、平方根、除法等。
 - 降低维数。（如把三维降为二维、二维降为一维）。
 - 尽量找出在相交测试之前可预先完成的计算。
 - 当相交测试比较复杂时，尽量先利用物体的包围球进行初步排除。
 - 上机实践进行计时比较，计时要采用真实的数据和测试环境。
 - 尽量使程序具有比较好的鲁棒性。能处理各种特殊情况，对浮点精度误差不敏感。

「定义和工具」

■ 射线及其参数



一条射线及其参数: $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$, 其中 $\mathbf{o}$ 为原点, $\mathbf{d}$ 是射线方向(归一化为单位矢量), t 为参数

「表面的定义」

- 隐式表面 $f(\mathbf{P}) = f(p_x, p_y, p_z) = 0$

- 例子: 中心在原点, 半径为 r 的球

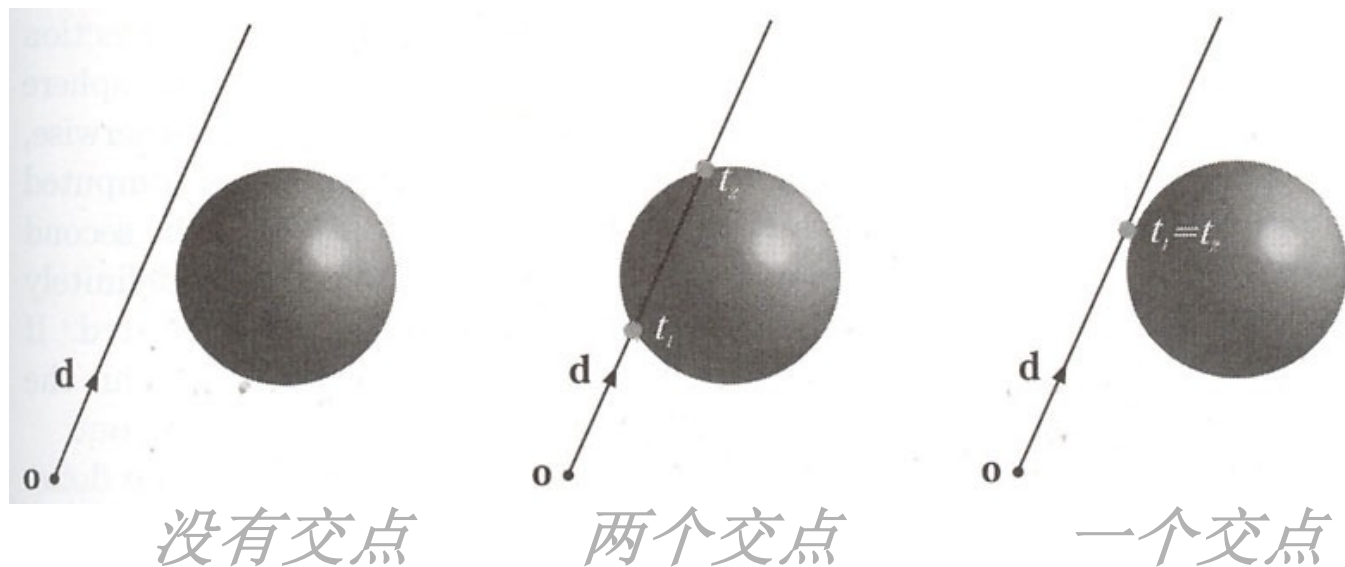
$$p_x^2 + p_y^2 + p_z^2 - r^2 = 0$$

- 显式表面

- 三角形 $\triangle \mathbf{v}_0 \mathbf{v}_1 \mathbf{v}_2$ 表示为显示形式

$$\mathbf{t}(u, v) = (1 - u - v)\mathbf{v}_0 + u\mathbf{v}_1 + v\mathbf{v}_2$$

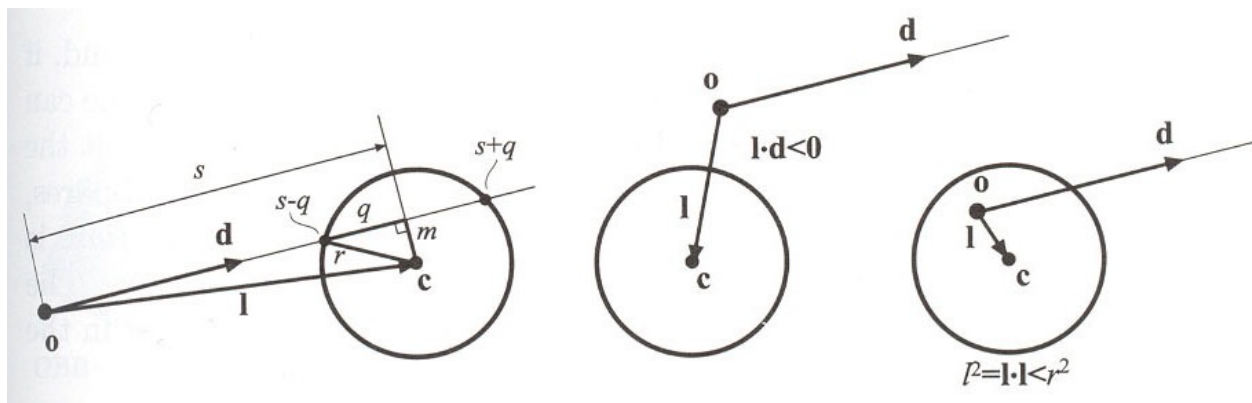
射线/球体相交测试



- 数学方法

- 把射线的参数方程代入球体的隐式方程，得到一个一元二次方程，利用方程的判别式进行无根判别和根的计算。

射线/球体相交测试



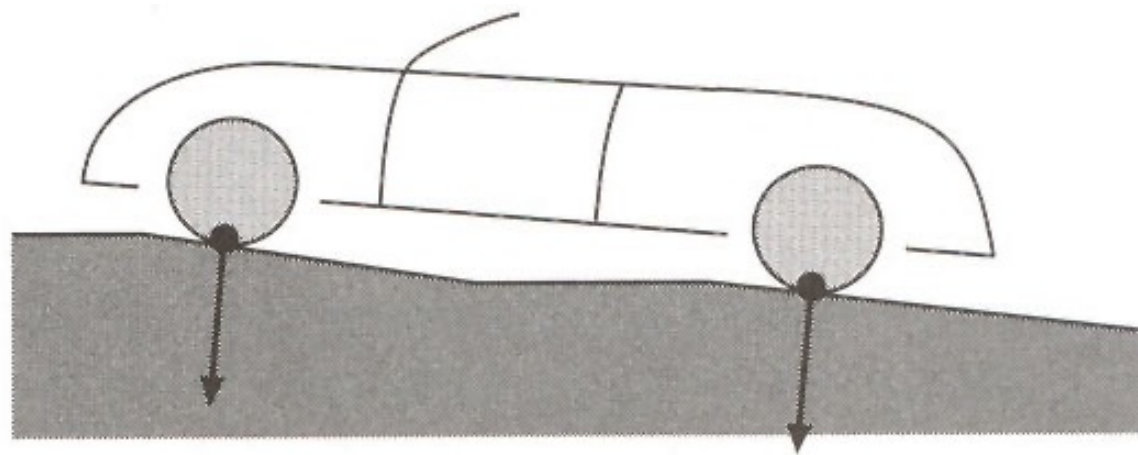
• 优化方法

■ 不需要对射线原点 o 后的相交进行计算。

- 计算射线原点 o 到球体中心的向量 $\mathbf{l} = \mathbf{c} - \mathbf{o}$ ，其平方长度为 $l^2 = \mathbf{l} \cdot \mathbf{l}$ 。如果 $l^2 < r^2$ ，则说明射线原点位于球体内部（见上图右），从而可以肯定这条射线肯定与球体相交。如果只需知道它们之间是否会相交，则结束，否则继续。
- 计算 $\mathbf{l}$ 在射线方向的投影： $s = \mathbf{l} \cdot \mathbf{d}$ 。如果 $s < 0$ 而且射线原点位于球体之外，则球体位于射线原点的后面，没有交点。（见上图左）
- 利用勾股定理计算球心与投影点之间的平方距离： $m^2 = l^2 - s^2$ 。如果 $m^2 > r^2$ ，则射线与球体无交，否则肯定有交点。

使用射线进行碰撞检测

- 是一种特定环境下的快速碰撞检测技术，实际应用效果非常好。
- 例子：一辆小汽车沿着斜坡向上行驶，可利用路上的信息（路面形成的图元）来引导汽车向上行驶。但对于游戏等应用，并不需要这种复杂的碰撞检测。可使用一组射线来近似运动物体。



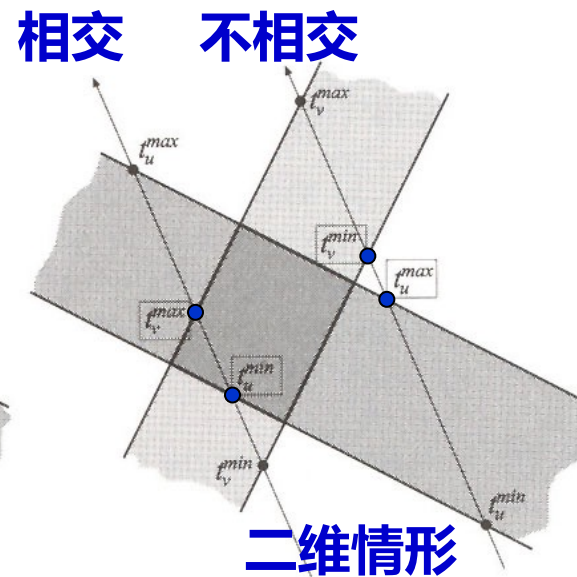
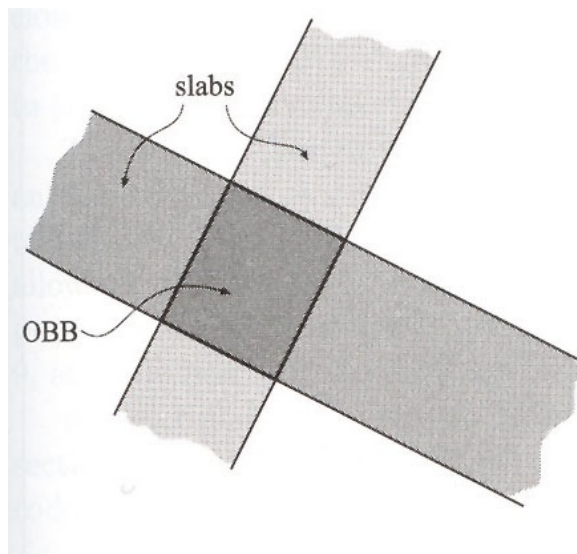
不是计算整辆测车子与环境（路面）之间的碰撞，而是在每个轮子上引出一条射线，然后对这些射线与环境进行相交测试。

使用射线进行碰撞检测

- 假设小汽车一开始在一个平面上，4个轮子是小汽车唯一与环境接触的地方。可在4个轮子上各生成一条射线，射线的起始点位于轮子与环境之间的接触点上，然后对轮子上的射线与环境进行相交测试。
- 如果从射线原点到环境的距离为0，则轮子在地面上；如果这个距离大于0，则轮子与环境没有接触；如果距离为负，则轮子陷在环境中。
- 可用这些值来计算碰撞响应：负距离可使小汽车（所在轮子处）向上运动，正距离可使小汽车向下运动。
- 为了避免射线在两个方向进行搜索，可将测试射线的原点向后移动直到它在环境包围盒的外面，然后再对这条射线和环境进行相交测试。

射线/平板层相交法(可以处理AABB、OBB、K-DOP等情况)

平板层(Slab)为两个相互平行的平面。



- 将长方体看成3个平板层。计算射线与平板层的交点。对于每个平板层，存在一个最小的 t 值和最大的 t 值，分别记做 t_i^{min} 和 t_i^{max} ， $i \in \{u, v, w\}$ ，计算

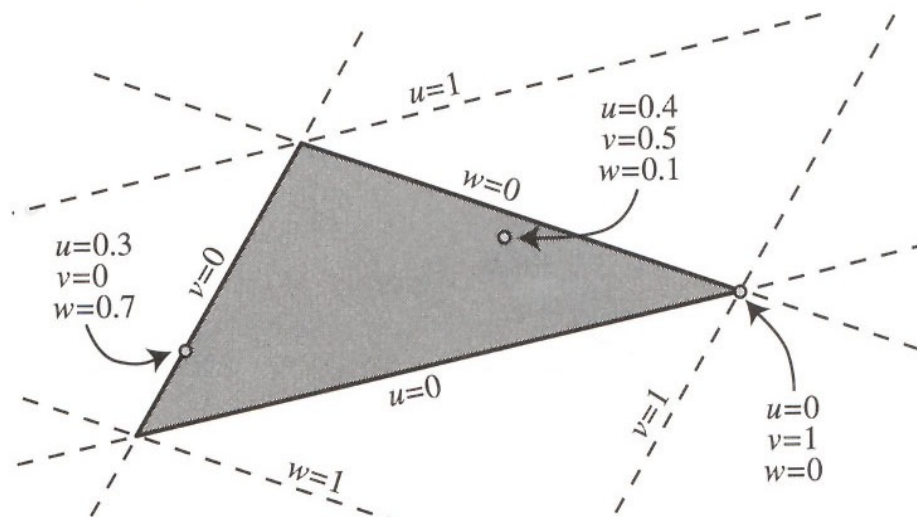
$$t_{min} = \max(t_u^{min}, t_v^{min}, t_w^{min})$$

$$t_{max} = \min(t_u^{max}, t_v^{max}, t_w^{max})$$

然后就可以进行巧妙测试：如果 $t_i^{min} \leq t_i^{max}$ ，射线与长方体相交，否则不相交。

射线/三角形相交测试

- 原理：首先计算出射线与三角形所在平面的交点，然后将交点和三角形顶点投影到一个轴对齐平面（ xy , yz 或 xz ）上，并使得投影三角形面积最大化。
- 将三维问题变为二维问题，从而只需判断二维点是否在二维三角形内部。
- 三角形上的点 $t(u,v)$ 可以由下面的公式给出：
 $t(u,v) = (1-u-v)\mathbf{v}_0 + u\mathbf{v}_1 + v\mathbf{v}_2$ ，其中 (u,v,w) 为重心坐标， (u,v,w) 在 $(0,1)$ 之间则点在内部



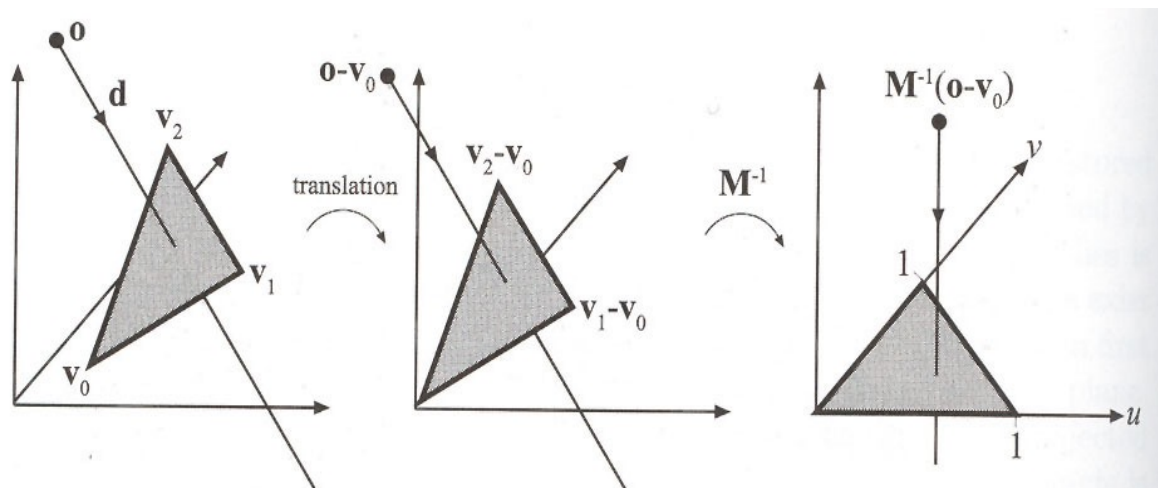
三角形的重心坐标

射线/三角形相交测试

- 射线 $\mathbf{r}(t)$ 与三角形 $\mathbf{t}(u,v)$ 之间的相交计算等同于对方程 $\mathbf{r}(t) = \mathbf{t}(u,v)$ 的求解，得到线性方程组：

$$(-\mathbf{d} \quad \mathbf{v}_1 - \mathbf{v}_0 \quad \mathbf{v}_2 - \mathbf{v}_0) \begin{pmatrix} t \\ u \\ v \end{pmatrix} = \mathbf{o} - \mathbf{v}_0$$

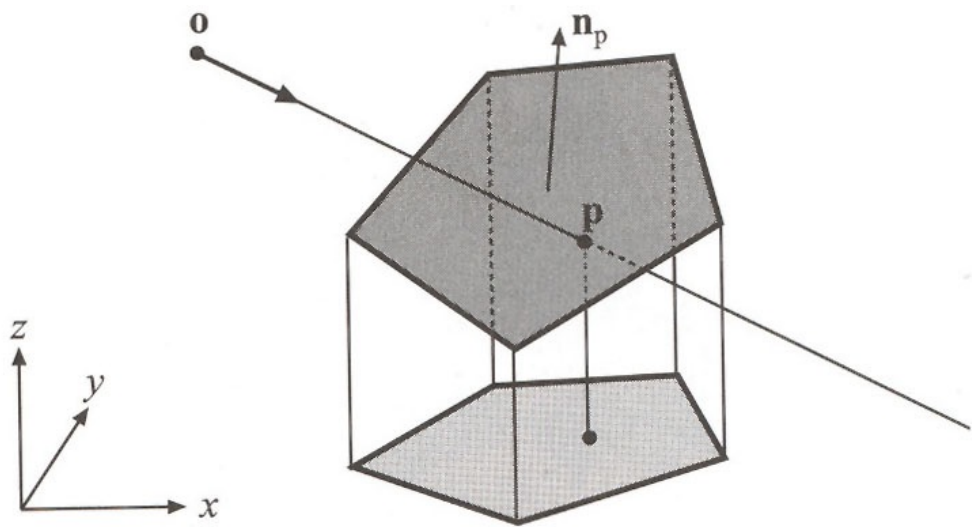
从而得到交点的重心坐标 (u, v) 和 t 值。



几何解释：把三角形平移到原点位置，然后在 y 方向和 z 方向将这个三角形变换为单位三角形，同时将射线方向与 x 轴方向对齐。

射线/多边形相交测试

- 首先计算射线与多边形所在平面方程的交点
- 将交点和多边形的所有顶点投影到 xy , yz 或 xz 平面上（使得投影面积最大：利用平面的法向来判断，若 x 分量最大，则舍弃 x 分量，投影到 yz 平面），从而将三维降为二维。
- 在二维空间，判断交点是否包含在二维的多边形中。

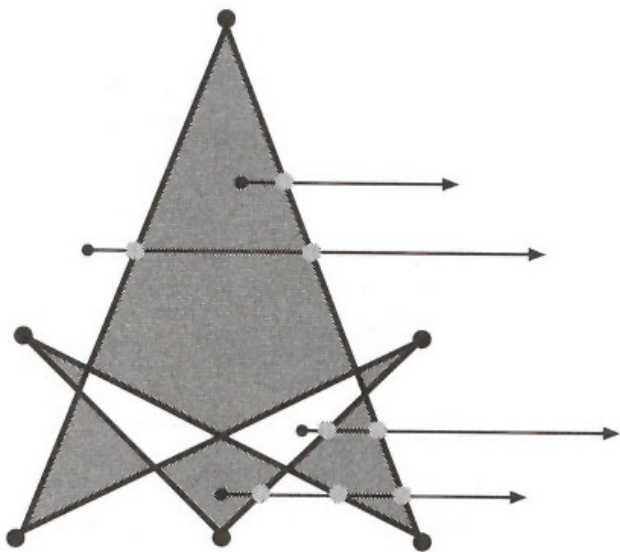


多边形顶点和交点 P 在 xy 平面上的正交投影（在 xy 平面上的投影面积最大）

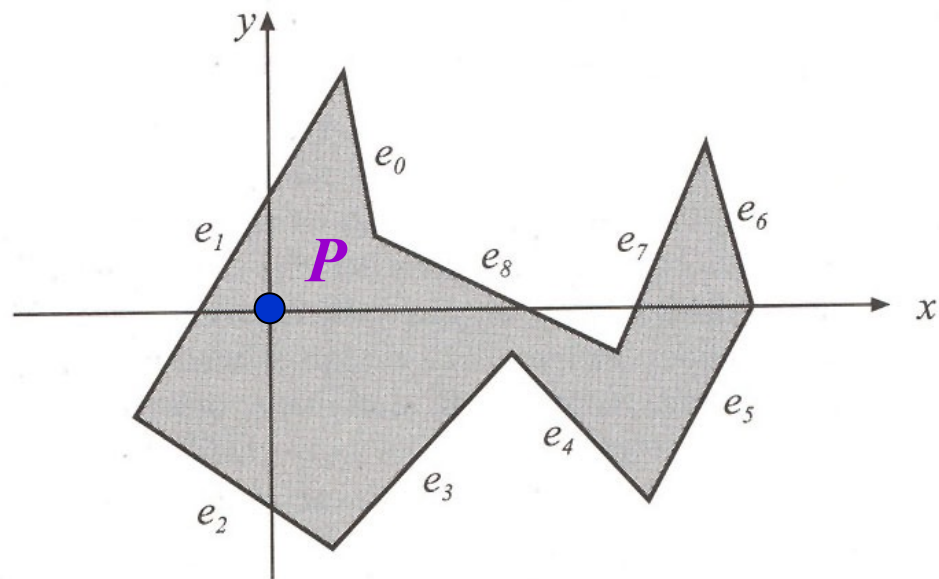
点是否在二维多边形内部的 Crossing Test

- 基于Jordan 定理

- 一条射线从某点出发，若与多边形的交点个数为奇数个，则点在多边形内部，否则在多边形的外部。



自交的凹多边形：并非所有被多边形包围的部分就一定是多边形的内部。



加速方法：将多边形平移- P , 这样通过计算与x轴正向的交叉点数量就可以确定 P 是否位于多边形内部。

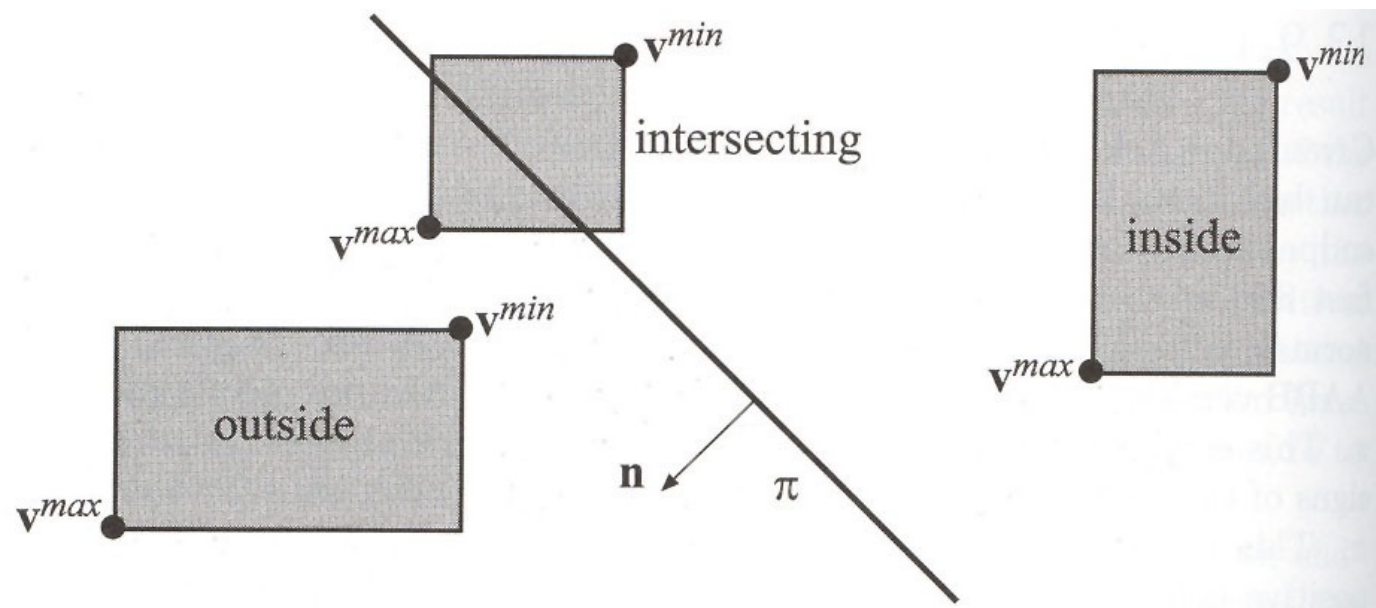
平面/长方体相交测试

- 要确定长方体与平面 $\Pi: \mathbf{n} \cdot \mathbf{x} + d = 0$ 之间是否相交，一种方法是将长方体的顶点代入该平面方程。如果所有的结果有正有负（或者为零），则表明这些顶点位于平面的两侧，因此存在相交关系。
- 针对AABB和OBB有更快的方法。
- 基本思想：只需要将两个顶点代入平面方程，这两个顶点构成了这个长方体的对角线。所取的对角线通过长方体的中心，而且与平面的法向 $\mathbf{n}$ 对齐程度最好。

平面/ AABB相交测试

- 给定一个由 $\mathbf{b}^{\min}$ 和 $\mathbf{b}^{\max}$ 定义的AABB，记做 B ，基于它可以生成4条不同的对角线，这些对角线都通过 B 的中心。
 - 首先找出哪条对角线与这个平面的法向最接近。
 - 然后将这条对角线的两个顶点代入平面方程中，从而可以测试出每个顶点位于平面的哪一侧。
 - 如果求解的结果符号相反，或者至少一个为0，则AABB与平面相交。
- 注意: 如果AABB位于平面的正半空间或负半空间，则无交。如果 $\mathbf{v}^{\min}$ 位于平面的正半空间，则 $\mathbf{v}^{\max}$ 一定位于正半空间。

平面/ AABB相交测试



3个AABB对于给定平面的 $v^{\min}$ 和 $v^{\max}$ ，如果这对顶点位于平面的同一侧，则AABB与平面不相交，否则相交。注意：可先检测 $v^{\min}$ ，如果发现它与平面法向位于同一侧，则说明AABB“在外面”（即位于该平面的正半空间）。

“三角形/三角形” 相交测试

- 三角形是图形硬件最重要的绘制单元。在碰撞检测算法的最底层，一般会有一个程序来判断两个三角形是否相交。
- 通常仅关心它们之间是否会发生碰撞，而不需要具体的交点信息。
- 给定两个三角形 $T_1 = \Delta \mathbf{u}_0 \mathbf{u}_1 \mathbf{u}_2$ 和 $T_2 = \Delta \mathbf{v}_0 \mathbf{v}_1 \mathbf{v}_2$ ，它们分别位于平面 Π_1 和 Π_2 上，需要判断它们之间是否相交
 - 区间重叠法
 - ERIT方法

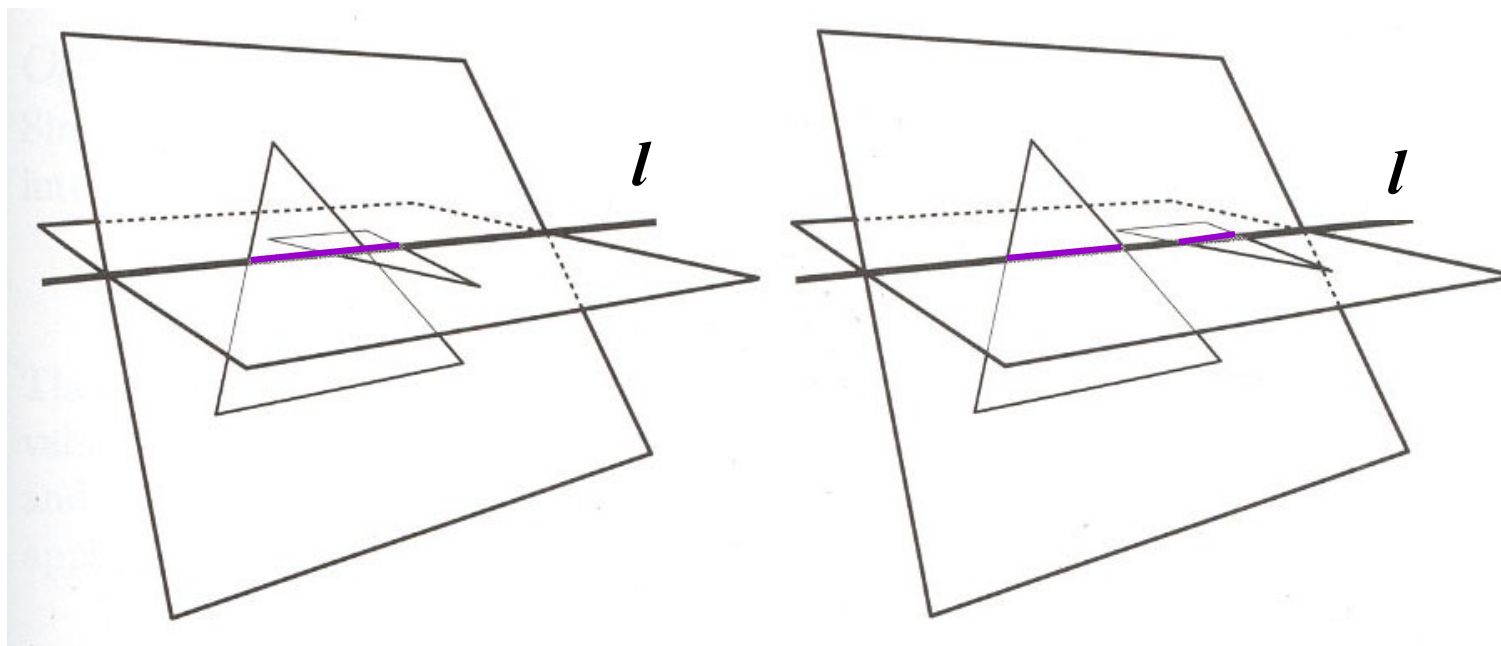
区间重叠法(Interval Overlap Method)

- 早期排除

- 计算平面方程 Π_2 ，并把三角形 T_1 的顶点代入该平面方程，得到三角形 T_1 的顶点到平面 Π_2 的符号距离
- 若所有的距离不等于0，且这些值具有相同的符号(说明三角形 T_1 位于平面 Π_2 的一侧)，则不相交。
- 对三角形 T_2 和平面方程 Π_1 ，进行同样的测试。

这两种早期排除测试可以节省很多计算量！！！！

- 判断交线上是否含有两个三角形的公共部分
 - 如果所有的距离都等于零，则这两个三角形共面，需单独处理。
(投影到二维空间进行处理)
 - 否则，平面 Π_1 和 Π_2 之间的相交部分应该是一条直线，可表示为 $\mathbf{l}=\mathbf{o}+t\mathbf{d}$ 。
 - 可能发生的情形分两种，见下图



两个三角形及其各自所在的平面。

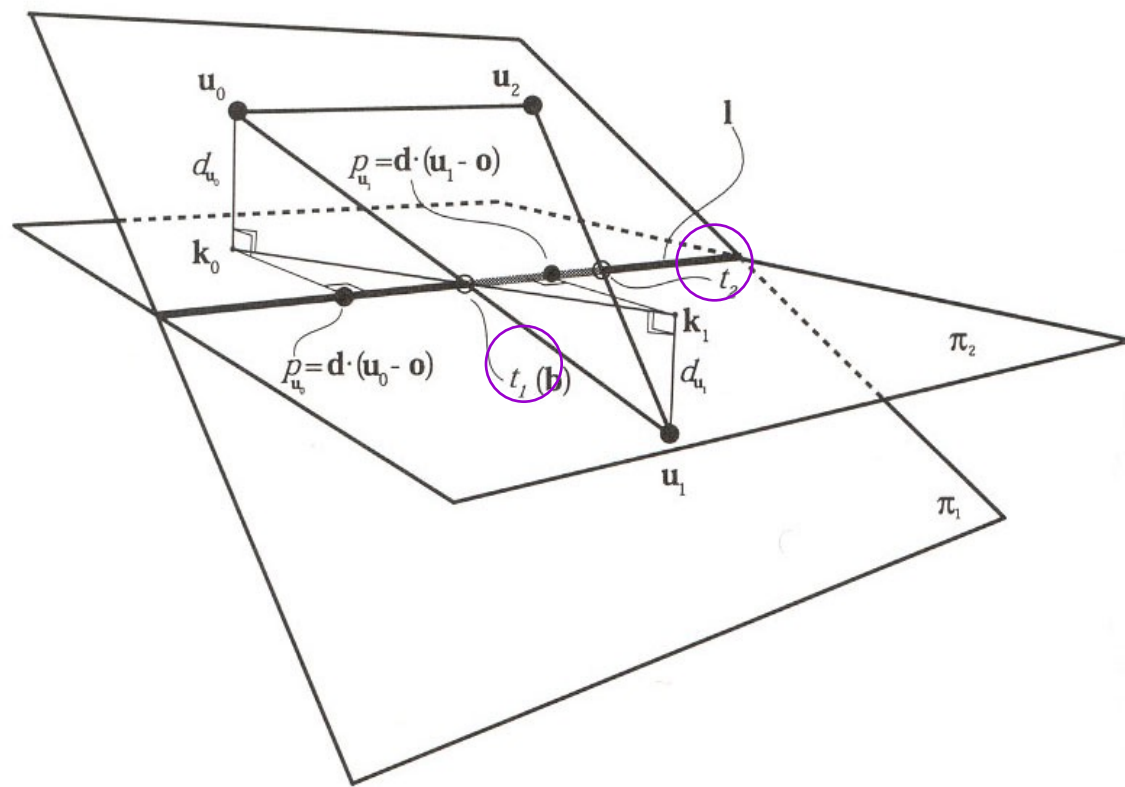
左图：与两个重叠三角形一样，沿直线 l 上的区间也相互重叠

右图：三角形之间不相交，直线上的区间也没有重叠

相交区间计算

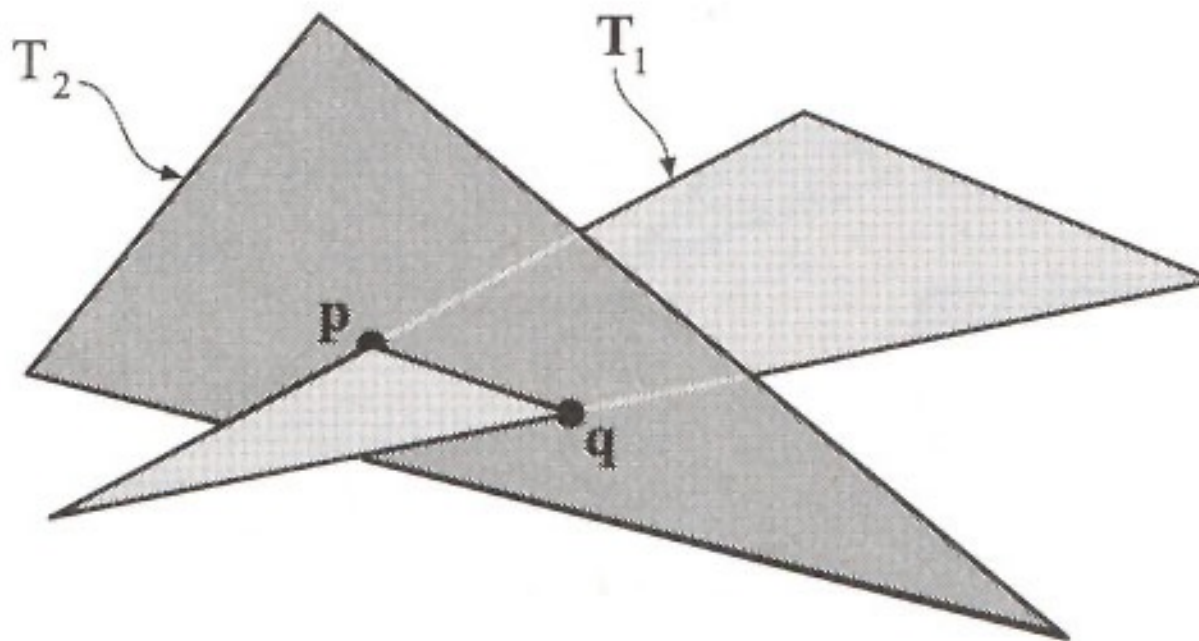
- 假设需要计算 T_1 和 l 之间的相交区间。
- 假设三角形的两个顶点 u_0 和 u_2 位于平面 Π_2 的同侧，而另外一个点 u_1 位于平面的另一侧。
- 利用几何关系可以算出直线 l 与边 u_0u_1 之间的交点 t_1 ，直线 l 与边 u_1u_2 之间的交点 t_2 。 $[t_1, t_2]$ 为三角形 T_1 和 l 之间的相交区间。
- 同理可以算出三角形 T_2 和 l 之间的相交区间。
- 如果区间相互重叠，则这两个三角形一定相交。否则不相交。

两个三角形之间的相交示意图



u_i 是三角形 T_1 的顶点， π_1 和 π_2 是三角形 T_1 和 T_2 所在的平面， d_{ui} 是从 u_i 到平面 π_2 的有向距离， k_i 是 u_i 在平面 π_2 上的投影点， p_{ui} 是 u_i 在两个平面交线 l 上的投影点。

ERIT方法



首先计算三角形 T_1 和 T_2 的两个交点 p 和 q ，如果 p 与 q 之间的线段完全包含在 T_2 中，或者与 T_2 的某条边相交，则这两个三角形相交，否则不相交。

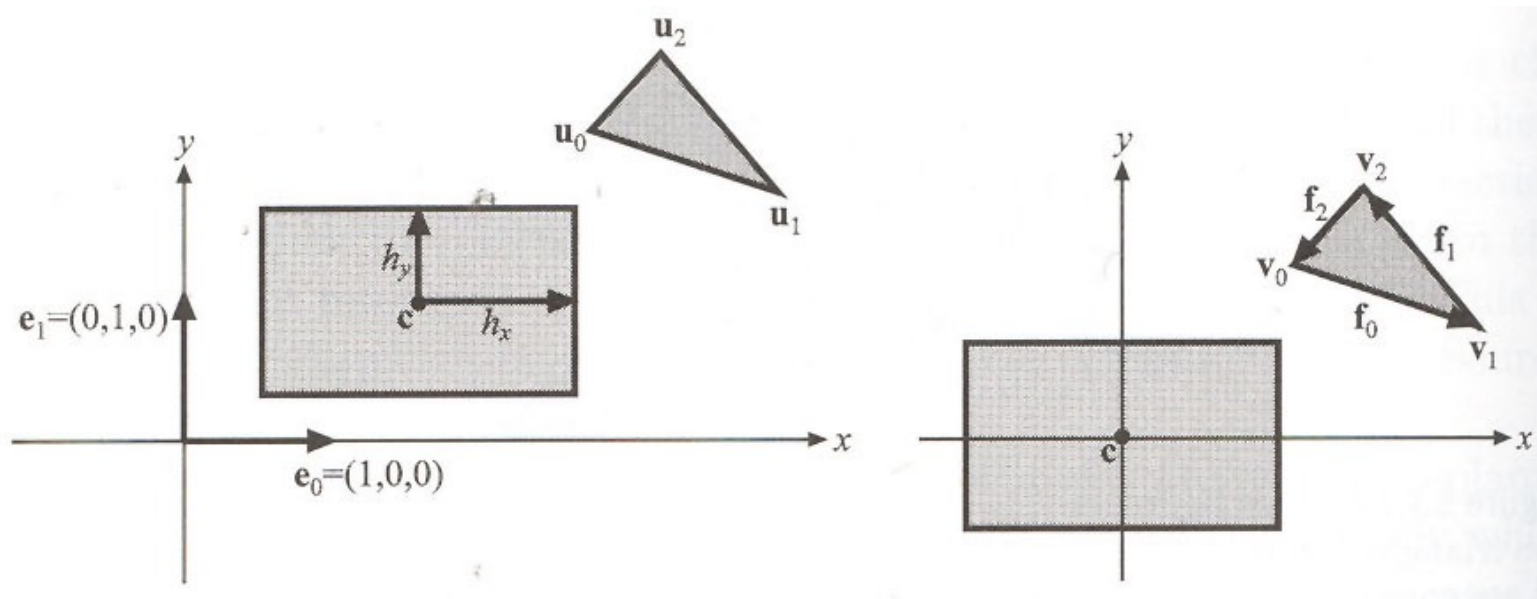
算法描述

1. 计算出三角形 T_2 所在平面的方程
2. 如果三角形 T_1 上的所有点均在三角形 T_2 的同一侧，则不相交。
3. 如果两个三角形共面，则使用区间重叠法进行处理。
4. 计算出三角形 T_1 和平面 Π_2 之间的交线，这条线位于平面 Π_2 上。
5. 如果这条线段与三角形 T_2 相交，或者完全包含在 T_2 中，则 T_1 和 T_2 相交，否则不相交（可简单地计算 p, q 的面积坐标）。

“三角形/长方体” 重叠测试

- 可用于
 - 体素空间的创建
 - 三角形与长方体的碰撞检测
 - 多边形与规范化视域体(Canonical View Volumes) 的相交测试
- 我们将介绍Akenine-Moller提出的三角形/AABB的重叠测试方法（对OBB需要先变换）
- 原理：基于分离轴定理

“三角形/长方体” 重叠测试中的符号表示



左图：为长方体和三角形在坐标系中原始位置

右图：对长方体和三角形进行了平移，使得长方体的中心与坐标系的原点重合

三角形/AABB的重叠测试

- 基于分离轴定理，对下面13条轴线进行测试
 - [3个测试] AABB的三个轴 $e_0=(1,0,0)$, $e_1=(0,1,0)$, $e_2=(0,0,1)$, 分别是AABB的3条法线
 - [1个测试] 三角线的法线 n 。只对与三角形法向最接近的长方体对角线的两个顶点进行测试
 - [9个测试] AABB的三个轴与三角形三条边的叉积
- 在测试中，一旦找到分离轴，算法即可中止，返回没有相交信息。如果通过所有测试，则三角形与长方体重叠。

BV/BV相交测试

- 完全包含物体的封闭体称为包围体(BV)。BV的主要目的是为了提供更加简便的相交测试和更加有效的预先排除。
- 例如，如果要测试两辆汽车是否相撞，可先计算各自的BV，然后测试两个BV是否发生重叠。如果没有重叠，则可以确保这两辆汽车不会相撞(这是最常见的情况)，而不必对汽车上的每个图元进行测试，从而节省计算量。
- 最常见的包围体：球体、AABB、 k -DOP、OBB

球体/球体相交测试

- 计算出两个球体球心距离
- 如果这个距离大于两个球体的半径之和，则不相交；否则，相交。

球体/长方体相交测试：

- 主要思想：在AABB上找到距离球心最近的一个点。用球心对AABB的每条轴进行一次一维测试。
- 对每条轴，用球心对AABB的该轴边界进行测试。如果在球心AABB的边界之外，则计算球心与AABB在这条轴的距离的平方。
- 在三条轴线全部计算完之后，将这些距离的平方和相加，并与球体半径的平方进行比较。
- 如果平方距离和小于球体半径平方，则最近点位于球体内部（相交）；否则不相交。
- 将球心坐标变换到OBB的轴上去，即可处理OBB

球体/长方体相交测试伪代码（带加速）：

```
bool SphereAABB_intersect( $c, r, A$ )  
returns({OVERLAP, DISJOINT});  
1:  $d = 0$   
2: for each  $i \in \{x, y, z\}$   
3:   if  $((e = c_i - a_i^{\min}) < 0)$   
4:     if  $(e < -r)$  return (DISJOINT);  
5:      $d = d + e^2$ ;  
6:   else if  $((e = c_i - a_i^{\max}) > 0)$   
7:     if  $(e > r)$  return (DISJOINT);  
8:      $d = d + e^2$ ;  
9:   if  $(d > r^2)$  return (DISJOINT);  
10: return (OVERLAP);
```

AABB/AABB相交测试

- 因AABB与轴线对齐，所以只需要两个点就可以描述这个体。由于AABB非常简单，因此通常将它应用于碰撞检测算法中。
- 思想：每个轴上区间是否相交

```
Bool AABB_intersect(A,B)
returns( {OVERLAP, DISJOINT} );
for each  $i \in \{x,y,z\}$ 
    if(  $a_i^{\min} > b_i^{\max}$  or  $b_i^{\min} > a_i^{\max}$  )
        return(DISJOINT);
return(OVERLAP);
```


k-DOP/k-DOP相交测试

- 一个 **k -DOP**是一个凸多面体，各个面由 **k** 个固定的法向确定，其中法向以外的半空间不作为包围体一部分来考虑。
- **AABB**实际上是一个特殊的**6-DOP**。当 **k** 值增大时，包围体越来越接近凸包，从而更接近物体。
- **Klosowski**等指出，对于适当的 **k** 值，两个 **k -DOP**相交测试速度要比两个**OBB**的相交测试速度快一个数量级。
- 假设两个 **k -DOP**分别为**A**和**B**，需要对它们进行相交测试，这时需对每一对平板层 (S_i^A, S_i^B) 进行重叠测试，而 $S_i = S_i^A \cap S_i^B$ 是一个非常容易解决的一维区间重叠测试。
- 如果在任何情况下 S_i 为空集，则这两对平板层相互分离，测试过程结束。如果所有 S_i 均不为空集，则可以确定这两个 **k -DOP**相交。

k-DOP/k-DOP相交测试算法描述

- KDOP_Intersect($d_1^{A,\min}, \dots, d_{k/2}^{A,\min}, d_1^{A,\max}, \dots, d_{k/2}^{A,\max},$
 $d_1^{B,\min}, \dots, d_{k/2}^{B,\min}, d_1^{B,\max}, \dots, d_{k/2}^{B,\max}$)

returns ({OVERLAP, DISJOINT});

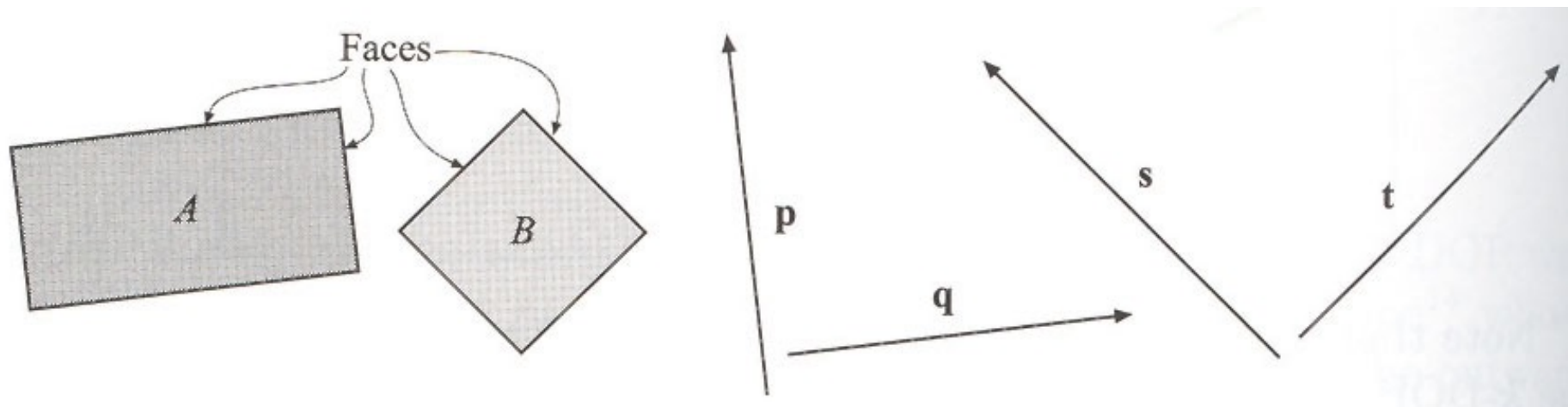
for each $i \in \{1, 2, \dots, k/2\}$

 if ($d_i^{B,\min} > d_i^{A,\max}$ or $d_i^{A,\min} > d_i^{B,\max}$)

 return (DISJOINT);

return (OVERLAP) ;

“OBB/OBB” 相交测试

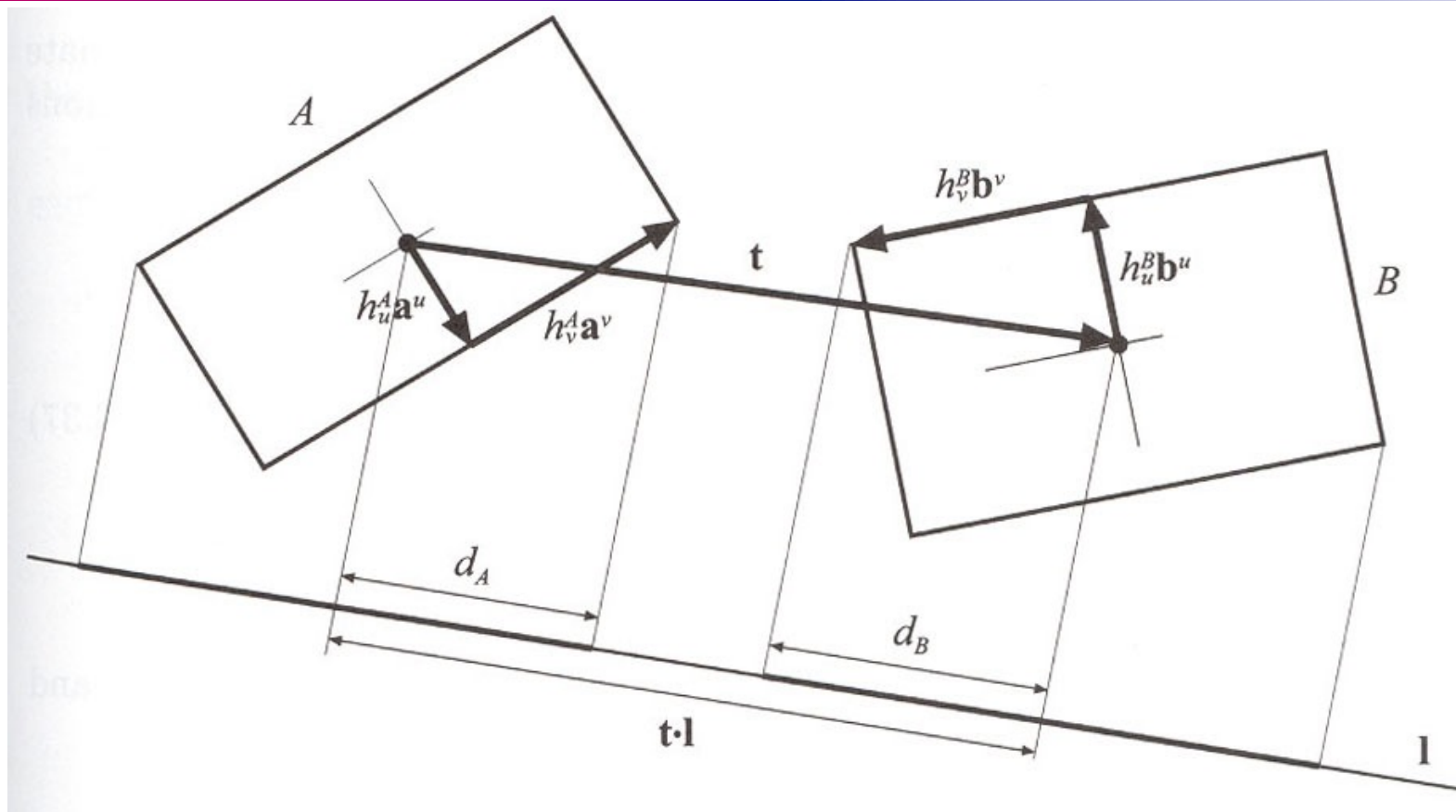


二维形式

为了判断两个OBB是否发生重叠，可使用分离轴定理。因为分离轴应与A和B的表面正交，可选 p, q, s, t 为分离轴，然后将OBB投影到这些轴上。如果两个OBB在**所有**这些轴上的投影相互重叠，则两者相交，否则不相交。

因此，为了知道两个OBB是否重叠，只需要找到一条可以使A和B的投影不重叠的轴线即可。在本例中，**q轴**是唯一能将这两个投影分开的轴。

“OBB/OBB” 相交测试



分离轴定理示意。由于A和B在轴线 l 上的投影的“半长”没有重叠，因此两个OBB不相交。

“OBB/OBB” 相交测试

- 对于两个OBB(A和B), 根据分离轴原理, 需要对15条轴线进行测试: 3条取自A的表面, 3条取自B的表面, 另外9条取自A和B边的叉乘组合。

- 定理: 当且仅当l是一条分离轴时, 在这条轴线上的投影才彼此分离, 不重叠:

$$| \mathbf{t} \cdot \mathbf{l} | > d_A + d_B$$

- 如果在这15个测试中有一个成立, 则这两个OBB不相交。
- 最好在开始真正的OBB/OBB测试之前, 用包围球进行快速排除。

“线/线” 相交测试

- 两条二维直线的相交测试

- 假设两条直线分别为

$$\mathbf{r}_1(s) = \mathbf{o}_1 + s\mathbf{d}_1, \text{ 梅 } \mathbf{r}_2(s) = \mathbf{o}_2 + s\mathbf{d}_2$$

- 则交点为:

$$s = \frac{(\mathbf{O}_2 - \mathbf{O}_1) \bullet \mathbf{d}_2^\perp}{\mathbf{d}_1 \bullet \mathbf{d}_2^\perp},$$

$$t = \frac{(\mathbf{O}_1 - \mathbf{O}_2) \bullet \mathbf{d}_1^\perp}{\mathbf{d}_2 \bullet \mathbf{d}_1^\perp}$$

两条二维线段的相交计算

- 假设两条线段分别为

$$\mathbf{r}_1(s) = \mathbf{p}_1 + s(\mathbf{p}_2 - \mathbf{p}_1), \text{ 梅 } \mathbf{r}_2(s) = \mathbf{q}_1 + s(\mathbf{q}_2 - \mathbf{q}_1)$$

- 则交点为:

$$s = \frac{-\mathbf{c} \bullet \mathbf{a}^\perp}{\mathbf{b} \bullet \mathbf{a}^\perp} = \frac{\mathbf{c} \bullet \mathbf{a}^\perp}{\mathbf{a} \bullet \mathbf{b}^\perp} = \frac{d}{f},$$

$$t = \frac{\mathbf{c} \bullet \mathbf{b}^\perp}{\mathbf{a} \bullet \mathbf{b}^\perp} = \frac{e}{f},$$

$$\text{猴濒 } \mathbf{a} = \mathbf{q}_2 - \mathbf{q}_1, \mathbf{b} = \mathbf{p}_2 - \mathbf{p}_1, \mathbf{c} = \mathbf{p}_1 - \mathbf{q}_1$$

伪
代
码

```
if ( $f > 0$ )  
    if ( $d < 0$  or  $d > f$ ) return NO_INTERSECATION  
    else  
        if ( $d > 0$  or  $d < f$ ) return NO_INTERSECATION
```

三个平面之间的相交测试

- 给定3个平面，法向量为 $\mathbf{n}_i$ ，平面上各有一个任意点 $\mathbf{p}_i$ ，则其交点为：

$$\mathbf{p} = \frac{(\mathbf{p}_1 \bullet \mathbf{n}_1)(\mathbf{n}_2 \times \mathbf{n}_3) + (\mathbf{p}_2 \bullet \mathbf{n}_2)(\mathbf{n}_3 \times \mathbf{n}_1) + (\mathbf{p}_3 \bullet \mathbf{n}_3)(\mathbf{n}_1 \times \mathbf{n}_2)}{\begin{vmatrix} \mathbf{n}_1 & \mathbf{n}_2 & \mathbf{n}_3 \end{vmatrix}}$$

- 当三个法向行列式为0时，有平面平行，无交。

动态相交测试

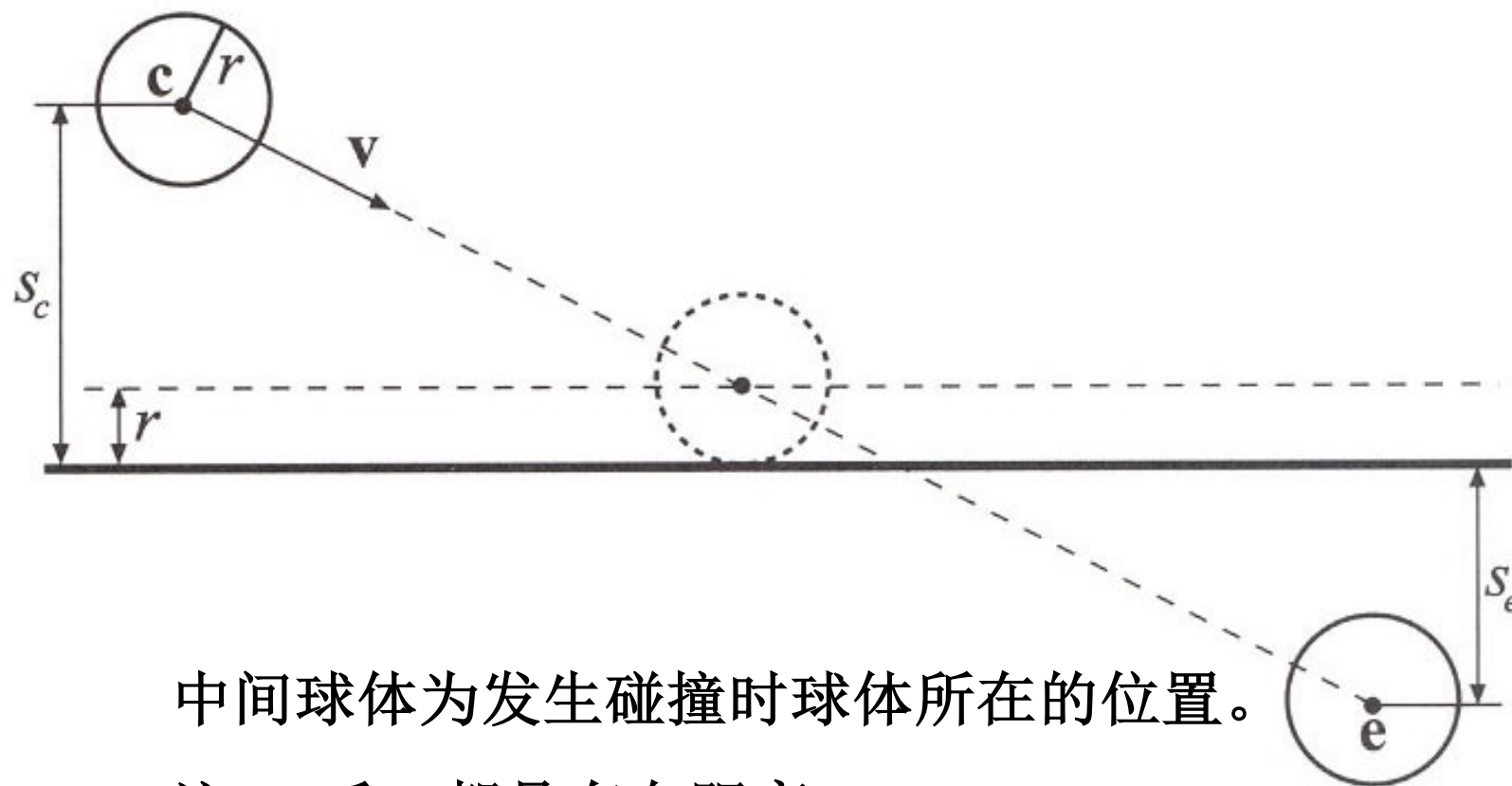
- 动机

- 在动态场景中，物体在时间轴上是连续运动，用离散的方式(t , $t+\Delta t, \dots$)，很可能漏掉相交情形。虽然可以通过加密时间间隔进行相交测试，但仍会错过部分相交测试，而且还会增加计算负荷。因此，需要设计动态相交测试来处理这种情况。

- 动态相交测试的运动相对原则

- 运动是相对的。
- 假设A和B均在运动，则可看成A是运动的（当然速度需改变），B是静止的。

“球体/平面” 动态相交测试



中间球体为发生碰撞时球体所在的位置。

注： s_c 和 s_e 都是有向距离

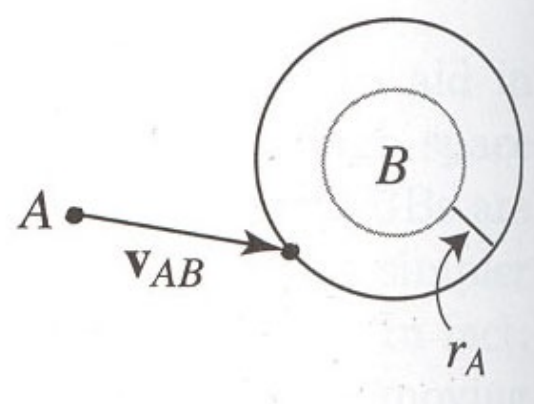
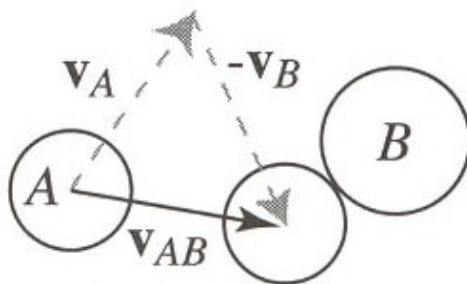
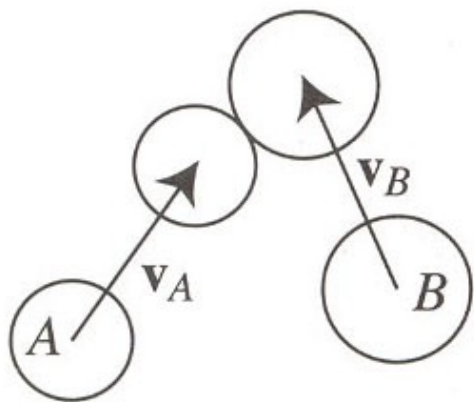
“球体/平面” 动态相交测试

- 假设球体的中心为 $\mathbf{c}$ ，半径为 r ，在 Δt 内物体匀速直线运动，则球心在一条直线上。
- 计算球心和平面之间的有向距离 s_c 和 s_e
- 如果球心位置在平面的同一侧(即 $s_c s_e > 0$)，而且 $|s_c| > r$ 和 $|s_e| > r$ ，则球体不会与平面相交。
- 否则需要计算相交的准确时刻和球体位置。

$$t = \frac{s_c - r}{s_c - s_e}$$

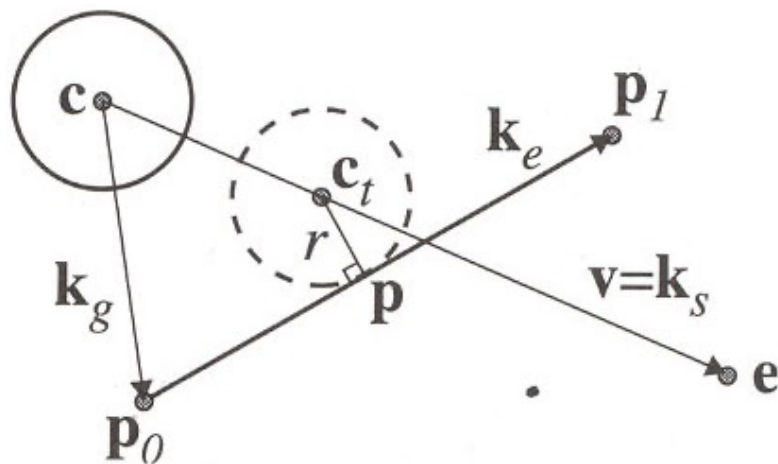
“球体/球体” 相交测试

- 两个球体间的相交测试等同于一条射线与静止球体之间的相交测试。
 - 首先，使用相对运动原则，把球体B看成是静止的。
 - 这里，可以使一个球在另外一个球的表面移动，从而得到一个新的球体，半径为两个球体的半径之和。
 - 假设球体B是新的球体且处于静止状态，半径是两个球体之和，则球体A就是沿着直线运动的一个点。

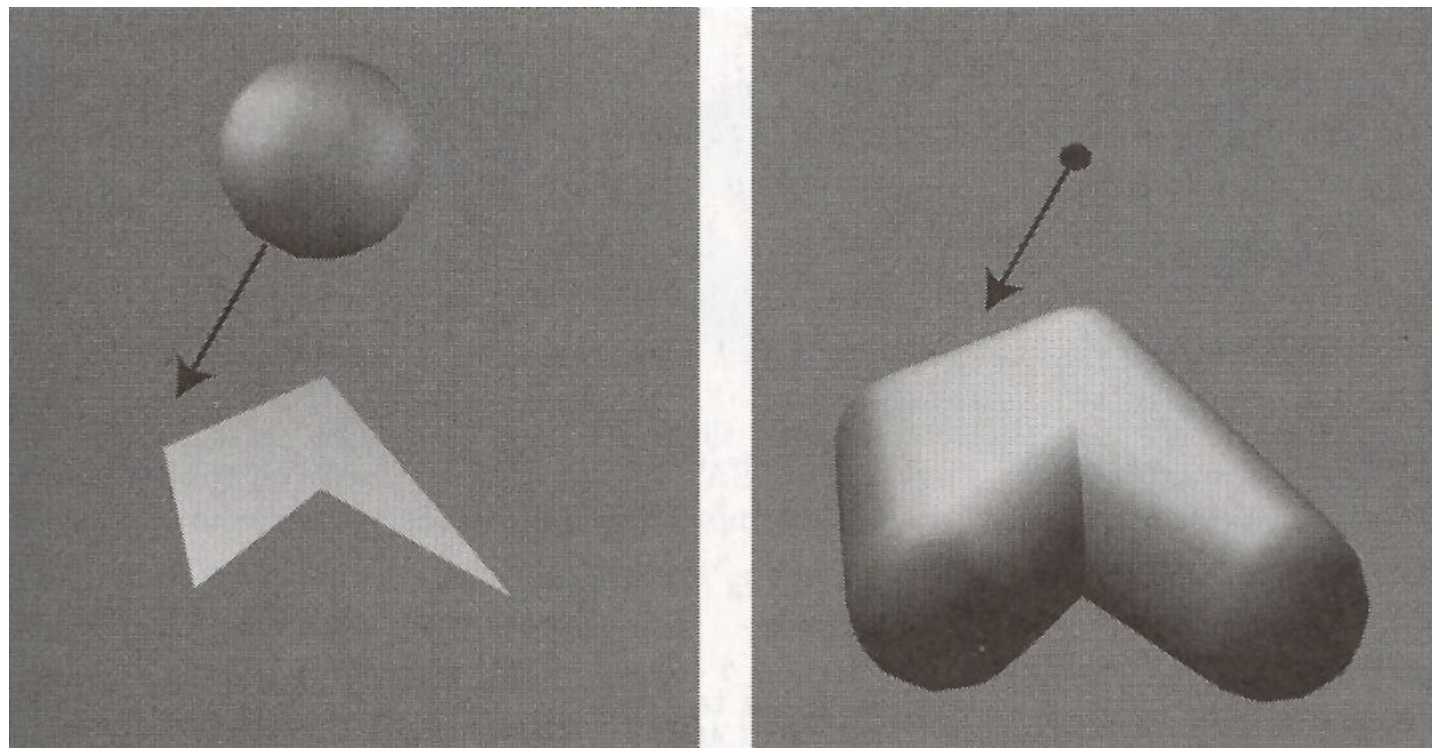


“球体/多边形” 动态相交测试

- 利用前面的平面与球体的相交方法，测试球体是否和多边形所在的平面相交，如果不相交，则结束。
- 否则，计算球体首次与平面相交的时刻，然后检查交点是否在多边形内部，如果是，则相交；否则需要进一步的检查球体是否与多边形的边或点相交。



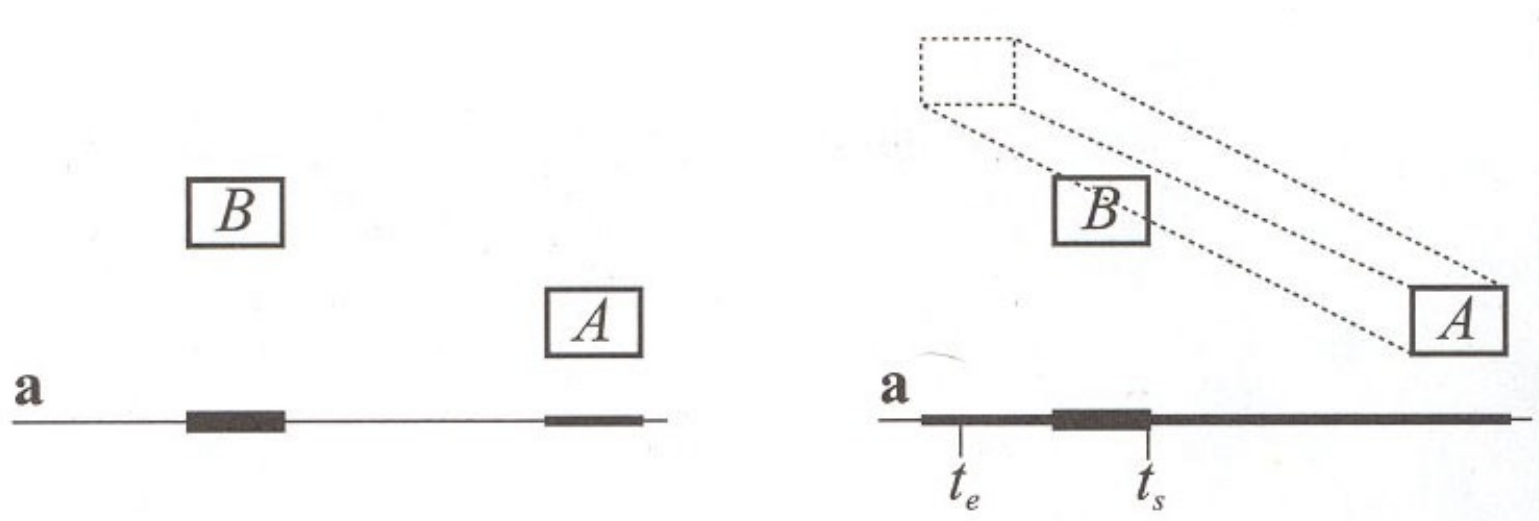
等价处理法



左图：一个球体正向一个多边形移动

右图：一条射线指向一个膨胀的多边形，这两种测试是等价的。

动态分离轴测试方法



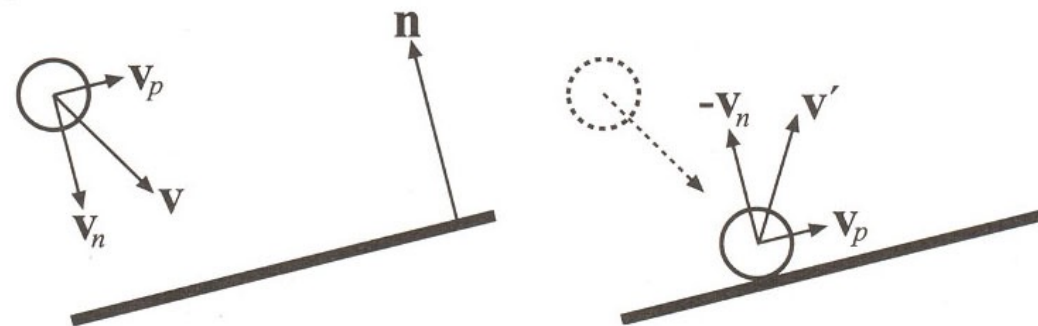
左图：对于轴 a 的分离轴定理(SAT)，其中A和B在该轴上没有重叠。

右图：对于轴 a 的SAT，其中A是移动的，在运动过程中对A和它在轴 a 上的区间投影进行跟踪，可以看出两个物体在轴 a 上是重叠的。

碰撞响应

- 碰撞响应作为一种动作，可以避免物体之间的相互贯穿。
- 球体和平面之间相互碰撞后的响应。
- 假设一个球体向一个平面运动，速度矢量为 $\mathbf{v}$ ，平面为 $\Pi: \mathbf{n} \cdot \mathbf{x} + d = 0$ ，其中 $\mathbf{n}$ 为单位化向量。
- 为了计算碰撞响应，可将速度向量表示为：
 $\mathbf{v} = \mathbf{v}_n + \mathbf{v}_p$ ，其中 $\mathbf{v}_n = (\mathbf{v} \cdot \mathbf{n})\mathbf{n}$ ， $\mathbf{v}_p = \mathbf{v} - \mathbf{v}_n$
- 得到碰撞之后的速度向量 $\mathbf{v}'$ 为

$$\mathbf{v}' = \mathbf{v}_p - \mathbf{v}_n$$



完全弹性碰撞。

对于非完全弹性碰撞，可以将 $-\mathbf{v}_n$ 长度减小。

「参考资料」

- ❑ Tomas Akenine-Möller, Eric Haines, and Naty Hoffman, **Real Time Rendering**, 1045 pages, A.K. Peters Ltd., 3rd edition, ISBN 978-1-56881-424-7, 2008
- ❑ <http://www.realtimerendering.com/>
- ❑ Ming C. Lin的主页 <https://www.cs.umd.edu/~lin/>
 - ✓ I-COLLIDE
 - ✓ RAPID
 - ✓ V-COLLIDE
 - ✓ PQP
 - ✓ H-COLLIDE
 - ✓ SWIFT
 - ✓ PIVOT
 - ✓ SWIFT++
 - ✓ DEEP



The End